U.PORTO

FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Fundamentos de Sistemas Computacionais

Licenciatura em Engenharia Informática e Computação

(L.EIC)

Setembro de 2026

Exercícios

João Canas Ferreira

António José Araújo

Bruno Lima

Esta coletânea de exercícios cobre a matéria lecionada na unidade curricular de "Fundamentos de Sistemas Computacionais", que faz parte do plano curricular da Licenciatura em Engenharia Informática e Computação da Faculdade de Engenharia da Universidade do Porto (FEUP).

Esta coletânea é derivada de edições anteriores, desenvolvidas pelos Prof. António Araújo, Prof. João Canas Ferreira, do Departamento de Engenharia Eletrotécnica e de Computadores (DEEC) da FEUP, e pelos Prof. Pedro Diniz e Prof. Bruno Lima, do Departamento de Engenharia Informática (DEI) da FEUP.

Data de revisão: 2026-10-18

Índice

I Representação de Informação e Aritmética 1

1 Representação de Informação Numérica 2
1.1 Exercícios Resolvidos 2
1.2 Exercícios Propostos 12

2 Aritmética de Vírgula Flutuante 16
2.1 Exercícios Resolvidos 16
2.2 Exercícios Propostos 22

II Sistemas Digitais 26

3 Circuitos Combinatórios 27
3.1 Exercícios Resolvidos 27
3.2 Exercícios Propostos 42

4 Circuitos Sequenciais 50
4.1 Exercícios Resolvidos 50
4.2 Exercícios Propostos 58

5 Circuitos e Organização de Memória 63
5.1 Exercícios Resolvidos 63
5.2 Exercícios Propostos 71

III Linguagem Assembly 73

6 Linguagem Assembly 74
6.1 Exercícios Resolvidos 79
6.2 Exercícios Propostos 84

IV Arquitetura de Processadores 94

7 Organização do Processador 95
7.1 Exercícios Resolvidos 95
7.2 Exercícios Propostos 99

8 Desempenho do Processador 103
8.1 Exercícios Resolvidos 103
8.2 Exercícios Propostos 108

V Soluções 112

Soluções dos Exercícios 113
1 Representação de Informação 113
2 Aritmética de Vírgula Flutuante . . 117
3 Circuitos Combinatórios 120
4 Circuitos Sequenciais 132
5 Organização de Memória 136
6 Linguagem Assembly 138
7 Organização do Processador 149
8 Desempenho de Processadores . . . 150

Parte I

Representação de Informação e Aritmética

1 Representação de Informação Numérica

1.1 Exercícios Resolvidos

▸ **Exercício 1**

Realize as conversões de base indicadas.

a) $72_{10} = ?_2 = ?_{16}$ b) $259_{10} = ?_2 = ?_{16}$ c) $1110_2 = ?_{10} = ?_{16}$

d) $100000.11_2 = ?_{10} = ?_{16}$ e) $010.110_2 = ?_{10} = ?_8$ f) $1BEEF_{16} = ?_2 = ?_8$

Solução

a) A conversão de decimal para qualquer outra base pode ser realizada dividindo sucessivamente o número decimal (e os sucessivos quocientes) pela base pretendida. Em alternativa, pode decompor-se o número decimal original numa soma ponderada das várias potências da base pretendida.

Usando o segundo método descrito acima, temos

$$\begin{aligned} 72_{10} = 64 + 8 = 2^6 + 2^3 \quad &= 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\ &= 1001000_2 \end{aligned}$$

Quanto à conversão para hexadecimal, a abordagem mais simples consiste em converter primeiro para binário e depois de binário para hexadecimal. A conversão para hexadecimal faz-se agrupando conjuntos consecutivos de 4 bits da direita (o bit menos significativo) para a esquerda (o bit mais significativo), seguida da conversão de cada grupo para o símbolo hexadecimal correspondente. Se necessário, devem acrescentar-se zeros adicionais à esquerda da representação binária, de modo que o número de bits seja múltiplo de 4.

$$\underbrace{100}_{4}\ \underbrace{1000}_{8} = 48_{16}$$

Nota: Usamos as notações 'H' e '16' indistintamente para denotar uma representação hexadecimal.

b) Use a definição de notação posicional para obter a representação decimal.

$$259_{10} = 256 + 2 + 1 = 2^8 + 2^1 + 2^0 = 100000011_2$$

$$100000011_2 = \underbrace{1}_{1}\ \underbrace{0000}_{0}\ \underbrace{0011}_{3} = 103_{16}$$

c) Novamente, aplique a definição de notação posicional para obter

$$1110_2 = 2^3 + 2^2 + 2^1 = 14_{10}$$

$$\underbrace{1110}_{\mathrm{E}} = \mathrm{E}_{16}$$

d) Parta da definição de notação posicional para obter

$$100000.11_2 = 2^5 + 2^{-1} + 2^{-2} = 32.75_{10}$$

Como o resultado é um número fracionário, a conversão de binário para hexadecimal agrupa os dígitos, 4 de cada vez, a partir do ponto fracionário.

$$100000.11_2 = \underbrace{10}_{2}\ \underbrace{0000}_{0}.\ \underbrace{1100}_{\mathrm{C}} = 20\mathrm{C}_{16}$$

e) Novamente, comece por usar a definição de notação posicional:

$$010.110_2 = 2^1 + 2^{-1} + 2^{-2} = 2.75_{10}$$

A conversão de binário para octal agrupa os dígitos, 3 de cada vez, a partir do ponto fracionário. Note que os zeros de ordem superior e de ordem inferior, mais afastados do ponto fracionário, não acrescentam valor à representação numérica, pois os pesos correspondentes são nulos.

$$010.110_2 = \underbrace{010}_{2}.\ \underbrace{110}_{6} = 2.6_8$$

f) Partindo da definição de sistema posicional, temos

$$1\mathrm{BEEF}_{16} = \underbrace{1}_{0001}\ \underbrace{\mathrm{B}}_{1011}\ \underbrace{\mathrm{E}}_{1110}\ \underbrace{\mathrm{E}}_{1110}\ \underbrace{\mathrm{F}}_{1111} = 11011111011101111_2$$

Como referido na alínea a), a abordagem mais simples para converter de hexadecimal para octal é usar a representação binária já determinada e agrupar cada 3 bits consecutivos com o símbolo octal correspondente, como se mostra abaixo.

$$1\mathrm{BEEF}_{16} = 11011111011101111_2 = \underbrace{011}_{3}\ \underbrace{011}_{3}\ \underbrace{111}_{7}\ \underbrace{011}_{3}\ \underbrace{101}_{5}\ \underbrace{111}_{7} = 337357_8$$

O grupo de 3 bits mais à esquerda foi obtido inserindo um bit ‘0’ adicional, de modo que o número total de bits fosse múltiplo de 3. O número octal resultante tem 6 dígitos octais.

▸ **Exercício 2**

Determine uma base positiva b tal que o número decimal 20_{10} seja representado na base b como 110_b.

Solução

Assumindo que o sistema de numeração é posicional e ponderado, o valor decimal de 110_b é dado por $1 \times b^2 + 1 \times b^1 + 0$. Este valor tem de ser igual a 20. As duas soluções possíveis são $b = -5$ e $b = 4$. Como se procura uma base positiva, a resposta correta é $b = 4$.

▸ **Exercício 3**

Considerando os operandos como números binários positivos, efetue as operações aritméticas indicadas abaixo.

a) $101110 + 100101$
b) $1110010 - 1101101$
c) 1001011×11001
d) $1101000 \div 100$

Solução

Usamos aqui os métodos básicos que aprendeu na escola primária para calcular em base 10.

a)

$$\begin{array}{r} 101110 \\ +\underline{100101} \\ 1010011 \end{array}$$

b)

$$\begin{array}{r} 01\,0 \\ 11\cancel{0}\cancel{0}1\cancel{0} \\ -\underline{1101101} \\ 101 \end{array}$$

c)

$$\begin{array}{r} 1001011 \\ \times\ \ 11001 \\ \hline 1001011 \\ 1001011 \\ +\underline{1001011} \\ 11101010011 \end{array}$$

d) Este exemplo de divisão tem uma particularidade. Aqui o divisor é uma potência de 2 ($100_2 = 2^2 = 4$). Neste caso o quociente da divisão pode ser obtido de forma rápida e correta deslocando para a direita (no sentido dos bits menos significativos) a representação do dividendo. Em geral, quando o divisor é 2^n, pode deslocar-se o dividendo n bits para a direita, pois isso corresponde a subtrair n ao expoente de cada potência de 2 na decomposição do dividendo. Neste exercício, $n = 2$, e portanto $1101000 \div 100 = 11010_2$.

▸ **Exercício 4**

Represente os seguintes números decimais em binário, tanto em sinal e grandeza como em complemento para 2, usando 6 bits.

a) +12 b) −12 c) −1 d) +32

Recorde que, em complemento para 2, o maior valor positivo que pode ser representado com n bits é $2^{n-1} - 1$ e o menor número (negativo) é -2^{n-1}. Por exemplo, com 6 bits, o maior valor representável é $2^{6-1} - 1 = 31$ e o menor valor é −32.

Solução

a) $12 = 8 + 4 = 1100_2$

- Sinal e grandeza: dado que o número é positivo, o bit de sinal é 0; a sua grandeza, usando 5 bits, é 01100. Assim, $12 = 001100_2$.
- Complemento para 2: a representação em complemento para 2 de um número positivo é simplesmente a representação binária do número sem sinal (representação binária natural), desde que este possa ser representado com o número de bits disponível. Portanto, $12 = 001100_2$.

b) O módulo de −12 é $12 = 1100_2$.

- Sinal e grandeza: dado que o número é negativo, o bit de sinal é 1; a sua grandeza, usando 5 bits, é 01100. Assim, $-12 = 101100_2$.
- Complemento para 2: a representação de um número negativo x em complemento para 2 pode ser obtida a partir da representação binária do seu simétrico (isto é, o número com sinal oposto) da seguinte forma. Percorre-se a representação binária de $-x$ da direita para a esquerda (isto é, do LSB para o MSB); depois de encontrar o primeiro '1', complementam-se os bits restantes. Neste exemplo, o simétrico de −12 é $12 = 001100_2$. Então, começando pela direita, encontra-se o primeiro '1' na posição 2 (em termos do seu peso numérico); invertem-se os bits seguintes para obter $-12 = 110100_2$.

c)
- Sinal e grandeza: o bit de sinal é 1, por se tratar de um número negativo; a grandeza usando 5 bits é 00001, resultando na representação $-1 = 100001_2$.
- Complemento para 2: seguindo o processo da alínea anterior, $1 = 000001_2$, e portanto $-1 = 111111_2$.

d) O número 32 representado necessita de 6 bits: $32 = 2^5 = 100000_2$.

- Sinal e grandeza: como se pode ver, já usamos 6 bits e, portanto, seria necessário um sétimo bit para incluir o sinal. Assim, o valor 32 **não pode ser** representado em sinal e grandeza usando apenas 6 bits.
- Complemento para 2: seis bits não são suficientes para representar o valor 32. Note que 100000_2 representa um número negativo (neste caso −32), pois o seu MSB é 1.

▸ **Exercício 5**

Considere a representação binária dos números $C1_{16}$ e $A7_{16}$ usando 8 bits.

a) Indique os valores decimais correspondentes, assumindo os seguintes sistemas de notação binária:

   i) binário natural (para números não negativos);

   ii) sinal e grandeza;

   iii) complemento para 2.

b) Indique a gama de valores que pode ser representada no formato de complemento para 2.

c) Assumindo uma representação em complemento para 2, calcule a sua soma e comente os resultados.

Solução

a) $C1_{16} = 11000001_2$ e $A7_{16} = 10100111_2$

   i) $11000001_2 = 2^7 + 2^6 + 2^0 = 128 + 64 + 1 = 193$

   $10100111_2 = 2^7 + 2^5 + 2^2 + 2^1 + 2^0 = 128 + 32 + 4 + 2 + 1 = 167$

   ii) $\underset{-}{\underbrace{1}}\ \underbrace{1000001}_{2^6 + 2^0} = -65$

   $\underset{-}{\underbrace{1}}\ \underbrace{0100111}_{2^5 + 7} = -39$

   iii) Ambos os valores são negativos. Assim, para determinar o seu valor, usamos o procedimento descrito no exercício anterior para determinar a sua representação em complemento para 2 e daí obter o seu valor decimal.

$$11000001_2 \xrightarrow{\text{compl. p/ 2}} 00111111_2 = 63$$

   Portanto, a representação em complemento para 2 11000001_2 denota o decimal −63.

$$10100111_2 \xrightarrow{\text{compl. p/ 2}} 01011001_2 = 89$$

   Do mesmo modo, a representação em complemento para 2 10100111_2 denota o decimal −89.

b) Com 8 bits podemos representar um total de $2^8 = 256$ números, metade dos quais são positivos (incluindo o valor zero) e a outra metade valores negativos. Esta representação com 8 bits abrange os valores inteiros no intervalo $[-128, +127]$. Ao contrário da representação em sinal e grandeza, o valor 0 tem apenas uma representação.

   Em geral, em complemento para 2, a gama de valores representáveis usando N bits é

$$[-2^{N-1}, +2^{N-1} - 1]$$

c) A adição efetua-se como se se tratasse de números sem bit de sinal (i. e., sem sinal), mas ignorando o transporte (*carry*) do bit mais significativo da representação, como se mostra abaixo. Note que esta operação tem de ter em conta o tamanho da representação em termos do número de bits usados.

$$\begin{array}{r} 11000001 \\ +\underline{10100111} \\ \not{1}01101000 \end{array}$$

O resultado, 01101000_2, é incorreto, uma vez que somámos dois números negativos e obtivemos um número positivo.

Este exemplo expôs o problema do *overflow*. Aqui, a gama de valores que pode ser representada com 8 bits foi excedida. Esta operação aritmética em decimal é: $(-63) + (-89) = -152 < -128$, ou seja, o valor desta adição não pode ser representado com 8 bits.

▸ **Exercício 6**

Dado que $A = 11001_2$ e $B = 11101_2$ estão representados em complemento para 2 usando 5 bits, calcule $A + B$ e esclareça se ocorreu *overflow* ou não.

Solução

Os cálculos aritméticos básicos são os seguintes:

$$\begin{array}{r} 11001 \\ +\underline{11101} \\ \not{1}10110 \end{array}$$

Dado que estamos a operar com representações em complemento para 2, o bit de *carry* do MSB pode ser ignorado, pois o resultado continua a ser negativo e dentro dos limites da gama de representação com 5 bits: $A + B = 10110$. Como discutido nas aulas, quando há uma diferença entre o transporte que entra e o que sai da coluna do bit de sinal desta representação em complemento para 2, ocorre um *overflow*. Quando o transporte que entra nesta coluna do bit de sinal é idêntico ao transporte que sai dela, não há *overflow* da representação.

Observamos que $A = 11001_2 = -7_{10}$ e $B = 11101_2 = -3_{10}$ e, portanto, a sua soma é $A + B = -10_{10}$, que em complemento para 2 se representa corretamente com 5 bits como 10110_2.

▸ **Exercício 7**

Considere dois números binários de 6 bits, respetivamente $M = 101100_2$ e $N = 110010_2$. Assuma que M está representado em sinal e grandeza e N em complemento para 2.

a) Represente o padrão de bits de M em hexadecimal e determine o valor de N em decimal.

b) Determine qual dos dois números tem maior valor absoluto.

c) Mostre que, usando complemento para 2, a operação $M + N$ não apresenta *overflow*.

Solução

a) A conversão para hexadecimal é imediata.

$$M = \underbrace{10}_{2}\ \underbrace{1100}_{\mathrm{C}} = 2\mathrm{C}_{16}$$

Como N está representado em complemento para 2, o seu valor é claramente negativo. Assim, $-N = 001110 = 14$ e, portanto, $N = -14$.

b) Como M está representado em sinal e grandeza, o seu valor absoluto é diretamente representado em binário pelos 5 bits menos significativos.

$$|M| = 01100 \qquad (= 12_{10})$$

Quanto a N, podemos obter o seu valor absoluto através do valor do seu complemento, *i.e.*,

$$|N| = -N = 01110 \qquad (= 14_{10})$$

Assim, entre os dois números, N é o número com maior valor absoluto.

c) Como M está representado em sinal e grandeza, é necessário convertê-lo para complemento para 2 antes de efetuar a adição.

$$-M = 001100 \xrightarrow{\text{compl. p/ 2}} M = 110100$$

Como N já está em complemento para 2, efetuamos a adição $M + N$ como se mostra abaixo:

$$\begin{array}{r} 110100 \\ +\underline{110010} \\ \not{1}100110 \end{array}$$

Tal como no exercício anterior, não se observa qualquer *overflow*. Ambos os operandos são negativos, assim como o resultado.

▸ **Exercício 8**

Os números $X = 10111100_2$ e $Y = 10100110_2$ estão representados usando 8 bits. Determine o número mínimo de bits necessário para representar corretamente o valor resultante da operação de adição aritmética $X + Y$, assumindo que os operandos estão representados usando:

a) sinal e grandeza;

b) complemento para 2.

Solução

a) Representação em sinal e grandeza:

$$X = \underbrace{1}_{S}\ \underbrace{0111100}_{M} \qquad \text{e} \qquad Y = \underbrace{1}_{S}\ \underbrace{0100110}_{M}$$

A adição $X + Y$ requer um mínimo de 7 bits.

$$\begin{array}{r} 0111100 \\ +\underline{0100110} \\ 1100010 \end{array}$$

A grandeza do resultado de $X + Y$ requer um mínimo de 7 bits e, portanto, é necessário um bit adicional para o sinal, levando a um mínimo de 8 bits para esta representação.

b) Representação em complemento para 2:

$$\begin{array}{rr} \text{X+Y:} & 10111100 \\ & +\underline{10100110} \\ & \not{1}01100010 \end{array}$$

Como estamos a somar dois números negativos, o resultado tem de ser negativo. No entanto, o cálculo mostrado acima com 8 bits produz um número positivo, revelando assim que ocorreu um *overflow*. Precisamos, portanto, de um mínimo de 9 bits.

▸ **Exercício 9**

Considere a representação em complemento para 2 de dois números X e Y, como se mostra abaixo:

$$X = 110010_2 \qquad \text{e} \qquad Y = 101011_2$$

a) Determine os valores decimais de X e Y.

b) Determine os valores de $X - Y$ e $X + Y$ e comente os resultados obtidos.

Solução

a) Os valores de X e Y são ambos negativos, pelo que determinamos os valores numéricos correspondentes calculando os respetivos complementos para 2, como se mostra abaixo:

$$-X = 001110_2 = 2^3 + 2^2 + 2^1 = 14$$

$$-Y = 010101_2 = 2^4 + 2^2 + 2^0 = 21$$

Assim, $X = -14$ e $Y = -21$.

b) O valor de $X - Y$ pode ser calculado pela adição dos valores simétricos:

$$\begin{array}{rr} X - Y = X + (-Y): & 110010 \\ & +\underline{010101} \\ & \not{1}000111 \end{array}$$

Não há *overflow*, pois os operandos têm sinais opostos. Consequentemente, o resultado numérico é correto.

$$\begin{array}{rr} X + Y: & 110010 \\ & +\underline{101011} \\ & \not{1}011101 \end{array}$$

Neste caso, há *overflow*, uma vez que os operandos têm o mesmo sinal (negativo) e o resultado mostrado acima revela um resultado positivo. Consequentemente, o resultado da operação aritmética é incorreto, pois o valor correto não pode ser representado em complemento para 2 usando apenas 6 bits.

▸ **Exercício 10**

Considere os dois números em complemento para 2 usando 8 bits, respetivamente $X = 11000001_2$, que denota o decimal −63, e $Y = 10100111_2$, que denota o decimal −89. Como visto num exercício anterior, a adição dos dois números gera uma condição de *overflow*.

a) Estenda o sinal de ambos os valores para 9 bits.

b) Mostre que, usando 9 bits, a adição $X + Y$ já não cria uma condição de *overflow*.

Solução

a) Para estender o sinal de uma dada representação, replicamos o bit de sinal (o MSB) tantas vezes quantas forem necessárias. Neste caso particular, para uma representação em complemento para 2 com 9 bits, obtemos:

$$X = 111000001_2 \text{ e } Y = 110100111_2.$$

Note que, em ambos os casos, o simétrico de cada representação com extensão de sinal permanece inalterado. Assim, $-X = 000111111_2 = 63$ e $-Y = 001011001_2 = 89$.

b) A soma $X + Y$ usando a representação em complemento para 2 com 9 bits é a seguinte:

$$\begin{array}{rr} X + Y: & 111000001 \\ & +\underline{110100111} \\ & \not{1}101101000 \end{array}$$

Como se pode ver, o resultado é também negativo, produzindo o valor numérico $101101000_2 = -152$. Não há, portanto, *overflow*.

▸ **Exercício 11**

Considere os três números em complemento para 2 usando 8 bits, respetivamente $X = 11000001_2$, que denota o decimal −63, $Y = 10100111_2$, que denota o decimal −89, e $Z = 00100111_2$, que denota o decimal +39. Use aritmética binária.

a) Calcule a soma $X + Y + Z$ somando X a Y e depois o resultado a Z.

b) Altere a ordem da soma, somando primeiro $Y + Z$ e depois o resultado a X.

c) Comente os resultados obtidos nas duas alíneas anteriores.

Solução

a) Como visto no exercício 10, $X + Y$ produzirá o valor 01101000_2 com *overflow* (o valor decimal obtido é 104, um valor positivo, enquanto o resultado correto é negativo, -152_{10}, que está fora da gama de representação).

b) Começamos por somar $Y + Z$ usando a representação em complemento para 2 com 8 bits, como se mostra abaixo, designando esta soma por W.

$$\begin{array}{rr} Y + Z: & 10100111 \\ & +\underline{00100111} \\ & 11001110 \end{array}$$

O valor $W = 11001110_2$ está correto ($-50_{10} = -89_{10} + 39_{10}$). Somando-o agora a X, obtemos o valor mostrado abaixo.

$$\begin{array}{rr} X + W: & 11000001 \\ & +\underline{11001110} \\ & \not{1}10001111 \end{array}$$

O valor resultante é -113_{10}, que está correto.

c) O que este exemplo mostra claramente é que as operações aritméticas com precisão limitada, como a adição e a subtração, não são associativas. Neste caso, $(X + Y) + Z$ levou a uma condição de *overflow* (e, portanto, a um resultado indefinido) e $X + (Y + Z)$ produziu um resultado correto, que é o resultado que teríamos obtido se tivéssemos estendido o sinal de todos os operandos para uma representação em complemento para 2 com 9 bits e calculado a soma $(X + Y) + Z$. Para uma representação em complemento para 2 com 9 bits, ambas as somas produzem o mesmo resultado correto.

1.2 Exercícios Propostos

▸ **Exercício 12**

Considere em cada alínea o número indicado à esquerda e represente-o nas bases numéricas indicadas.

a) $256_{10} = ?_2 = ?_{16}$
b) $2047_{10} = ?_2 = ?_{16}$
c) $24{,}25_{10} = ?_2 = ?_{16}$
d) $4{,}2_{10} = ?_2 = ?_{16}$
e) $10000_2 = ?_{10} = ?_{16}$
f) $100{,}001_2 = ?_{10} = ?_{16}$
g) $1E_{16} = ?_2 = ?_{10}$
h) $ABCD_{16} = ?_{10} = ?_2$
i) $AB{,}C_{16} = ?_{10} = ?_2$
j) $1110_{10} = ?_{16}$

▸ **Exercício 13**

Considere em cada alínea o número indicado à esquerda e represente-o nas bases numéricas indicadas.

a) $10{,}01_5 = ?_{10}$
b) $121{,}2_3 = ?_{10}$
c) $256_{10} = ?_3$
d) $346_{10} = ?_5$
e) $18{,}25_{10} = ?_4$
f) $14{,}2_{10} = ?_6$

▸ **Exercício 14**

Para cada alínea, determine se se trata de uma representação numérica válida e, em caso afirmativo, indique o valor decimal correspondente.

a) $10{,}51_5$
b) $1B1{,}2_{12}$
c) $3H1_{16}$
d) $18{,}25_7$
e) $H1_{18}$
f) $14{,}2_8$

▸ **Exercício 15**

Efetue as seguintes operações aritméticas binárias, considerando os operandos representados como números sem sinal, isto é, números positivos.

a) 110101 + 11001
b) 101,01 + 100,111
c) 101110 − 100101
d) 1000010 − 101101
e) 11011101 − 1100011
f) 11011101 − 11000,11
g) 1011×100

▸ **Exercício 16**

Os métodos usados para conversão de e para uma representação em base 2 podem ser aplicados a representações noutras bases inteiras (e positivas). Assumir que se pretende usar base 3.

a) Apresente a tabela da adição para base 3.

b) Apresente a tabela da multiplicação em base 3.

c) Calcule $121_3 + 211_3$ sem converter para outras bases.

d) Calcule $1201_3 - 102_3$ sem converter para outras bases.

e) Calcule $121_3 \times 20_3$ sem converter para outras bases.

▸ **Exercício 17**

Considere os números $X = 3_{10}$, $Y = 2_{10}$ e $Z = -3_{10}$ representados em sinal e grandeza usando 4 bits.

a) Represente os números em base 2.

b) Calcule $X + Y$ e $Y + Z$ usando a representação em sinal e grandeza.

c) Calcule na mesma representação $X + 6_{10}$ e comente o resultado.

▸ **Exercício 18**

Indique os valores decimais que podem ser representados com 4 bits usando as representações sinal e grandeza e complemento para 2. Em ambos os casos, indique a gama de valores possíveis que cada representação cobre, relacionando os valores extremos (maior positivo e menor negativo) com o número de bits usados na representação.

▸ **Exercício 19**

Usando 8 bits, represente em sinal e grandeza e em complemento para 2 os seguintes números decimais:

a) 18 b) 49 c) −49

d) −3 e) −100 f) 115

g) −127 h) −128

▸ **Exercício 20**

Usando representações de 8 bits, considere os números representados por $M\colon 33_{16}$ e $N\colon 33_{10}$.

a) Calcule $M + N$ em binário, assumindo que M e N são números sem sign.

b) Assumindo que os números estão representados em complemento para 2, verifique se ocorre *overflow* ao calcular $N - M$.

▸ **Exercício 21**

Dado que P e Q representam dois números binários de 8 bits codificados no formato de complemento para 2, efetue a sua adição binária e analise o resultado.

a) $P = \mathrm{DE_H}$ e $Q = \mathrm{A3_H}$ b) $P = \mathrm{8C_H}$ e $Q = \mathrm{D3_H}$ c) $P = \mathrm{8C_H}$ e $Q = 74_{\mathrm{H}}$

▸ **Exercício 22**

Considere as seguintes representações binárias: $X{:}11100011_2$ e $Y{:}01001000_2$.

a) Determine os valores decimais de X e Y, quando a representação binária é interpretada como sinal e grandeza e como complemento para 2.

b) Calcule $X + Y$ tanto para a representação em sinal e grandeza como em complemento para 2, usando 8 bits, e comente os resultados obtidos.

▸ **Exercício 23**

Considere as representações binárias $P = 1111010_2$ e $Q = 0100010_2$ usando 7 bits.

a) Represente P em hexadecimal e Q em decimal.

b) Calcule $P + Q$ e comente o resultado, assumindo as seguintes representações numéricas para P e Q:

i) binário sem sinal;

ii) complemento para 2.

▸ **Exercício 24**

Considere as representações binárias $A = 11010_2$ (usando 5 bits) e $B = 0101010_2$ (usando 7 bits).

a) Assumindo que A e B representam números sem sinal, calcule $A + B$ usando 7 bits e converta o resultado para decimal.

b) Repita a alínea anterior, assumindo que A e B representam números em sinal e grandeza.

c) Repita a alínea anterior, assumindo que A e B representam números em complemento para 2.

▸ **Exercício 25**

Considere as representações binárias $M = 4A_{16}$ e $N = A4_{16}$, que representam números em complemento para 2 usando 8 bits.

a) Converta M para binário e determine o valor decimal de N;

b) Calcule $M - N$ e $N - M$ e determine se há *overflow*.

▸ **Exercício 26**

Considere as representações binárias $S = 11001000_2$ e $T = 00010001_2$ representados em complemento para 2 com 8 bits.

a) Determine o valor decimal de S e T.

b) Represente S e T em sinal e grandeza.

c) Calcule $S + T$ em sinal e grandeza e comente o resultado.

▸ **Exercício 27**

Considere os seguintes números de 8 bits expressos nas bases indicadas: $X = 10111100_2$, $Y = 73_{10}$ e $Z = 9E_{16}$.

a) Converta a representação de Y para hexadecimal e a de Z para binário.

b) Calcule $X + Z$, considerando que X e Z estão representados em complemento para 2. Esclareça se ocorreu *overflow*.

c) Determine o maior número N, em complemento para 2, que somado a X conduz a um resultado negativo.

▸ **Exercício 28**

Suponha que precisava de calcular a expressão abaixo, onde x toma apenas valores inteiros.

$$y = x^2 - 30x + 161$$

Assuma que o valor de x está disponível como um número sem sinal de 5 bits.

a) Qual é a gama de valores de x?

b) Qual é o maior valor positivo de y?

c) Qual é o menor valor negativo de y?

d) Qual é o número mínimo de bits necessário para representar o valor de y em sinal e grandeza?

e) Represente os valores de b) e c) em complemento para 2 com o número mínimo de bits.

▸ **Exercício 29**

Dados os dois números de 8 bits em representação de complemento para 2 X:11110001_2 e Y:00001111_2, responda às seguintes questões:

a) Quais são os valores decimais de X e Y?

b) Estenda o sinal de ambos os valores para 16 bits.

c) Calcule $X + Y$ usando ambas as representações e comente os resultados.

▸ **Exercício 30**

Determinar o número mínimo de dígitos para representar 249_{10} em cada uma das bases indicadas a seguir e mostrar a respetiva representação.

a) base 2

b) base 3

c) base 5

d) base 8

2 Aritmética de Vírgula Flutuante

2.1 Exercícios Resolvidos

▸ **Exercício 1**

Considere os números reais X e Y representados no formato IEEE 754 de precisão simples, como se mostra abaixo.

$$X : C3800000_{16} \qquad Y : 00111111100000000000000000000000_2$$

a) Determine o valor do expoente verdadeiro de X.

b) Determine o sinal de $X + Y$.

c) Justifique a seguinte afirmação: para qualquer número X, $X \times Y = X$.

Solução

a)

$$X = \text{C3800000}_{16} = 1 \quad \underbrace{10000111}_{Exp_X = 135} \quad 00000000000000000000000_2$$

Uma vez que, nesta representação de vírgula flutuante, os valores do expoente representados estão em excesso de 127, temos de subtrair este excesso ao valor numérico encontrado na representação.

O valor do expoente verdadeiro de X é $Exp_X^{\text{verd}} = 135 - 127 = 8$.

b) Como $Exp_Y = 127$, $Exp_X > Exp_Y$ e, portanto, $|X| > |Y|$. Consequentemente, $Sinal_{X+Y} = Sinal_X = 1$ e, portanto, $X + Y$ é um número negativo.

c) $Exp_Y = 127$, $Exp_Y^{\text{verd}} = 0$, e todos os 23 bits da mantissa são nulos. Portanto, $Mantissa_Y = 1.0$ e concluímos que $Y = 1$, pelo que, para qualquer X, temos $X \times Y = X$.

▸ **Exercício 2**

Considere os números reais $Y = 25{,}25_{10}$ e $X = \text{BF400000}_{16}$, este último no formato IEEE 754 de precisão simples.

a) Determine a representação de Y no formato IEEE 754 de precisão simples, usando uma representação binária.

b) Calcule $X \times Y$, indicando os passos intermédios.

Solução

a) Começamos por encontrar a decomposição de Y, para facilmente obter o seu sinal, expoente e mantissa na forma normalizada.

$$Y = 25{,}25_{10} = 2^4 + 2^3 + 2^0 + 2^{-2} = 11001{,}01_2 = 1{,}100101_2 \times 2^4$$

- $Sinal_Y = 0$
- $Exp_Y = Exp_Y^{\text{verd}} + 127 = 4 + 127 = 131 = 10000011_2$
- $Mantissa_Y = 1{,}100101_2$

A representação binária resultante é apresentada abaixo, onde isolámos cada grupo de 8 bits consecutivos:

$$Y = 01000001\ 11001010\ 00000000\ 00000000_2$$

b) $X = \text{BF400000}_{16} = \underbrace{1}_{Sinal_X}\ \underbrace{01111110}_{Exp_X}\ \underbrace{100\cdots0_2}_{Mantissa_X}$

Resultando em:

- $Sinal_{X\times Y} = 1$
- $Exp_{X\times Y} = Exp_X + Exp_Y - 127 = (131 + 126) - 127 = 130 = 10000010_2$
- $Mantissa_{X\times Y}$: (em binário)

$$\begin{array}{r} 1{,}1001010\cdots0 \\ \times 1{,}1000000\cdots0 \\ \hline 0\ 0000000\cdots0 \\ \cdots \\ 11001010\cdots0 \\ 11001010\cdots0 \\ \hline 10{,}01011110 \quad \cdots \quad 0 \end{array}$$

Como a mantissa tem de ser normalizada (*i.e.*, tem de ter um bit 1 à cabeça, que, no entanto, não é mostrado na representação binária), temos de deslocar os bits para a "direita"(na direção do LSB), ajustando o expoente em conformidade (somando uma unidade ao expoente).

$$Mantissa_{X\times Y} = 10{,}01011110\cdots0_2 = 1{,}00101111_2 \times 2^1$$

Portanto, $Exp_{X\times Y} = 131_{10} = 10000011_2$, e assim

$$X \times Y = 11000001\ 10010111\ 10000000\ 00000000_2$$

▸ **Exercício 3**

Considere o número $X = \text{C1200000}_{16}$ codificado em hexadecimal. Para esta codificação, determine o valor decimal correspondente, assumindo as seguintes representações:

a) inteiro sem sinal (inteiro positivo);

b) complemento para 2;

c) formato IEEE 754 de precisão simples.

Solução

a) A codificação binária de X é:

$$\text{C1200000}_{16} = 11000001001000000000000000000000_2$$

Como número inteiro positivo, cada bit define o valor de uma potência de dois determinada pela posição do bit, que começa em zero no lado direito (LSB) e é incrementada de um por cada posição sucessiva em direção ao lado esquerdo (MSB).

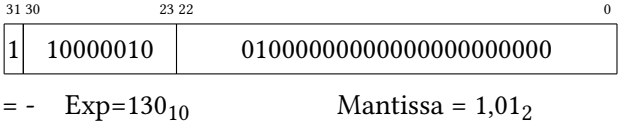
$$\begin{aligned} 11000001001000000000000000000000_2 &= 2^{31} + 2^{30} + 2^{24} + 2^{21} \\ &= 2^{21} \times (2^{10} + 2^9 + 2^3 + 2^0) \\ &= 2 \times 2^{10} \times 2^{10} \times (1024 + 512 + 9) \\ &= 2097152 \times 1545 \\ &= 3240099840_{10} \end{aligned}$$

b) $\text{C1200000}_{16} = 11000001001000000000000000000000_2$. Como representação em complemento para 2, podemos observar que se trata de um número negativo (o bit mais significativo é 1). Assim, obtemos a representação do seu valor simétrico copiando os bits da direita (LSB) para a esquerda (MSB) e invertendo todos os bits depois de encontrado o primeiro bit 1. A representação do valor simétrico, e portanto positivo, é:

$$\begin{aligned} &11000001001000000000000000000000_2 \\ &\downarrow \qquad\qquad \downarrow \\ &-01111101110000000000000000000000_2 \\ \\ &= 2^{29} + 2^{28} + 2^{27} + 2^{26} + 2^{25} + 2^{23} + 2^{22} + 2^{21} \\ &= 1054867456_{10} \end{aligned}$$

Assim, C1200000_{16} denota -1054867456_{10} em complemento para 2.

c) Para o formato de vírgula flutuante de precisão simples de 32 bits, a figura abaixo apresenta as codificações associadas a cada campo desta representação.



Sinal = - Exp=130_{10} Mantissa = $1{,}01_2$

Dado que o expoente é representado com um excesso de 127, o seu valor verdadeiro é 3. O valor numérico decimal é, portanto:

$$-1{,}01_2 \times 2^3 = -1010_2 = -10_{10}$$

▸ **Exercício 4**

Sejam X e Y dois números reais representados no formato IEEE 754 de vírgula flutuante de precisão simples, como se mostra abaixo:

$$X : 11000011000000110000000000000000_2$$
$$Y : 11000011011111000000000000000000_2$$

a) Mostre que X é um número inteiro (*i.e.*, um número real sem parte fracionária).

b) Determine o valor e a representação de um número real Z tal que $X + Z = 0$.

c) Mostre que o expoente verdadeiro de $X + Y$ é 8.

Solução

a) $X = \underbrace{1}_{-} \quad \underbrace{10000110}_{\text{Exp}_X = 134} \quad \underbrace{00000110000000000000000_2}_{\text{Mantissa}_X = 1.0000011}$

O expoente verdadeiro de X é:

$$\text{Exp}_X^{\text{verd}} = 134 - 127 = 7$$

Portanto,

$$\text{X}_{10} = -1{,}0000011_2 \times 2^7 = -10000011_2 = -131_{10}$$

b) $X + Z = 0 \quad \Leftrightarrow \quad Z = -X$

Em termos da sua representação em vírgula flutuante, temos:

$$\begin{aligned} \text{Sinal}_Z &= -\text{Sinal}_X \\ \text{Exp}_Z &= \text{Exp}_X \\ \text{Mantissa}_Z &= \text{Mantissa}_X \end{aligned}$$

Portanto, Z = 0100 0011 0000 0011 0000 0000 0000 0000_2

c) $\text{Exp}_{X+Y}^{\text{verd}} = \text{Exp}_Y^{\text{verd}} \equiv \text{Exp}_X^{\text{verd}} = 7$

Contudo, quando as mantissas são somadas, o resultado pode não estar normalizado:

$$\begin{array}{lrcl} \text{Mantissa}_X : & 1.0000011 & & \\ \text{Mantissa}_Y : & \underline{+1{,}1111100} & & \\ \text{Mantissa}_{X+Y} & 10{,}1111111 & = & 1{,}01111111_2 \times 2^1 \end{array}$$

Portanto, é necessário ajustar o expoente, neste caso incrementando-o de 1, obtendo-se:

$$\text{Exp}_{X+Y}^{\text{verd}} = 8$$

▸ **Exercício 5**

Considere os números reais X, Y e Z representados no formato IEEE 754 de vírgula flutuante de precisão simples.

a) Determine o sinal de X, dado que está representado por 23456765432_8.

b) Complete a codificação binária em falta nos campos da representação abaixo:

$$Y = +\underline{\qquad\qquad}\times 2^3 = \underbrace{\quad}_{\text{1 bit}}\ \underbrace{\qquad\qquad\qquad}_{\text{8 bits}}\ \underbrace{001010\ \ \ldots\ \ 0}_{\text{23 bits}}$$

c) Determine o valor e a representação de Z tal que $Y \times Z$ seja representado no formato IEEE 754 de precisão simples por 11000001 10010100 00000000 00000000_2.

Solução

a) Temos $X = 23456765432_8 = \underset{\downarrow}{1}\underbrace{00111001}_{\text{Exp}_X}\underbrace{01110111110101100011010}_{\text{Mantissa}_X}$

$-$

Dado que o bit de sinal é 1, o valor de X é negativo.

Note que a conversão de octal para binário criaria uma codificação com 33 bits (uma vez que temos 11 dígitos octais). No entanto, X tem de estar codificado em apenas 32 bits. Esta aparente inconsistência resolve-se facilmente ignorando o MSB do dígito octal mais significativo, considerando-o 10_2 em vez de 010_2.

b) Temos
$Y > 0 \Rightarrow \text{Sinal(Y)} = 0$
$\text{Exp}_Y^{\text{verd}} = 3 \Rightarrow \text{Exp}_Y = \text{Exp}_Y^{\text{verd}} + 127 = 130 = 10000010_2$
$\text{Mantissa}_Y = 0{,}00101 \Rightarrow \text{Mantissa}_Y = 1{,}00101_2$

resultando em: $Y = +1{,}00101_2 \times 2^3 = \underbrace{0}_{\text{1 bit}}\ \underbrace{10000010}_{\text{8 bits}}\ \underbrace{001010...0}_{\text{23 bits}}$

c) $\text{Sinal}_{Y\times Z} = - \Rightarrow \text{Sinal}_Z = -\text{Sinal}_Y = -$
$\text{Exp}_{Y\times Z} = \text{Exp}_Y + \text{Exp}_Z - 127 \Leftrightarrow 131 = 130 + \text{Exp}_Z - 127 \Leftrightarrow \text{Exp}_Z = 128$
$\text{Mantissa}_{Y\times Z} \equiv \text{Mantissa}_Y \Rightarrow \text{Mantissa}_Z = 1{,}0$

Portanto, $Z = \underset{\text{Sinal}_Z}{\underbrace{1}}\ \underbrace{10000000}_{\text{Exp}_Z}\ \underbrace{00...0}_{\text{Mantissa}_Z}$

▸ **Exercício 6**

Considere que todos os valores reais estão representados usando a norma IEEE 754 de precisão simples.

a) Determine o valor decimal de $X = 01000010000111010000000000000000_2$.

b) Dado Y com mantissa nula (*i.e.*, todos os seus 23 bits são 0), calcule:

i) A mantissa de $Z = X \times Y$;

ii) O expoente de Y, sabendo que o expoente de Z é 10000000_2.

Solução

a) $X = \underbrace{0}\ \underbrace{10000100}\ \underbrace{00111010000000000000000}$
$+ \quad \text{Exp}_X = 132 \quad \text{Mantissa}_X = 1.0011101$

O expoente verdadeiro de X é $\text{Exp}_X^{\text{verd}} = 132 - 127 = 5$ e, portanto,

$$X_{10} = +1.0011101_2 \times 2^5 = 100111.01_2 = 39.25_{10}$$

b) i) Dado que a parte fracionária da mantissa é nula (recorde que o valor normalizado da mantissa tem um bit 1 à cabeça que não é explicitamente representado), concluímos que $Mantissa_Y = 1.0$ e, portanto,

$$\text{Mantissa}_{X\times Y} = \text{Mantissa}_X \times \text{Mantissa}_Y = \text{Mantissa}_X = 1.0011101_2$$

ii) Recordar que o expoente de uma representação de vírgula flutuante como a da norma IEEE 754 pode ser calculado somando os valores dos expoentes dos operandos e subtraindo depois o excesso (ou *bias*) da representação; no caso da norma IEEE 754 de precisão simples, este valor de excesso é 127. Contudo, num produto, também temos de multiplicar as mantissas, o que pode resultar num valor inferior a $1{,}0_2$, o que exige uma normalização e, consequentemente, um ajuste do valor do expoente. Neste problema, porém, não há ajuste do expoente, uma vez que nos é dito que $\text{Significando}_Y = 1{,}0$.
Assim,

$$\begin{aligned} \text{Exp}_{X\times Y} &= \text{Exp}_X + \text{Exp}_Y - 127 \\ 128 &= 132 + \text{Exp}_Y - 127 \\ \text{Exp}_Y &= 123 \quad (\text{Exp}_Y^{\text{verd}} = -4) \end{aligned}$$

2.2 Exercícios Propostos

▸ **Exercício 7**

Considerando que $11000001101100000000000000000000_2$ representa um número no formato IEEE 754 de vírgula flutuante de precisão simples, determine o seguinte:

a) A mantissa do número;

b) O expoente do número (o valor verdadeiro e o valor na representação);

c) O valor decimal denotado pela representação.

▸ **Exercício 8**

Obtenha a representação, no formato IEEE 754 de vírgula flutuante de precisão simples, dos valores decimais indicados abaixo:

a) 31,25 b) −0,625 c) 0,0 d) 1026,5

▸ **Exercício 9**

Considere o formato IEEE 754 de vírgula flutuante de precisão simples e responda às questões abaixo. Para este problema, pode usar um programa de conversão *online*[1] para obter os números decimais pretendidos.

a) Qual é o menor número positivo (normalizado) que pode ser representado?

b) Qual é o maior número positivo (normalizado) que pode ser representado?

c) Qual é o maior número negativo (normalizado) que pode ser representado?

d) Qual é o menor número (normalizado) que pode ser representado?

▸ **Exercício 10**

Considere uma representação normalizada de vírgula flutuante com a mesma estrutura da norma IEEE 754, mas com 8 bits no total. Nesta representação, o bit mais significativo (MSB) denota o sinal, seguido de 3 bits que representam o expoente e 4 bits que representam a mantissa normalizada. Considere também que o valor numérico do expoente usa uma representação em excesso de 3.

Usando esta codificação de valores de vírgula flutuante, determine o seguinte:

a) Qual é o valor numérico denotado por 1 001 1101?

b) Qual é o valor numérico denotado por 0 101 1001?

c) Qual é a representação do valor numérico 3/8 neste formato?

[1] Por exemplo, `http://www.h-schmidt.net/FloatConverter/IEEE754.html`.

▸ **Exercício 11**

Considere os números decimais $A = 33$ e $B = -2{,}875$.

a) Represente A e B no formato IEEE 754 de vírgula flutuante de precisão simples.

b) Efetue as seguintes operações neste mesmo formato de vírgula flutuante:

i) $A + B$ ii) $B - A$ iii) $3 \times B$

c) Determine os valores resultantes em decimal e confirme a correção das operações.

▸ **Exercício 12**

Considere os dois números, A e B, representados no formato IEEE 754 de vírgula flutuante de precisão simples, como mostrado abaixo em hexadecimal.

$$A : 421\text{D}0000_{16} \qquad B : \text{C}0000000_{16}$$

Para estes números, calcule as seguintes quantidades:

a) $-B$ b) $A + B$ c) $B - A$ d) $A \times B$

▸ **Exercício 13**

Considere os dois números reais: X, representado no formato IEEE 754 de vírgula flutuante de precisão simples pela codificação hexadecimal $3\text{F}400000_{16}$, e $Y = 11{,}625_{10}$.

a) Determine a representação binária de Y no mesmo formato de vírgula flutuante.

b) Calcule $X + Y$, indicando os passos principais no cálculo do sinal, do expoente e da mantissa.

c) Confirme a correção da sua operação, indicando o valor do resultado em decimal.

▸ **Exercício 14**

Considere os dois números reais: X, representado no formato IEEE 754 de vírgula flutuante de precisão simples pela codificação hexadecimal 40400000_{16}, e $Y = -10{,}25_{10}$.

a) Determine a representação binária de Y no mesmo formato de vírgula flutuante.

b) Calcule $X + Y$, indicando os passos principais no cálculo do sinal, do expoente e da mantissa.

c) Confirme a correção da sua operação, indicando o valor do resultado em decimal.

▸ **Exercício 15**

Considere os dois números reais, X e Y, representados no formato IEEE 754 de vírgula flutuante de precisão simples pelas codificações binárias mostradas abaixo:

$$X : 11000001010000000000000000000000_2 \qquad Y : 11000001101100000000000000000000_2$$

a) Determine a representação de X e Y no formato IEEE 754 de vírgula flutuante de precisão simples, indicando os valores das suas componentes: sinal, expoente e mantissa.

b) Determine os valores decimais correspondentes.

c) Calcule $X + Y$, indicando os passos principais no cálculo do sinal, do expoente e da mantissa.

d) Confirme a correção da sua operação, indicando o valor do resultado em decimal.

▸ **Exercício 16**

Considere os dois números reais, X, representado no formato IEEE 754 de vírgula flutuante de precisão simples por $C3800000_{16}$, e $Y = 00111111100000000000000000000000_2$, respetivamente.
Escolha a afirmação correta abaixo e justifique.

a) O expoente verdadeiro de X é 8.

b) $X - Y$ é positivo.

c) O expoente verdadeiro de X é 8 e Y é negativo.

d) $X > Y$.

▸ **Exercício 17**

Em aplicações de aprendizagem computacional (*machine learning*) está a tornar-se comum o uso do formato de vírgula flutuante `Bfloat16`, que usa 8 e 7 bits para codificar o expoente e a mantissa, respetivamente. Além disso, há também o bit de sinal como MSB da representação. Nesta representação, tal como na norma IEEE 754, há arredondamento para o número mais próximo (positivo ou negativo). Adicionalmente, neste formato, todas as representações não normalizadas são consideradas como o valor zero.

a) Qual é o maior número positivo (normalizado) que pode ser representado?

b) Qual é o menor número positivo (normalizado) que pode ser representado?

c) Determine $(2{,}6 \times 10^6) \times (3{,}1 \times 10^4)$ usando a representação`Bfloat16`. Comente o resultado.

▸ **Exercício 18**

Considere o formato normalizado de vírgula flutuante de precisão simples.

a) Quantos valores do intervalo [1;2[podem ser representados desta forma? Encontre o menor e o maior intervalo entre números representáveis nesta gama.

b) Repita a alínea anterior para o intervalo [2;3[.

▸ **Exercício 19**

O grupo de trabalho P3109 do IEEE (`https://github.com/P3109/Public`) está a estudar propostas de novos formatos de representação em vírgula flutuante para utilização em aprendizagem computacional. Um dos formatos propostos, designado por «binary8p4», destina-se a ser usdo para representar pesos de redes neuronais e tem a seguinte estrutura:

- Comprimento total: 8 bits.
- Expoente: 4 bits, representação excesso-8.
- Parte fracionária do significando: 3 bits.
- Sinal: 1 bit.

A codificação segue as regras da norma IEEE 754 para números normalizados e subnormais.

a) Determinar a representação do menor número $x > 0$, do maior número $y > 0$ e do menor número normalizado $z > 0$ e os respetivos valores decimais.

b) O *erro de representação* é o valor absoluto da diferença entre o valor original e o valor representado. Assuma que ao converter de decimal para «binary8p4» se escolhe a representação correspondente ao valor mais próximo de original (método «round to nearest»). Determinar a representação de $W = 1{,}01_{10}$ e o *erro de representação* associado.

Para valores no intervalo [1; 2[, qual é o maior erro de representação possível?

c) Calcular usando diretamente a representação «binary8p4» o valor $W + W$ e determinar o erro de representação do resultado.

Parte II

Sistemas Digitais

3 Circuitos Combinatórios

3.1 Exercícios Resolvidos

▸ **Exercício 1**

Usando a álgebra de Boole e os seus teoremas, simplifique as seguintes expressões:

a) $F(A,B,C) = A\cdot B + \overline{\overline{A} + B} + A\cdot C$.

b) $F(A,B,C) = (A + B + C)\cdot(A + B + \overline{C})\cdot(\overline{A} + B + C)\cdot(\overline{A} + B + \overline{C})$.

c) $F(X,Y,Z) = X + \overline{X}\cdot Z + X\cdot\overline{Y}$.

d) $G(X,Y,Z) = X\cdot\overline{Y}\cdot Z + X\cdot\overline{Y}\cdot\overline{Z} + \overline{X}\cdot\overline{Y}\cdot\overline{Z}$.

e) $F(A,B,C) = A\cdot(\overline{B} + C) + B\cdot\overline{C}$.

Solução

a)
$$\begin{aligned}
F(A,B,C) &= A\cdot B + \overline{\overline{A} + B} + A\cdot C \\
&= A\cdot B + A\cdot\overline{B} + A\cdot C && \text{leis de De Morgan} \\
&= A\cdot(B + \overline{B}) + A\cdot C && \text{propriedade distributiva, em sentido inverso} \\
&= A + A\cdot C && (B + \overline{B}) = 1 \\
&= A\cdot(1 + C) && \text{propriedade distributiva, em sentido inverso} \\
&= A && (1 + C) = 1 \text{ e } (A\cdot 1 = A)
\end{aligned}$$

b)
$$\begin{aligned}
F(A,B,C) &= (A + B + C)\cdot(A + B + \overline{C})\cdot(\overline{A} + B + C)\cdot(\overline{A} + B + \overline{C}) \\
&= ((A + B) + (C\cdot\overline{C}))\cdot((\overline{A} + B) + (C\cdot\overline{C})) \\
&= (A + B)\cdot(C + \overline{C})\cdot(\overline{A} + B)\cdot(C + \overline{C}) \\
&= (A + B)\cdot(\overline{A} + B) \\
&= B + A\cdot\overline{A} \\
&= B
\end{aligned}$$

c)
$$\begin{aligned}
F(X,Y,Z) &= X + \overline{X}\cdot Z + X\cdot\overline{Y} \\
&= X\cdot(1 + \overline{Y}) + \overline{X}\cdot Z && \text{propriedade distributiva, em sentido inverso} \\
&= X + \overline{X}\cdot Z && (1+\text{A}) = 1 \\
&= X\cdot(Z + \overline{Z}) + \overline{X}\cdot Z && (A + \overline{A} = 1) \text{ e } (A\cdot 1 = A) \\
&= X\cdot Z + X\cdot\overline{Z} + X\cdot Z + \overline{X}\cdot Z && \text{distributividade e termos duplicados} \\
&= X\cdot(Z + \overline{Z}) + (X + \overline{X})\cdot Z && (A + \overline{A} = 1) \text{ e } (A\cdot 1 = A) \\
&= X + Z
\end{aligned}$$

d) $$\begin{aligned} G(X,Y,Z) &= X\cdot\overline{Y}\cdot Z + X\cdot\overline{Y}\cdot\overline{Z} + \overline{X}\cdot\overline{Y}\cdot\overline{Z} \\ &= X\cdot\overline{Y}(Z+\overline{Z}) + \overline{X}\cdot\overline{Y}\cdot\overline{Z} \\ &= \overline{Y}\cdot(X+\overline{X}\cdot\overline{Z}) \\ &= \overline{Y}\cdot(X\cdot(Z+\overline{Z}) + \overline{X}\cdot\overline{Z}) \\ &= \overline{Y}\cdot(X\cdot Z + X\cdot\overline{Z} + X\cdot\overline{Z} + \overline{X}\cdot\overline{Z}) \\ &= \overline{Y}\cdot(X+\overline{Z}) \\ &= X\cdot\overline{Y} + \overline{Y}\cdot\overline{Z} \end{aligned}$$

e) $$\begin{aligned} F(A,B,C) &= A\cdot(\overline{B}+C) + B\cdot\overline{C} \\ &= A\cdot(\overline{\overline{B}+C}) + B\cdot\overline{C} && (\overline{\overline{X}} = X) \\ &= A\cdot(\overline{B}\cdot\overline{\overline{C}}) + B\cdot\overline{C}\cdot(A+\overline{A}) && \text{lei de De Morgan} \\ &= A\cdot\overline{B}\cdot\overline{\overline{C}} + A\cdot B\cdot\overline{C} + B\cdot\overline{C}\cdot\overline{A} + A\cdot B\cdot\overline{C} && \text{distributividade} \\ &= A\cdot(\overline{B}\cdot\overline{\overline{C}} + B\cdot\overline{C}) + B\cdot\overline{C}\cdot(\overline{A}+A) && \text{distributividade} \\ &= A + B\cdot\overline{C} \end{aligned}$$

▸ **Exercício 2**

Obtenha a tabela de verdade de cada uma das seguintes expressões booleanas.

a) $F(A,B,C) = (A+\overline{B})\cdot(A+B+\overline{C})$

b) $F(X,Y,Z) = X\cdot\overline{Y} + \overline{X}\cdot Y\cdot\overline{Z}$

Solução

a) Uma forma expedita de obter a tabela de verdade de funções simples expressas como produtos de somas é perceber para que atribuições de valores às suas variáveis o produto será 0: se um dos termos do produto for 0, também o será necessariamente a função no seu todo. Para este caso particular, temos:

- se $A = 0$ e $B = 1$, então $F = 0$;
- se $A = 0$ e $B = 0$ e $C = 1$, então $F = 0$;
- caso contrário, $F = 1$.

Com base nestas observações, podemos facilmente deduzir as combinações de valores das variáveis de entrada para as quais as entradas da tabela de verdade de F têm de ser 0. Para as restantes combinações das variáveis de entrada, F tem o valor 1.

A	B	C	F		
0	0	0	1		
0	0	1	0	⟵	$A+B+\overline{C}$
0	1	0	0	⟵	$A+\overline{B}$
0	1	1	0	⟵	$A+\overline{B}$
1	0	0	1		
1	0	1	1		
1	1	0	1		
1	1	1	1		

b) Para funções simples expressas como soma de produtos, pode perceber-se para que atribuições de valores às suas variáveis um dos seus termos de produto será 1, pois isso conduzirá a um 1 para a função no seu todo. Para este caso particular, temos:

- se $X = 1$ e $Y = 0$, então $F = 1$;
- se $X = 0$ e $Y = 1$ e $Z = 0$, então $F = 1$;
- caso contrário, $F = 0$.

Com base nestas observações, podemos facilmente deduzir as combinações de valores das variáveis de entrada para as quais as entradas da tabela de verdade de F têm de ser 1. Para as restantes combinações das variáveis de entrada, F tem o valor 0.

X	Y	Z	F		
0	0	0	0		
0	0	1	0		
0	1	0	1	⟵	$\overline{X}\cdot Y\cdot\overline{Z}$
0	1	1	0		
1	0	0	1	⟵	$X\cdot\overline{Y}$
1	0	1	1	⟵	$X\cdot\overline{Y}$
1	1	0	0		
1	1	1	0		

▸ **Exercício 3**

Obtenha uma expressão booleana para as funções lógicas representadas pelas tabelas de verdade abaixo.

a)

X	Y	F
0	0	1
0	1	0
1	0	0
1	1	1

b)

X	Y	Z	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Solução

A partir de uma tabela de verdade podemos facilmente obter uma expressão para uma função como uma *soma de produtos* ou como um *produto de somas*, examinando as linhas da tabela onde a função é, respetivamente, 1 ou 0.

Para uma expressão na forma de *soma de produtos*, cada produto incluirá as variáveis de entrada, na sua forma não complementada ou complementada, de modo que o valor do produto seja 1 para aquela combinação particular de valores de entrada. Por exemplo, para a linha $X = 0$, $Y = 0$ e $Z = 1$, o termo $\overline{X}\cdot\overline{Y}\cdot Z$ será 1. A expressão da função no seu todo é assim simplesmente a soma de todos os termos de

produto formados desta maneira. O processo é análogo, mas dual, no caso de uma função expressa na forma de *produto de somas*, identificando agora as linhas da tabela onde a função é 0. Naturalmente, uma vez obtidas estas expressões a partir da tabela de verdade, podem aplicar-se os teoremas da álgebra booleana para as simplificar.

a) Optando por exprimir F como uma *soma de produtos*, temos:

X	Y	F		
0	0	1	$\longrightarrow$	$\overline{X}\cdot\overline{Y}$
0	1	0		
1	0	0		
1	1	1	$\longrightarrow$	$X\cdot Y$

Assim, $F(X, Y) = \overline{X}\cdot\overline{Y} + X\cdot Y = \overline{(X \oplus Y)}$.

b) Optando por exprimir G como um *produto de somas*, temos:

X	Y	Z	G		
0	0	0	1		
0	0	1	1		
0	1	0	1		
0	1	1	0	$\longrightarrow$	$X + \overline{Y} + \overline{Z}$
1	0	0	1		
1	0	1	1		
1	1	0	0	$\longrightarrow$	$\overline{X} + \overline{Y} + Z$
1	1	1	0	$\longrightarrow$	$\overline{X} + \overline{Y} + \overline{Z}$

Assim, $G(X, Y, Z) = (X + \overline{Y} + \overline{Z})\cdot(\overline{X} + \overline{Y} + Z)\cdot(\overline{X} + \overline{Y} + \overline{Z})$.

▸ **Exercício 4**

Considere a seguinte função booleana $F(X, Y, Z)$ expressa como:

$$F(X, Y, Z) = \overline{X}\cdot\overline{Y}\cdot\overline{Z} + \overline{X}\cdot\overline{Y}\cdot Z + \overline{X}\cdot\ Y\cdot\overline{Z} + X\cdot\overline{Y}\cdot\overline{Z} + X\cdot\overline{Y}\cdot Z + X\cdot Y\cdot Z$$

a) Simplifique a expressão de $F(X, Y, Z)$.

b) Obtenha a tabela de verdade de F.

c) Exprima $F(X, Y, Z)$ como um produto de somas.

d) Obtenha um circuito lógico que implemente $F(X, Y, Z)$, usando exclusivamente portas lógicas NOR.

Solução

a) $$\begin{aligned} F(X,Y,Z) &= \overline{X}\cdot\overline{Y}\cdot\overline{Z} + \overline{X}\cdot\overline{Y}\cdot Z + \overline{X}\cdot Y\cdot\overline{Z} + X\cdot\overline{Y}\cdot\overline{Z} + X\cdot\overline{Y}\cdot Z + X\cdot Y\cdot Z \\ &= \overline{X}\cdot\overline{Y}\cdot(\overline{Z}+Z) + \overline{X}\cdot Y\cdot\overline{Z} + X\cdot\overline{Y}\cdot(\overline{Z}+Z) + X\cdot Y\cdot Z \\ &= \overline{X}\cdot\overline{Y} + X\cdot\overline{Y} + \overline{X}\cdot Y\cdot\overline{Z} + X\cdot Y\cdot Z \\ &= \overline{Y} + \overline{X}\cdot Y\cdot\overline{Z} + X\cdot Y\cdot Z \\ &= \overline{Y} + \overline{X}\cdot\overline{Z} + X\cdot Z \end{aligned}$$

b) Como F está na forma de soma de produtos, cada termo indica diretamente a combinação de valores de entrada de X, Y, Z para a qual $F = 1$. Na tabela abaixo apresentamos estas entradas e os termos de produto correspondentes.

X	Y	Z	F		
0	0	0	1	⟵	$\overline{Y}$ e $\overline{X}\cdot\overline{Z}$
0	0	1	1	⟵	$\overline{Y}$
0	1	0	1	⟵	$\overline{X}\cdot\overline{Z}$
0	1	1	0		
1	0	0	1	⟵	$\overline{Y}$
1	0	1	1	⟵	$\overline{Y}$ e $X\cdot Z$
1	1	0	0		
1	1	1	1	⟵	$X\cdot Z$

c) A tabela de verdade também revela as combinações de entrada para as quais $F = 0$ e, portanto, diretamente os termos de soma para a obtenção de uma expressão como produto de somas, nomeadamente:

$$X + \overline{Y} + \overline{Z} \qquad \text{e} \qquad \overline{X} + \overline{Y} + Z$$

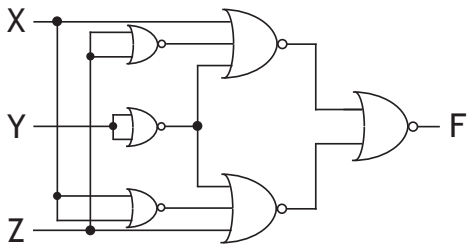
conduzindo à expressão como produto de somas apresentada abaixo.

$$F(X,Y,Z) = (X + \overline{Y} + \overline{Z})\cdot(\overline{X} + \overline{Y} + Z)$$

d) Uma função expressa como produto de somas pode ser facilmente implementada usando portas lógicas OR. Podemos trivialmente implementar uma porta lógica OR com qualquer número de entradas usando uma porta NOR com as mesmas entradas e complementando depois a sua saída com uma segunda porta NOR de 2 entradas com as duas entradas ligadas. De facto, tanto uma porta NOR como uma NAND podem servir de inversores de entrada única, pois $\overline{(A + A)} = \overline{A}$ e $\overline{(A.A)} = \overline{A}$. Assim,

$$F(X,Y,Z) = \overline{\overline{(X + \overline{Y} + \overline{Z})\cdot(\overline{X} + \overline{Y} + Z)}} = \overline{\overline{X + \overline{Y} + \overline{Z}} + \overline{\overline{X} + \overline{Y} + Z}}$$

conduzindo ao seguinte circuito lógico:



▸ **Exercício 5**

Considerando a seguinte tabela de verdade para a função $F(X, Y, Z)$, escolha a expressão booleana que corresponde à sua **soma de produtos (SOP)**:

X	Y	Z	$F(X, Y, Z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

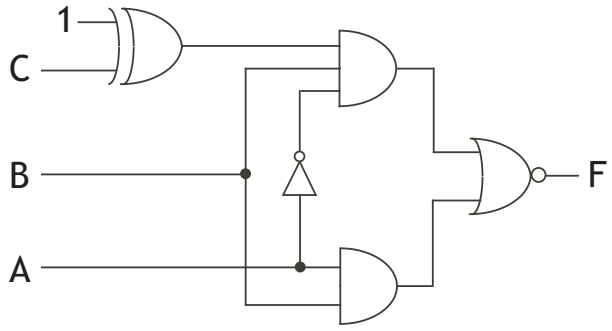
Solução

Podemos simplesmente escrever cada mintermo diretamente a partir da combinação dos valores de entrada que tornam o valor da função igual a 1. Por exemplo, o primeiro 1 corresponde ao termo mínimo $\overline{X}{\cdot}Y{\cdot}Z$, pois a atribuição $X = 0$, $Y = 1$ e $Z = 1$ resulta em $F = 1$. Assim, uma expressão booleana completa para F, expressa como soma de produtos, é:

$$F(X, Y, Z) = \overline{X} \cdot Y \cdot Z + X \cdot \overline{Y} \cdot Z + X \cdot Y \cdot \overline{Z} + X \cdot Y \cdot Z$$

▸ **Exercício 6**

A figura abaixo implementa a função lógica booleana $F(A, B, C)$



a) Obtenha uma expressão simplificada de F como produto de somas.

b) Obtenha uma expressão simplificada de F como soma de produtos.

Solução

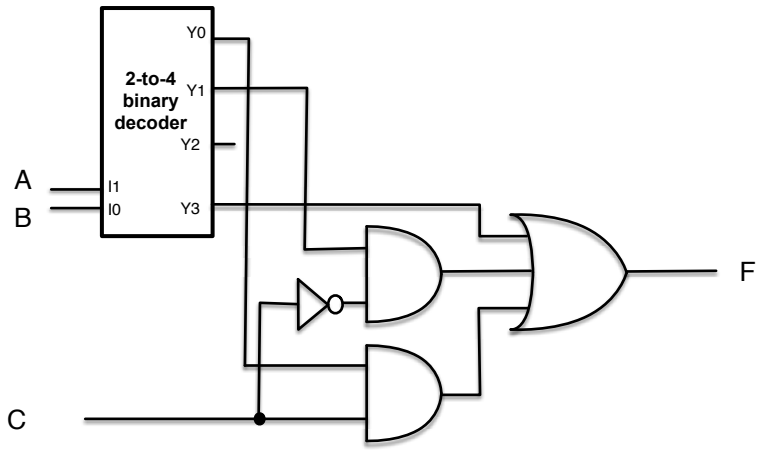
a) Uma dedução simples das expressões pode ser feita compondo a expressão booleana em cada ponto do circuito e combinando-as usando as expressões booleanas correspondentes a cada porta lógica. Aqui, fazemos a observação de que uma porta lógica XOR com uma das entradas a 1 produz na sua saída o complemento da segunda entrada. Ou seja, $(A \oplus 1) = \overline{A}$.

$$\begin{aligned} F(A,B,C) &= \overline{\overline{A}\cdot B\cdot(1 \oplus C) + A\cdot B} \\ &= \overline{\overline{A}\cdot B\cdot\overline{C} + A\cdot B} \\ &= \overline{\overline{A}\cdot B\cdot\overline{C}}\cdot\overline{A\cdot B} \\ &= (A + \overline{B} + C)\cdot(\overline{A} + \overline{B}) \end{aligned}$$

b)
$$\begin{aligned} F(A,B,C) &= (A + \overline{B} + C)\cdot(\overline{A} + \overline{B}) \\ &= A\cdot\overline{A} + A\cdot\overline{B} + \overline{A}\cdot\overline{B} + \overline{B} + \overline{A}\cdot C + \overline{B}\cdot C \\ &= (A + \overline{A} + 1 + C)\cdot\overline{B} + \overline{A}\cdot C \\ &= \overline{B} + \overline{A}\cdot C \end{aligned}$$

▸ **Exercício 7**

Considere a implementação de uma função booleana $F(A, B, C)$ pelo circuito apresentado abaixo, onde usámos, para além de portas lógicas, um descodificador binário de 2 para 4.



Solução

Na base deste problema está a descrição algébrica de cada uma das saídas do descodificador em termos das suas variáveis de entrada. Por exemplo, a saída Y_0 pode ser expressa como $\overline{AB}$. De forma semelhante, $Y_1 = \overline{A}\cdot B$, pois A está diretamente ligada à entrada mais significativa do descodificador. Aqui, quando $A = 0$ e $B = 1$, é a saída Y_1 que estará *ativa* e, portanto, assumirá o valor 1. Um raciocínio análogo permite-nos deduzir as expressões booleanas para Y_2 e Y_3. Depois, por composição usando as funções de transferência das portas lógicas, podemos obter a expressão abaixo como soma de produtos.

$$F(A,B,C) = (\overline{A}\cdot\overline{B}\cdot C) + (\overline{A}\cdot B\cdot\overline{C}) + (A\cdot B)$$

De seguida, simplificamo-la como se mostra abaixo, resultando numa expressão booleana simples na última linha.

$$\begin{aligned} F(A,B,C) &= (\overline{A}\cdot\overline{B}\cdot C) + (\overline{A}\cdot B\cdot\overline{C}) + (A\cdot B) && \\ &= \overline{A}\cdot(\overline{B}\cdot C + B\cdot\overline{C}) + (A\cdot B) && \text{distributividade, em sentido inverso} \\ &= \overline{A}\cdot(B \oplus C) + (A\cdot B) && \text{definição da função XOR} \end{aligned}$$

▸ **Exercício 8**

Pede-se que desenvolva um circuito que, dada uma entrada de 3 bits (n_2, n_1, n_0), representando um número binário positivo, detete se o seu valor está entre 2 e 5 (inclusive). Nesse caso, a saída do circuito será 1 e 0 caso contrário.

a) Defina a tabela de verdade da função $G(n_2, n_1, n_0)$.

b) Com base na tabela de verdade de G, obtenha uma soma de produtos simplificada.

Solução

a)

n_2	n_1	n_0	G
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

b)
$$\begin{aligned} G(n_2, n_1, n_0) &= (\overline{n_2}.n_1.\overline{n_0}) + (\overline{n_2}.n_1.n_0) + (n_2.\overline{n_1}.\overline{n_0}) + (n_2.\overline{n_1}.n_0) \\ &= \overline{n_2}.n_1.(n_0 + \overline{n_0}) + n_2.\overline{n_1}.(n_0 + \overline{n_0}) \\ &= \overline{n_2}.n_1 + n_2.\overline{n_1} \\ &= n_2 \oplus n_1 \end{aligned}$$

▸ **Exercício 9**

Considere a tabela de verdade da função booleana $F(A, B, C)$ apresentada abaixo.

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Implemente a função $F(A, B, C)$ conforme indicado a seguir. Desenhe os vários diagramas lógicos e explique brevemente o seu raciocínio.

a) Usar um único multiplexador 8-para-1, assumindo que as entradas A, B e C são, respetivamente, as entradas de seleção de maior a menor peso.

b) Usar um único multiplexador de 4-para-1, assumindo que as entradas A e B são, respetivamente, as entradas de seleção de maior e menor peso.

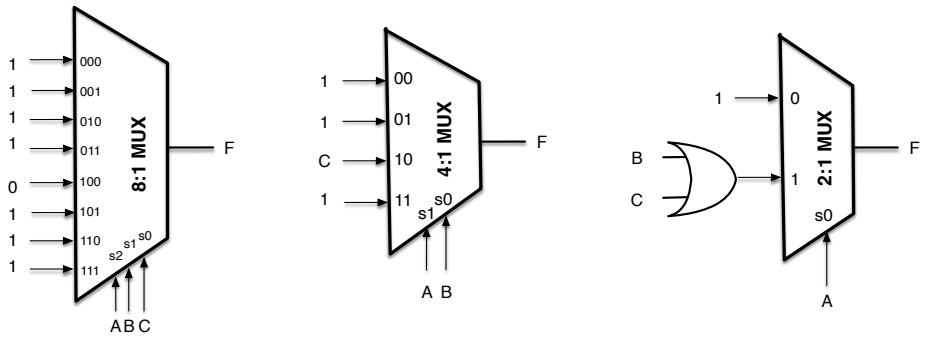
c) Usar um único multiplexador 2-para-1, assumindo que usa a variável A como entrada de seleção do multiplexador.

Solução

a) Conforme mostrado no primeiro diagrama, o circuito mapeia trivialmente os valores da tabela de verdade de F em cada entrada do multiplexador, pois cada uma delas é selecionada para cada uma das 8 combinações únicas de entrada.

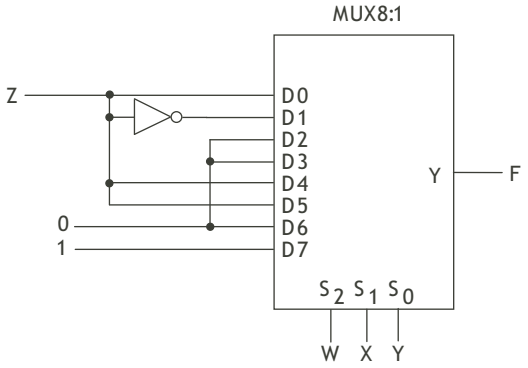
b) O segundo diagrama segue um raciocínio semelhante, mas em que agregamos cada duas linhas consecutivas da tabela de verdade, por corresponderem à mesma seleção das variáveis de entrada A e B. Neste caso, em cada entrada do multiplexador pode estar 0, 1, C ou $\overline{C}$.

c) Para a solução com um único multiplexador 2-para-1, dado que ligamos a variável de entrada A à única entrada de seleção do multiplexador, quando $A = 0$ temos que o valor de F é sempre 1. Para realizar esta função (quando $A = 0$), basta colocar a 1 a entrada do multiplexador correspondente à combinação da sua entrada de seleção ser 0 (pois $A = 0$). Quando $A = 1$, e portanto é selecionada a entrada do multiplexador correspondente a 1, temos de garantir que a função na sua entrada reflete o valor da função na secção da tabela de verdade em que $A = 1$. Neste caso, podemos ver que F é o OU lógico das duas restantes entradas, nomeadamente B e C, pois, nesta secção da tabela de verdade, F é 1 quando B ou C são 1. O terceiro diagrama reflete este raciocínio.



▸ **Exercício 10**

A figura abaixo apresenta um circuito baseado num multiplexador de 8 para 1 que implementa a função $F(W, X, Y, Z)$.



a) Obtenha a tabela de verdade de $F(W, X, Y, Z)$.

b) Represente $F(W, X, Y, Z)$ como soma de produtos (SOP).

Solução

a) Dada a associação entre W, X e Y e as entradas de seleção do multiplexador, respetivamente S_2, S_1 e S_0, podemos organizar a tabela de verdade como se mostra abaixo, onde cada duas linhas consecutivas da tabela correspondem a uma única combinação de valores de W, X e Y. Desta forma, "mapeamos"os valores da última coluna da tabela, observando a relação entre a variável Z e as várias entradas do multiplexador. Como exemplo, para as duas últimas linhas da tabela, que correspondem à combinação das entradas $W = 1$, $X = 1$ e $Y = 1$ e, portanto, à entrada `D7` do multiplexador, o valor é 1, independentemente do valor de Z.

W	X	Y	Z	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

b) A dedução de uma expressão para F pode ser feita de forma mecânica como uma soma de produtos, onde cada termo de produto é a conjunção de uma expressão que é 1 para cada uma das combinações selecionadas das variáveis do multiplexador com o valor, ou variável, na entrada correspondente do multiplexador. Como exemplo, para a combinação $W = 1$, $X = 0$ e $Y = 1$, que corresponde à entrada `D5` do multiplexador, temos de fazer a conjunção com a variável Z, conduzindo ao termo de produto $W \cdot \overline{X} \cdot Y \cdot Z$. A dedução completa e a correspondente simplificação são apresentadas abaixo.

$$
\begin{aligned}
F &= \overline{W}\cdot\overline{X}\cdot\overline{Y}\cdot Z + \overline{W}\cdot\overline{X}\cdot Y\cdot\overline{Z} + W\cdot\overline{X}\cdot\overline{Y}\cdot Z + W\cdot\overline{X}\cdot Y\cdot Z + W\cdot X\cdot Y\cdot\overline{Z} + W\cdot X\cdot Y\cdot Z \\
&= \overline{W}\cdot\overline{X}\cdot\overline{Y}\cdot Z + \overline{W}\cdot\overline{X}\cdot Y\cdot\overline{Z} + W\cdot\overline{X}\cdot Z\cdot(\overline{Y} + Y) + W\cdot X\cdot Y\cdot(\overline{Z} + Z) \\
&= \overline{W}\cdot\overline{X}\cdot\overline{Y}\cdot Z + \overline{W}\cdot\overline{X}\cdot Y\cdot\overline{Z} + W\cdot\overline{X}\cdot Z + W\cdot X\cdot Y \\
&= \overline{X}\cdot(Z\cdot(\overline{W}\cdot\overline{Y} + W) + \overline{W}\cdot Y\cdot\overline{Z}) + W\cdot X\cdot Y \\
&= \overline{X}\cdot(\overline{Y}\cdot Z + W\cdot Z + \overline{W}\cdot Y\cdot\overline{Z}) + W\cdot X\cdot Y \\
&= \overline{X}\cdot\overline{Y}\cdot Z + W\cdot\overline{X}\cdot Z + \overline{W}\cdot\overline{X}\cdot Y\cdot\overline{Z} + W\cdot X\cdot Y
\end{aligned}
$$

▸ **Exercício 11**

Considere a função booleana de 3 entradas F definida por $F = (A \oplus C)\cdot B + \overline{B}\cdot C + A\cdot C$. Implemente F usando uma estrutura lógica habitualmente designada por "And-Or-Inverter"ou AOI. Esta estrutura AOI está organizada em dois andares de portas lógicas. O primeiro andar usa exclusivamente portas AND, possivelmente com algumas das suas entradas invertidas, e o segundo andar consiste numa única porta NOR de múltiplas entradas.

Solução

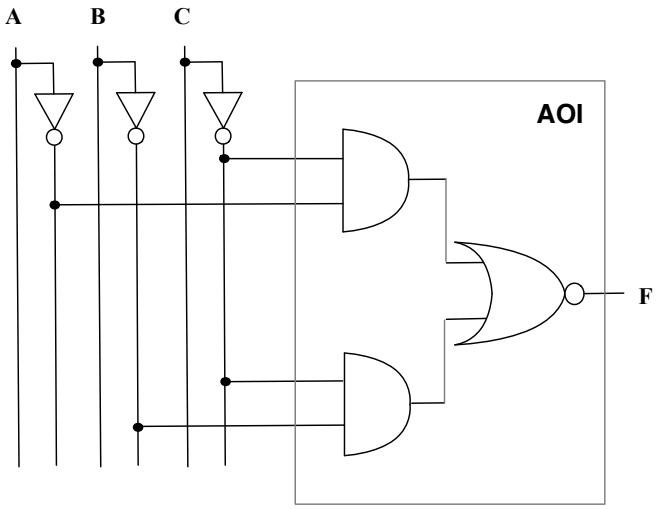
A abordagem aqui é obter uma representação em soma de produtos (SOP) do complemento de F, de modo que, quando mapeada num AOI, tenhamos os valores corretos na saída da porta NOR do andar final. Primeiro, simplificamos esta função, escrevendo-a na forma SOP.

$$\begin{aligned} F &= (A \oplus C)\cdot B + \overline{B}\cdot C + A\cdot C \\ &= A\cdot B\overline{C} + \overline{A}\cdot B\cdot C + \overline{B}\cdot C + A\cdot C \\ &= A\cdot(C + B\cdot\overline{C}) + (\overline{B} + \overline{A}\cdot B)\cdot C \\ &= A\cdot(C + B) + (\overline{B} + \overline{A})\cdot C \\ &= A\cdot B + C + \overline{B}C \\ &= A\cdot B + C \end{aligned}$$

Dada esta última representação, temos $\overline{F} = \overline{A\cdot B + C}$, que se pode simplicar como se mostra abaixo.

$$\begin{aligned} \overline{F} &= \overline{A\cdot B + C} \\ &= \overline{A\cdot B}\cdot\overline{C} \\ &= (\overline{A} + \overline{B})\cdot\overline{C} \\ &= \overline{A}\cdot\overline{C} + \overline{B}\cdot\overline{C} \end{aligned}$$

Assim, $F = \overline{\overline{A}\cdot\overline{C} + \overline{B}\cdot\overline{C}}$, que está agora diretamente expressa na forma AOI, como representado no diagrama abaixo.

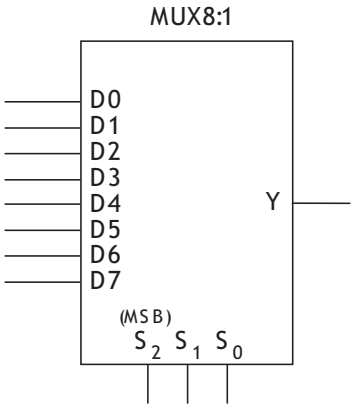


▸ **Exercício 12**

Considere a função booleana de 3 entradas F definida como $F = (A + B + C) \cdot (A + \overline{B} + \overline{C})$.

a) Obtenha a tabela de verdade de F.

b) Implemente F recorrendo a um multiplexador de 8 para 1.



Solução

a)

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

b) A implementação usando o multiplexador de 8 para 1 pode ser conseguida fazendo as seguintes atribuições às entradas de dados do multiplexador: D0 = 0, D1 = D2 = 1, D3 = 0, D4 = D5 = D6 = D7 = 1.
As entradas de seleção do multiplexador devem ser atribuídas como S2 = A, S1 = B, S0 = C e, finalmente, a saída do multiplexador $Y = F$.

▸ **Exercício 13**

Pede-se que projete um circuito lógico booleano com duas entradas de 2 bits, A e B, representando inteiros positivos denotados, respetivamente, por (a_1, a_0) e (b_1, b_0), e que produza duas saídas, X e Y. A saída X deve ser 1 se e só se $A = B$, e a saída Y deve ser 1 se e só se $A > B$.

a) Obtenha a tabela de verdade de X e Y em função de a_1, a_0, b_1 e b_0.

b) Desenvolva um circuito lógico que implemente X e Y.

c) Mostre, com o mínimo possível de lógica adicional, como poderia incluir uma terceira saída Z que seja 1 sempre que $A < B$.

d) Assumindo que tem disponíveis circuitos lógicos de comparação de 2 bits como os anteriormente desenvolvidos, mostre como os poderia combinar para desenvolver um circuito lógico que detete as mesmas relações aritméticas $A = B$, $A > B$ e $A < B$, para números inteiros positivos com 6 bits, *i.e.*, $A = (a_5 a_4 a_3 a_2 a_1 a_0)$ e $B = (b_5 b_4 b_3 b_2 b_1 b_0)$.

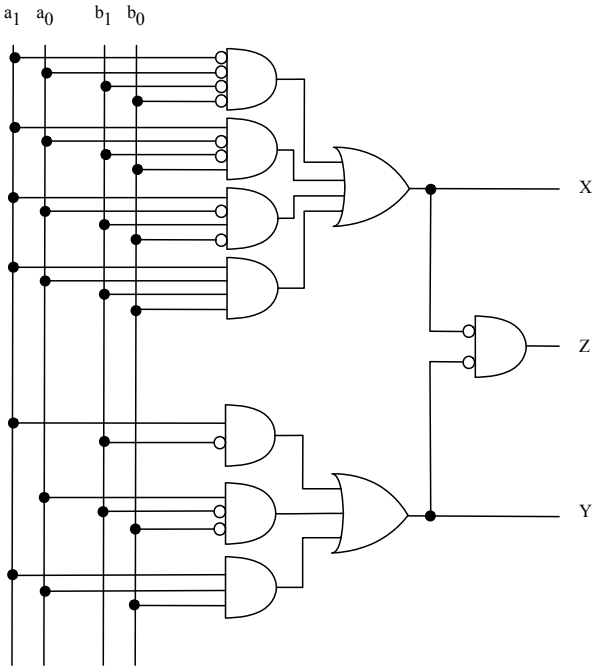
Solução

a)

a_1	a_0	b_1	b_0	X	Y
0	0	0	0	1	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	0
1	0	1	1	0	0
1	1	0	0	0	1
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	1	0

b)
$$
\begin{aligned}
Y &= \overline{a_1}\cdot a_0\cdot\overline{b_1}\cdot\overline{b_0} + a_1\cdot\overline{a_0}\cdot\overline{b_1} + a_1\cdot a_0\cdot\overline{b_1} \\
&\quad + a_1\cdot a_0\cdot b_1\cdot\overline{b_0} \\
&= \overline{a_1}\cdot a_0\cdot\overline{b_1}\cdot\overline{b_0} + a_1\cdot\overline{b_1} + a_1\cdot a_0\cdot b_1\cdot\overline{b_0} \\
&= \overline{a_1}\cdot a_0\cdot\overline{b_1}\cdot\overline{b_0} + a_1\cdot(\overline{b_1} + a_0\cdot b_1\cdot\overline{b_0}) \\
&= \overline{a_1}\cdot a_0\cdot\overline{b_1}\cdot\overline{b_0} + a_1\cdot(\overline{b_1} + a_0\cdot\overline{b_0}) \\
&= a_1\cdot\overline{b_1} + a_1\cdot a_0\cdot\overline{b_0} + \overline{a_1}\cdot a_0\cdot\overline{b_1}\cdot\overline{b_0} \\
&= a_1\cdot\overline{b_1} + a_0\cdot\overline{b_0}\cdot(a_1 + \overline{a_1}\cdot\overline{b_1}) \\
&= a_1\cdot\overline{b_1} + a_0\cdot\overline{b_0}\cdot(a_1 + \overline{b_1}) \\
&= a_1\cdot\overline{b_1} + a_0\cdot\overline{b_1}\cdot\overline{b_0} + a_1\cdot a_0\cdot\overline{b_0}
\end{aligned}
$$

A figura abaixo mostra a implementação usando portas lógicas OR e AND, onde os círculos denotam complementos lógicos (sendo, portanto, uma representação abreviada de portas inversoras).



c) Dado que $A > B$ sempre que nem $A = B$ nem $A < B$ são 1, temos que:

$$Z = \overline{X}\cdot\overline{Y}$$

d) Podemos usar três circuitos comparadores de 2 bits, combinando as saídas individuais X_2, X_1 e X_0, bem como Y_2, Y_1 e Y_0, como se mostra abaixo:

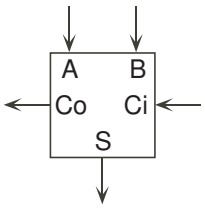
$$X = X_2 \cdot X_1 \cdot X_0 \qquad \text{e} \qquad Y = Y_2 + X_2 \cdot Y_1 + X_2 \cdot X_1 \cdot Y_0$$

onde:

- X_0 é a saída X do circuito que compara a_1, a_0 com b_1, b_0;
- X_1 é a saída X do circuito que compara a_3, a_2 com b_3, b_2;
- X_2 é a saída X do circuito que compara a_5, a_4 com b_5, b_4;
- Y_0, Y_1 e Y_2 são as saídas Y dos comparadores correspondentes.

▸ **Exercício 14**

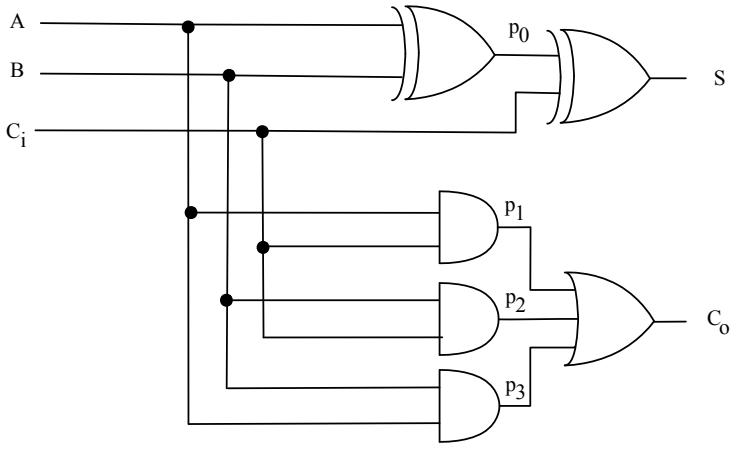
O *full-adder* (somador completo) é um circuito lógico aritmético básico e pode ser representado como se mostra na figura abaixo. Este somador tem duas entradas de dados, A e B, e duas saídas, nomeadamente a soma (*Sum*), ou S, e o transporte de saída (*Carry-Out*), ou C_o. Adicionalmente, o circuito inclui também uma terceira entrada, o transporte de entrada (*Carry-In*), ou C_i, que é usada na composição de operações com múltiplos bits como o transporte de saída das adições dos bits anteriores.



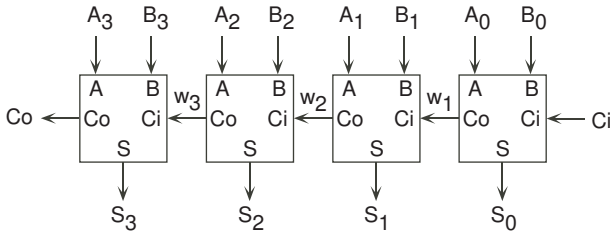
a) Com base nas noções de adição, obtenha a tabela de verdade das saídas deste circuito, nomeadamente $S(A, B, C_i)$ e $C_o(A, B, C_i)$.

b) Obtenha expressões booleanas para $S(A, B, C_i)$ e $C_o(A, B, C_i)$ na forma de soma de produtos.

c) Verifique que o circuito lógico abaixo implementa as funções lógicas $S(A, B, C_i)$ e $C_o(A, B, C_i)$.



d) Considere o somador de 4 bits representado na figura abaixo:



Determine o valor lógico de cada linha (ou fio) do circuito, assumindo que os valores lógicos de entrada são $A = A_3A_2A_1A_0 = 1010$ e $B = B_3B_2B_1B_0 = 1110$.

Solução

a)

A	B	C_i	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

b)

$$S(A,B,C_i) = \overline{A}\cdot\overline{B}\cdot C_i + \overline{A}\cdot B\cdot\overline{C_i} + A\cdot\overline{B}\cdot\overline{C_i} + A\cdot B\cdot C_i$$
$$C_o(A,B,C_i) = \overline{A}\cdot B\cdot C_i + A\cdot\overline{B}\cdot C_i + A\cdot B\cdot\overline{C_i} + A\cdot B\cdot C_i$$

c) O processo de verificação mais simples consiste em obter a tabela de verdade do circuito lógico e verificar, linha a linha, que é idêntica à tabela de verdade obtida na primeira alínea deste exercício. A partir do diagrama lógico, temos $p_0 = (A \oplus B)$, $p_1 = A\cdot C_i$, $p_2 = B\cdot C_i$, $p_3 = A\cdot B$. Neste circuito lógico, $C_o = (p_1 + p_2 + p_3)$ e $S = (C_i \oplus p_0)$.

A tabela do lado esquerdo abaixo corresponde à descrição original enquanto a tabela do lado direito corresponde ao circuito lógico descrito pelas equações. As duas últimas colunas de ambas as tabelas são idênticas (para a mesma ordem das variáveis) e, como tal, os dois são equivalentes.

A	B	C_i	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

A	B	C_i	p_0	p_1	p_2	p_3	C_o	S
0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1
0	1	0	1	0	0	0	0	1
0	1	1	1	0	1	0	1	0
1	0	0	1	0	0	0	0	1
1	0	1	1	1	0	0	1	0
1	1	0	0	0	0	1	1	0
1	1	1	0	1	1	1	1	1

d) $C_o = 1, w_3 = 1, w_2 = 1, w_1 = 0, C_i = 0, S_3 = 1, S_2 = 0, S_1 = 0$ e $S_0 = 0$.

3.2 Exercícios Propostos

▸ **Exercício 15**

Simplifique algebricamente as seguintes funções booleanas usando os teoremas da álgebra de Boole.

a) $F(A, B, C, D, E) = A \cdot B \cdot \overline{C} + \overline{C} \cdot \overline{D} \cdot E + A \cdot B + A \cdot B \cdot \overline{C} \cdot \overline{D} \cdot E + A \cdot B \cdot D \cdot \overline{E} + \overline{C} \cdot D \cdot E$.

b) $F(A, B, C) = \overline{A + A \cdot \overline{B} + \overline{A} \cdot C}$.

c) $G(A, B, C) = A \cdot B \cdot \overline{C} + A \cdot \overline{B} \cdot C + A \cdot \overline{B} \cdot \overline{C}$.

d) $F(A, B, C, D) = \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot (\overline{\overline{C} + B}) + A \cdot C \cdot \overline{D} + A \cdot \overline{B} \cdot C \cdot D$.

e) $F(W, X, Y, Z) = \overline{\overline{W} \cdot (\overline{X} + Y \cdot (Z + W))}$.

f) $F(A, B, C, D) = \overline{A} \cdot B \cdot C + \overline{B} \cdot C \cdot D + \overline{A} \cdot \overline{B} \cdot C \cdot \overline{D} + A \cdot B \cdot C \cdot D$.

▸ **Exercício 16**

Obtenha a tabela de verdade das seguintes funções booleanas.

a) $F(A, B, C) = A \cdot \overline{B} + \overline{A} \cdot \overline{C}$.

b) $G(X, Y, Z) = (X + \overline{Z}) \cdot (\overline{X} + \overline{Y} + Z)$.

c) $F(W, X, Y, Z) = \overline{W \cdot X} \cdot (\overline{\overline{Y} + \overline{Z}})$.

▸ **Exercício 17**

Para cada mapa de Karnaugh, escreva F como expressões mínimas SOP (soma de produtos) e POS (produto de somas).

a) $F(X, Y, Z)$

X \ YZ	00	01	11	10
0	1	0	1	0
1	1	0	1	1

b) $F(W, X, Y, Z)$

WX \ YZ	00	01	11	10
00	0	0	0	1
01	1	0	1	1
11	0	1	0	1
10	1	0	1	1

▸ **Exercício 18**

Usando mapas de Karnaugh, obtenha expressões SOP mínimas para as seguintes funções booleanas:

a) $F(X, Y, Z) = \overline{X} \cdot \overline{Y} \cdot Z + X \cdot \overline{Y} \cdot Z + X \cdot Y$;

b) $F(X, Y, Z) = \overline{X} \cdot (\overline{Y} \cdot \overline{Z} + Y \cdot \overline{Z}) + X \cdot (\overline{Y} \cdot Z + Y \cdot Z)$;

c) $F(W, X, Y, Z) = \overline{W} \cdot Y \cdot \overline{Z} + X \cdot Y \cdot \overline{Z} + W \cdot X + W \cdot Z + X \cdot \overline{Y} \cdot \overline{Z}$;

d) $F(W, X, Y, Z) = Y \cdot (\overline{Z} + W \cdot \overline{X} \cdot Y) + \overline{Y} \cdot \overline{Z} + \overline{W} \cdot X$.

▸ **Exercício 19**

Usando mapas de Karnaugh, obtenha expressões POS mínimas para as seguintes funções booleanas.

a) $F(X, Y, Z) = \overline{X} \cdot \overline{Y} \cdot Z + X \cdot \overline{Y} \cdot Z + X \cdot Y$;

b) $F(X, Y, Z) = \overline{X} \cdot (\overline{Y} \cdot \overline{Z} + Y \cdot \overline{Z}) + X \cdot (\overline{Y} \cdot Z + Y \cdot Z)$;

c) $F(W, X, Y, Z) = \overline{W} \cdot Y \cdot \overline{Z} + X \cdot Y \cdot \overline{Z} + W \cdot X + W \cdot Z + X \cdot \overline{Y} \cdot \overline{Z}$;

d) $F(W, X, Y, Z) = Y \cdot (\overline{Z} + W \cdot \overline{X} \cdot Y) + \overline{Y} \cdot \overline{Z} + \overline{W} \cdot X$.

▸ **Exercício 20**

Considere a função booleana $F(X, Y, Z) = X \cdot \overline{Y} + X \cdot Y \cdot \overline{Z} + \overline{X} \cdot Z$.

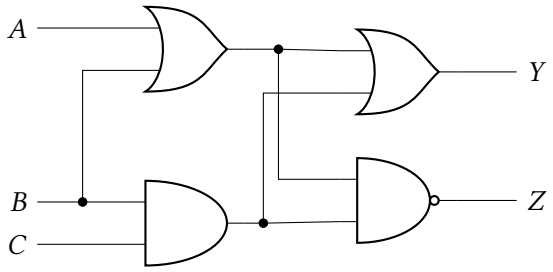
a) Obtenha a tabela de verdade correspondente.

b) Exprima $F(X, Y, Z)$ como produto de somas.

c) Desenhe o circuito lógico correspondente.

▸ **Exercício 21**

Considere o circuito lógico apresentado na figura abaixo.



a) Obtenha a expressão booleana simplificada da saída Y do circuito.

b) Mostre que Z pode ser expressa como $Z(A, B, C) = \overline{B} + \overline{C}$.

▸ **Exercício 22**

Considere a função booleana $F(A, B, C) = \overline{A + \overline{B + \overline{C}}} + \overline{\overline{A} \cdot B} \cdot C$.

a) Exprima $F(A, B, C)$ como soma de produtos simplificada.

b) Obtenha a tabela de verdade de $F(A, B, C)$.

c) Obtenha um circuito lógico que implemente $F(A, B, C)$.

▸ **Exercício 23**

Considere as seguintes funções booleanas:

$$F = \overline{X} \cdot Y + \overline{X} \cdot \overline{Y} \cdot Z \quad \text{e} \quad G = (\overline{A} + B + C) \cdot (A + \overline{B} + \overline{D}) \cdot (B + \overline{C} + \overline{D}) \cdot (A + B + C + D).$$

a) Obtenha a tabela de verdade de $F(X, Y, Z)$ e exprima-a como produto de somas (não simplificado).

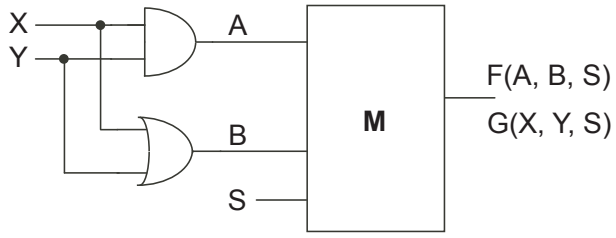
b) Obtenha a tabela de verdade de $G(A, B, C, D)$ e exprima-a como soma de produtos (não simplificada).

▸ **Exercício 24**

Considere a expressão booleana $G(A, B, C) = A \cdot \overline{C} + A \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot C$. Exprima $G(A, B, C)$ como produto de somas simplificado.

▸ **Exercício 25**

A figura abaixo apresenta um circuito lógico que implementa $G(X, Y, S)$. Para além de portas lógicas, o circuito usa um bloco M que implementa outra função lógica, $F(A, B, S)$, definida como $F = A$ se $S = 0$ e $F = B$ se $S = 1$.



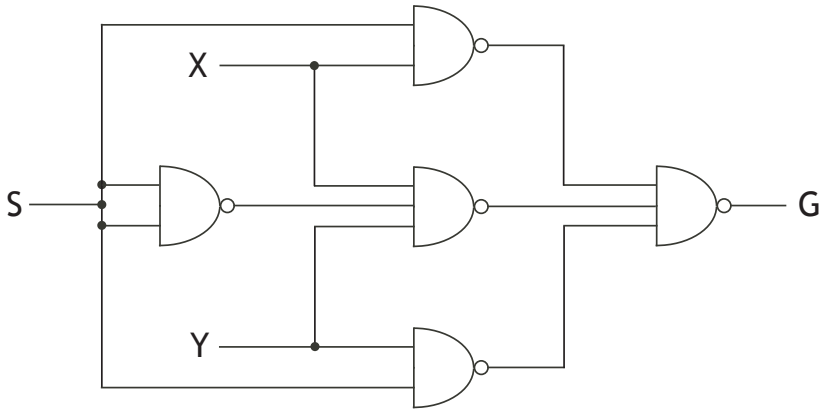
a) Obtenha a tabela de verdade de F.

b) Obtenha uma expressão algébrica simplificada de $F(A, B, S)$.

c) Mostre a implementação de F usando um multiplexador de 2 para 1.

d) Obtenha uma expressão simplificada, como soma de produtos, de $G(X, Y, S)$.

e) Mostre que o circuito abaixo, usando apenas portas lógicas NAND, implementa a função G.



▸ **Exercício 26**

Considere um circuito com uma única saída S, que apresenta o valor lógico 1 sempre que a sua entrada, um número positivo de 4 bits ($a_3a_2a_1a_0$), é maior do que 5 e múltiplo de 4.

a) Obtenha a tabela de verdade de $S(a_3, a_2, a_1, a_0)$.

b) Obtenha uma expressão para $S(a_3, a_2, a_1, a_0)$ e simplifique-a.

▸ **Exercício 27**

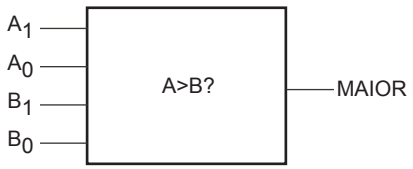
O exercício resolvido 14 implementa o circuito lógico *full-adder*, que permite somar dois valores inteiros positivos codificados em binário. Implementar agora um circuito que realize também a operação de subtração (assumindo codificação em complemento para 2).

Conceba e represente uma implementação de um circuito que possa, com base numa entrada adicional de *modo* m, efetuar a adição de dois valores de 4 bits, A e B, ou, quando $m = 1$, calcular $A - B$. O seu circuito deve usar blocos *full-adder* tanto quanto possível e o mínimo possível de portas lógicas adicionais.

Sugestão: Pode usar a entrada m para complementar diretamente B (ou não) e tirar partido do facto de o complemento para 2 se poder obter invertendo todos os bits de uma codificação binária e somando depois um 1 adicional na posição menos significativa.

▸ **Exercício 28**

Desenvolva um circuito lógico combinatório para comparar dois números inteiros positivos de 2 bits, respetivamente $A = a_1a_0$ e $B = b_1b_0$. A saída deste circuito, designada Maior, é 1 quando A é maior do que B e 0 caso contrário.

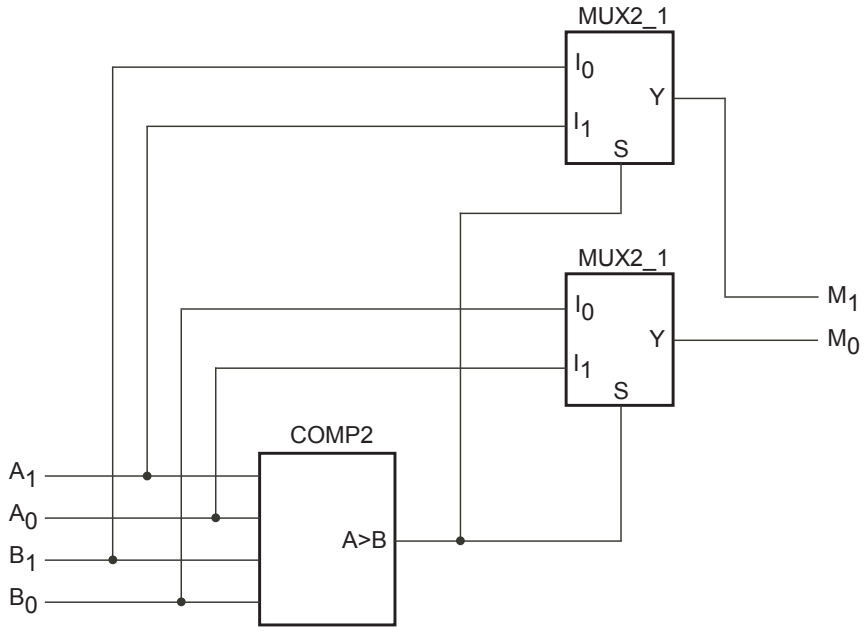


a) Obtenha a tabela de verdade desta função.

b) Obtenha uma expressão de Maior(a_1, a_0, b_1, b_0).

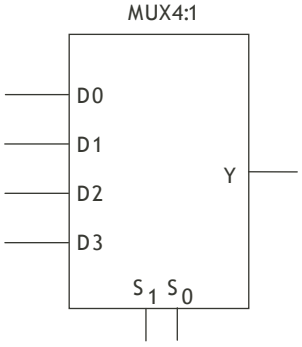
▸ **Exercício 29**

O circuito apresentado na figura abaixo inclui um comparador de 2 bits e dois multiplexadores de 2 para 1. As entradas do circuito representam dois números inteiros positivos de 2 bits cada, nomeadamente $A = a_1 a_0$ e $B = b_1 b_0$. As saídas definem também um número de 2 bits, $M = m_1 m_0$. Analise e descreva a funcionalidade do circuito.



▸ **Exercício 30**

Um multiplexador de n entradas pode ser usado para implementar uma função lógica arbitrária de n variáveis. A figura abaixo apresenta o símbolo de um multiplexador de 4 para 1, que inclui 2 entradas de seleção.



a) Mostre como implementar o produto lógico de duas variáveis com um multiplexador de 4 para 1.

b) Usando um multiplexador de 4 para 1, implemente a função lógica $F(X, Y) = X \cdot \overline{Y} + \overline{X} \cdot Y$.

c) Mostre como implementar, usando um multiplexador de 4 para 1, funções de 3 variáveis de entrada.

▸ **Exercício 31**

Descodificadores binários de n para 2^n são circuitos útieis.

a) Mostre como construir um descodificador binário de 2 para 4 usando portas lógicas simples.

b) Mostre como incluir uma entrada de ativação (*enable*) no circuito desenvolvido na alínea anterior.

c) Mostre como construir um descodificador binário de 3 para 8 (sem *enable*) usando 2 descodificadores binários de 2 para 4 com *enable*.

d) Mostre como incluir uma entrada de *enable* no circuito desenvolvido na alínea anterior.

▸ **Exercício 32**

Considere um descodificador binário de 3 para 8 com entradas $I_2 \ldots I_0$ e saídas $Q_0 \ldots Q_7$.

a) Obtenha as expressões lógicas das saídas Q_0 e Q_3 do descodificador.

b) Mostre como implementar a função booleana $F(X, Y, Z) = X \cdot Y \cdot \overline{Z} + \overline{X} \cdot Z$ usando apenas um descodificador binário de 3 para 8 e portas lógicas OR.

▸ **Exercício 33**

Pretende-se projetar um módulo lógico que calcula a diferença entre dois números não-negativos $Z = A - B$ se $A \geq B$. Cada número tem dois bits: $A(b_1 a_0)$, $B(b_1 b_0)$ e $C(c_1 c_0)$.

a) Apresente as tabelas de verdade que definem $c_0 = F_0(b_1, b_0, a_1, a_0)$ e $c_1 = F_1(b_1, b_0, a_1, a_0)$

b) Use mapas de Karnaugh para para determinar expressões mínimas de F_0 e F_1. Aproveite o facto destas funções serem incompletamente especificadas.

c) Apresente os circuitos lógicos que implementam F_0 e F_1.

d) Determine o tempo de propagação (máximo) do módulo, sabendo que os tempos de propagação das portas lógicas elementares são:

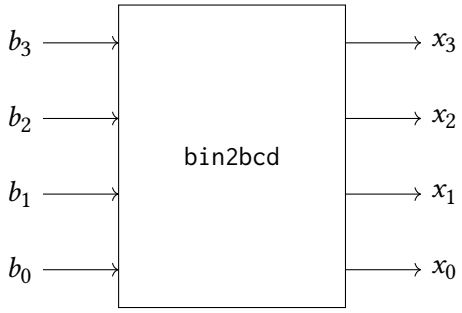
- Portas inversoras: 10 ns;
- Portas AND e OR: $N \times 10$ ns, em que N é o número de entradas da porta.

▸ **Exercício 34**

Os códigos BCD (*Binary-Coded Decimal*) são sistemas de codificação em que cada algarismo decimal (de "0"a "9") é representado individualmente pelo seu equivalente em código binário de 4 bits. O código BCD «Excess-3» está apresentado na tabela:

Dígito decimal	BCD «Excess-3»			
	x_3	x_2	x_1	x_0
"0"	0	0	1	1
"1"	0	1	0	0
"2"	0	1	0	1
"3"	0	1	1	0
"4"	0	1	1	1
"5"	1	0	0	0
"6"	1	0	0	1
"7"	1	0	1	0
"8"	1	0	1	1
"9"	1	1	0	0

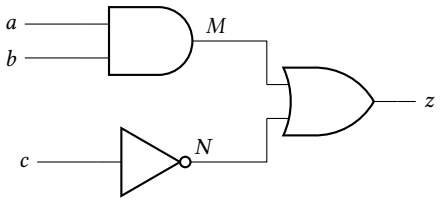
Pretende-se projetar um codificador de binário puro para BCD «Excess-3». Cada dígito decimal à entrada é representado pelo seu valor em binário ($b_3 b_2 b_1 b_0$); os valores das saídas são os indicados na tabela acima.



Aplique a abordagem do exercício 33 a este problema e apresente as expressões simplificadas de cada uma das quatro saídas (x_3, x_2, x_1, x_0) em função das entradas, bem como os respetivos circuitos. Aproveite o facto de as funções serem incompletamente especificadas.

▸ **Exercício 35**

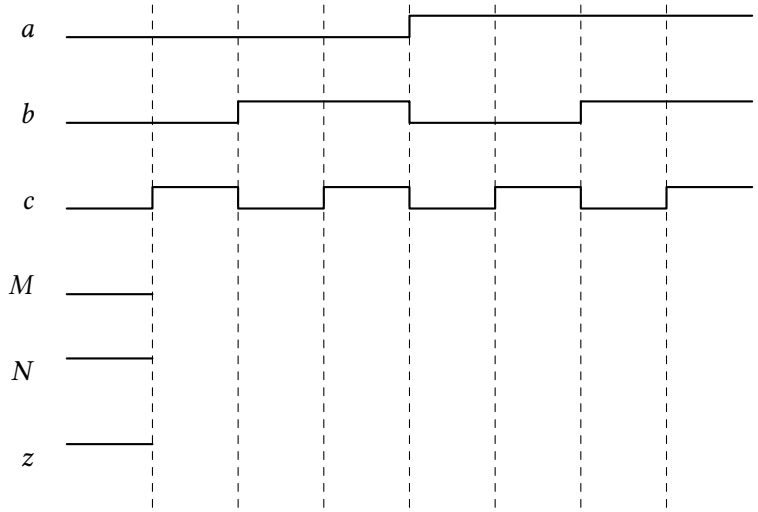
Considere o circuito lógico apresentado na figura. As entradas são a, b e C; a saída é z. Os pontos M e N indicam nós internos do circuito.



Para as formas de onda indicadas na figura seguinte, determinar as formas de onda nos pontos M, N e z do circuito assumindo que o tempo de propagação de cada porta lógica é:

a) zero;

b) um valor fixo, mas pequeno, e igual para todas as portas.

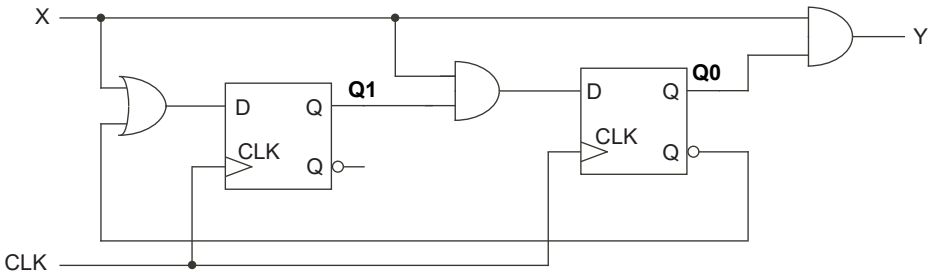


4 Circuitos Sequenciais

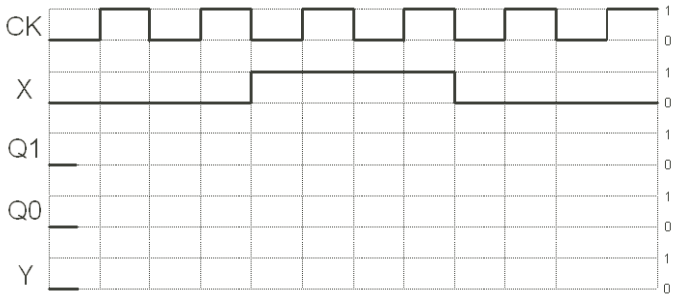
4.1 Exercícios Resolvidos

▸ **Exercício 1**

Considere o seguinte circuito digital com entrada X e saída Y.



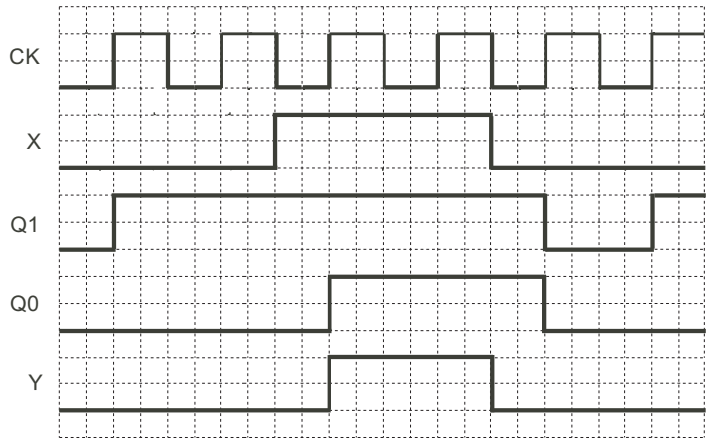
a) Desenhe o diagrama temporal das saídas dos *flip-flops* (Q_1 e Q_0), bem como da saída Y, dada a evolução temporal do sinal de entrada X. **Nota**: Considere que inicialmente Q_1 e Q_0 têm o valor lógico 0.



b) Assumindo um período de relógio de 20 ns, determine quanto tempo demora até a saída Y exibir, pela primeira vez, uma transição de 0 para 1.

Solução

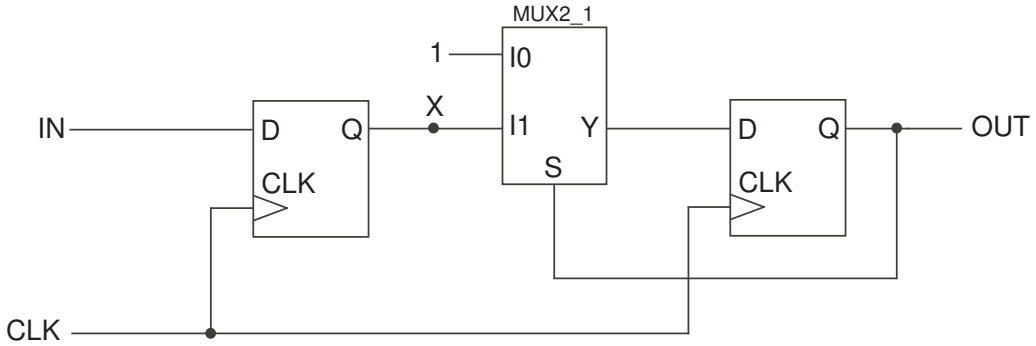
a) Como inicialmente $Q_0 = 0$ e $Q_1 = 0$, nas entradas dos *flip-flops* temos $D_0 = 0$ e $D_1 = 1$, uma vez que $X = 0$. Na primeira transição do sinal de relógio, estes valores são capturados pelas entradas dos *flip-flops* e, consequentemente, refletidos nas suas saídas. Na transição seguinte do sinal de relógio, o estado das saídas e o estado do sinal X determinam os valores dos *flip-flops*.



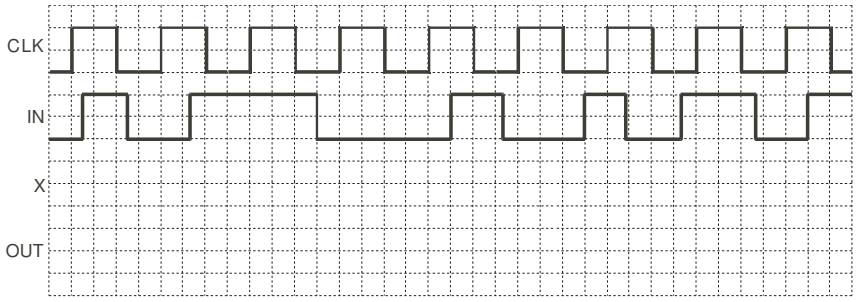
b) Considerando $T = 20\,\text{ns}$, e tendo em conta a descrição acima, o sinal Y passa de 0 para 1 ao fim de $2.5 \times T = 50\,\text{ns}$.

▸ **Exercício 2**

Considere o seguinte circuito sequencial, que inclui dois *flip-flops* ativados pelo flanco ascendente e um multiplexador. Os *flip-flops* estão inicialmente a 0.

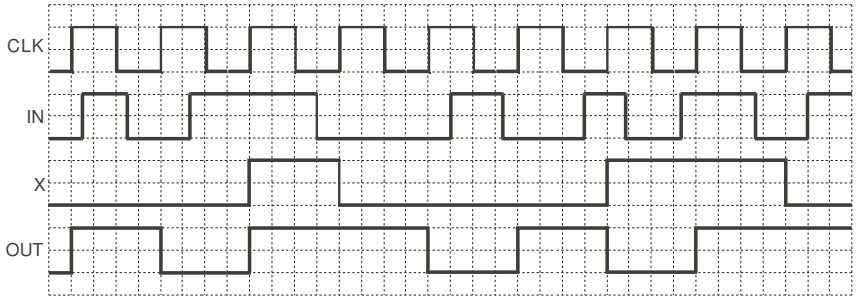


Assumindo a sequência de valores no sinal de entrada IN mostrada abaixo, obtenha as formas de onda dos sinais X e OUT.



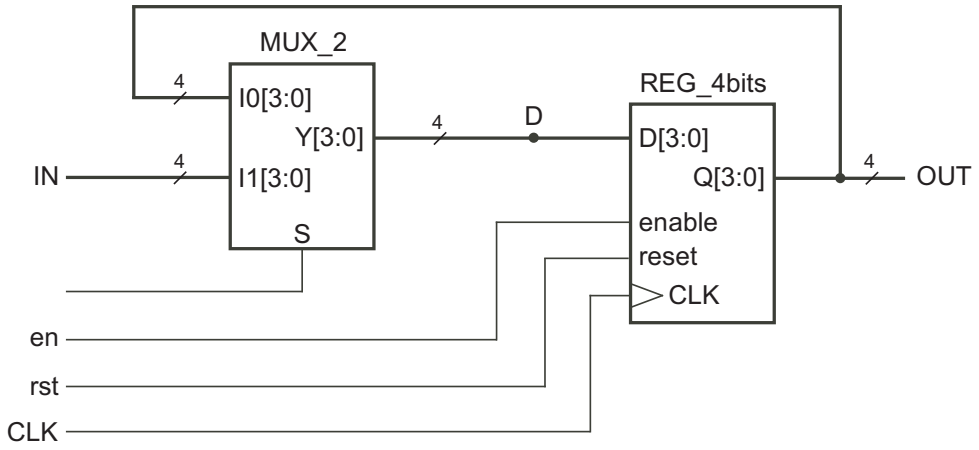
Solução

Inicialmente, X e OUT estão no valor lógico 0. A entrada de seleção do multiplexador tem o valor 0 e, como tal, o valor da sua saída reflete a entrada I_0, que é 1. Como $IN = 0$ e $Y = 1$, na primeira transição ascendente do sinal de relógio, X e OUT assumirão os valores 0 e 1, respetivamente. Uma análise de sinais semelhante conduz ao diagrama de formas de onda apresentado abaixo.



▸ **Exercício 3**

Considere o circuito abaixo, composto por um registo de 4 bits e um multiplexador com duas entradas de 4 bits e entrada de seleção S. Assuma que as entradas *enable* e *reset* do registo são ativas ao nível alto (*i.e.*, estão ativas quando estão no estado lógico 1).



De seguida, considere os valores das entradas em cada uma das transições do sinal de relógio e, dadas essas transições, determine o valor das saídas OUT após cada transição.

a)

Transição	D	en	rst
1	0110	1	0
2	0100	1	1
3	0100	0	0
4	1101	1	0

b)

Transição	IN	sel	en	rst
1	1001	1	1	0
2	1111	1	1	0
3	0101	0	1	0
4	1000	1	0	0
5	1000	1	1	0
6	1111	1	1	1
7	0101	0	1	0
8	0000	1	1	0
9	0001	1	1	0

Solução

a) Observamos que os sinais relevantes envolvem apenas o registo de 4 bits. Para cada transição do sinal de relógio, temos de ter em conta que a saída do registo assume o valor atual da entrada D se a entrada de ativação do registo estiver ativa ($en = 1$), mantendo-se depois inalterada até à transição

seguinte do sinal de relógio. Se, no entanto, a ativação estiver desativada ($en = 0$), a saída do registo mantém-se inalterada.

Quanto à entrada de *reset* do registo, sempre que esta esteja ativa ($rst = 1$), a saída do registo é colocada a 0000, independentemente dos valores das suas entradas.

Com base nestas observações, obtemos os valores dos sinais OUT apresentados na tabela abaixo.

Transição (CLK)	D	en	rst	OUT
1	0110	1	0	0110
2	0100	1	1	0000
3	0010	0	0	0000
4	1101	1	0	1101

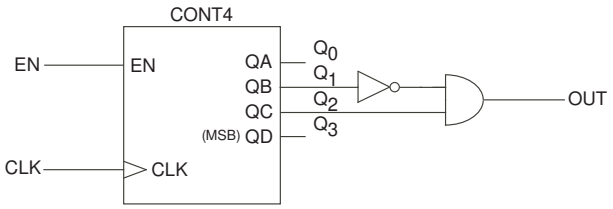
b) Para além das considerações sobre os sinais descritas acima, consideramos agora o valor da entrada do registo sempre que $sel = 1$, dado que o multiplexador pode ter como saída a saída OUT do registo sempre que $sel = 0$.

Uma análise semelhante permite-nos obter os valores dos sinais OUT apresentados na tabela abaixo.

Transição (CLK)	IN	sel	en	rst	OUT
1	1001	1	1	0	1001
2	1111	1	1	0	1111
3	0101	0	1	0	1111
4	1000	1	0	0	1111
5	1000	1	1	0	1000
6	1111	1	1	1	0000
7	0101	0	1	0	0000
8	0000	1	1	0	0000
9	0001	1	1	0	0001

▸ **Exercício 4**

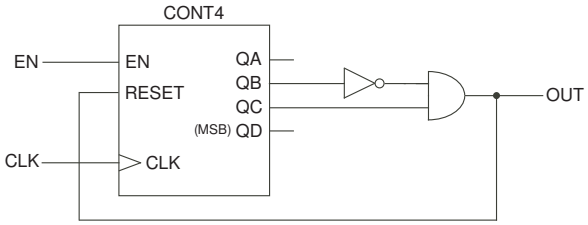
A figura abaixo apresenta um circuito sequencial baseado num contador de 4 bits.



a) Obtenha as formas de onda nas saídas do contador (Q_3, Q_2, Q_1 e Q_0) e na saída do circuito (OUT).

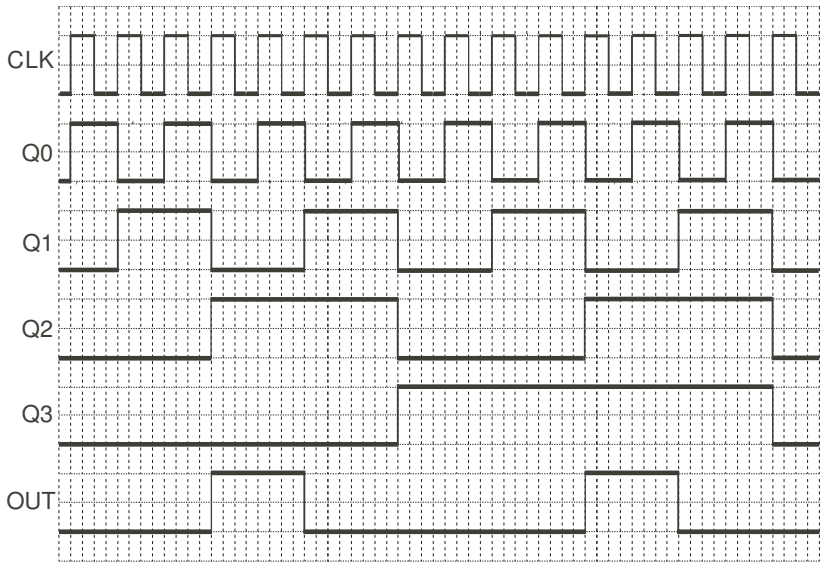
b) Indique a sequência de estados definida pelos sinais Q_3, Q_2, Q_1 e Q_0, como representações decimais destas quantidades de 4 bits.

c) Considere que o contador tem um sinal de entrada *reset* (ativo ao nível alto), como mostrado na figura abaixo. Determine a sequência de estados na saída do contador.



Solução

a) Como referido, o circuito inclui um contador de 4 bits. Considerando que inicialmente o estado do contador tem todas as saídas no valor lógico 0, em cada transição do sinal de relógio o valor do estado (refletido pelas suas saídas) é incrementado de 1. Isto resulta nas formas de onda apresentadas abaixo.



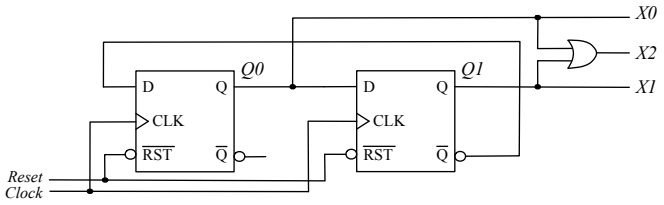
b) A sequência de valores de estado (representados em decimal a partir dos 4 bits $Q_3Q_2Q_1Q_0$):

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, \cdots$$

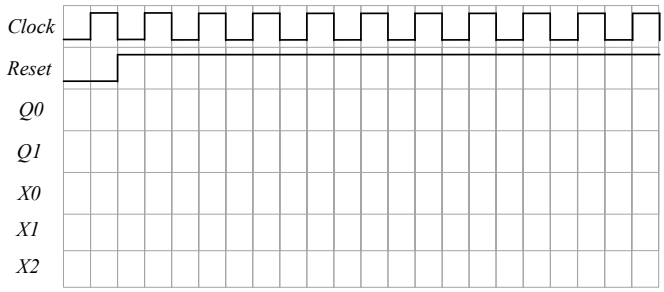
c) No estado $Q_DQ_CQ_BQ_A$ = 0100 a entrada de *reset* do contador está ativa. Na transição seguinte do sinal de relógio, o estado do contador passa a 0000. Assim, a sequência de estados do contador (em representação decimal) é $0, 1, 2, 3, 4, 0, \cdots$.

▸ **Exercício 5**

Considere o circuito apresentado no diagrama abaixo, onde os flip-flops são ativados pelo flanco ascendente e as respetivas entradas de *reset* são síncronas em relação aos flancos do sinal de relógio.



Assuma também que a evolução temporal do *clock* e do *reset* é a apresentada no diagrama temporal abaixo.



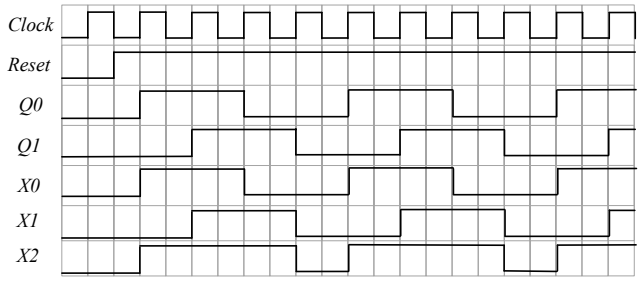
a) Determine a sequência de estados na saída do circuito sequencial, definida pelas saídas dos dois *flip-flops*, respetivamente Q_1 e Q_0.

b) Complete o diagrama temporal acima para os sinais de saída X_2, X_1 e X_0.

Solução

a) Os 2 flip-flops implementam um simples contador de 2 bits em *código de Gray*, onde, em cada flanco de relógio, apenas uma das saídas dos 2 *flip-flops* muda. Por outras palavras, o estado de reset é $Q_1, Q_0 = 00$, seguido dos estados $Q_1, Q_0 = 01$, $Q_1, Q_0 = 11$ e $Q_1, Q_0 = 10$. Após este último estado, o ciclo de estados repete-se, começando no estado $Q_1, Q_0 = 00$.

b) Com base na descrição acima, na evolução das saídas dos 2 *flip-flops* Q_1 e Q_0 e na lógica simples do circuito, podemos rapidamente obter as formas de onda das saídas do circuito X_2, X_1 e X_0, como se mostra no diagrama temporal abaixo.



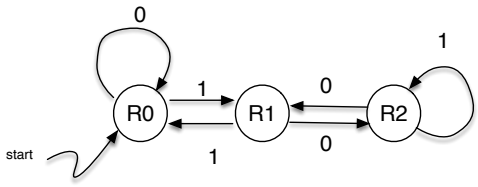
▸ **Exercício 6**

Projete uma máquina de estados finitos (FSM) para um circuito sequencial com uma única entrada e uma única saída, capaz de reconhecer o conjunto de todas as sequências que começam por 1 e que, interpretadas como a representação binária de um inteiro (assumindo que o último dígito a ser processado é o menos significativo), são congruentes com zero módulo 3, *i.e.*, o valor numérico desta representação binária é múltiplo de 3. Sempre que a entrada lida até ao momento tiver a propriedade descrita acima, a FSM deve apresentar imediatamente o valor 1 na saída, independentemente do valor no final da leitura da sequência de entrada. Tente desenvolver uma FSM com o número mínimo de estados.

Use a mesma abordagem para o problema em que a entrada é congruente com zero módulo 5.

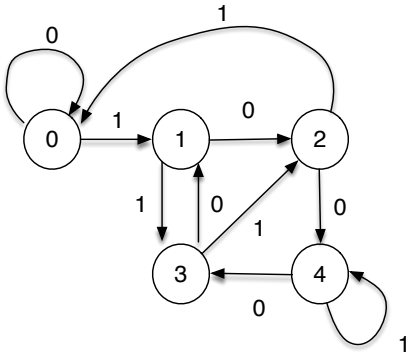
Solução

A parte difícil deste problema é que é necessário acompanhar, com os bits já observados, qual é o resto da divisão por 3. Tendo o resto, não são necessários mais do que 2 estados adicionais, um para cada um dos valores de resto 1 e 2, sendo o estado que representa um resto de zero o estado de aceitação, neste caso, digamos, um estado designado $S3$. A FSM abaixo consegue isto. Pode verificar esta FSM experimentando o número 12 em binário, 1100, ou o 21 em binário, 10101. Note que, no último estado, $S3$, qualquer 0 adicional significa que se está a deslocar os bits uma posição, *i.e.*, a multiplicar por 2, permanecendo assim no mesmo estado.



State	Output	Encoding (2 FF)
R0	1	00
R1	0	01
R2	0	10

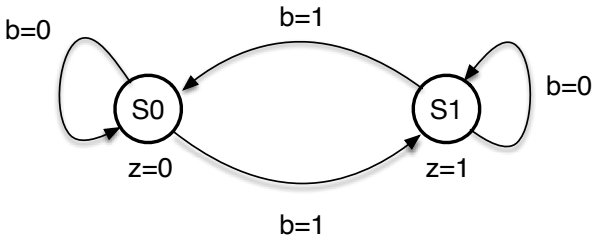
Para o caso do projeto da FSM congruente com zero módulo 5, a figura abaixo apresenta uma solução possível, onde indicámos uma possível codificação binária dos estados usando 3 FFs.



State	Output	Encoding (3 FFs)
0	1	000
1	0	001
2	0	010
3	0	011
4	0	100

▸ **Exercício 7**

A figura abaixo apresenta uma máquina de estados finitos (FSM) simples, com uma única entrada b e uma única saída z. Esta FSM tem dois estados, pelo que pode ser implementada como uma máquina de Moore com um único flip-flop (FF) do tipo D.



Nas condições acima, obtenha a função de transição de estados e a função de saída e apresente a implementação do circuito lógico usando um único FF do tipo D ativado por flanco como implementação do registo de estado, num estilo de implementação de máquina de Moore.

Solução

A implementação como máquina de Moore é bastante simples, pois, com 2 estados, precisamos efetivamente de apenas um FF do tipo D. Estado $S0 \mapsto Q = 0$ e estado $S1 \mapsto Q = 1$. Com esta codificação binária dos estados, podemos obter a função de transição de estados e a função de saída como se mostra abaixo:

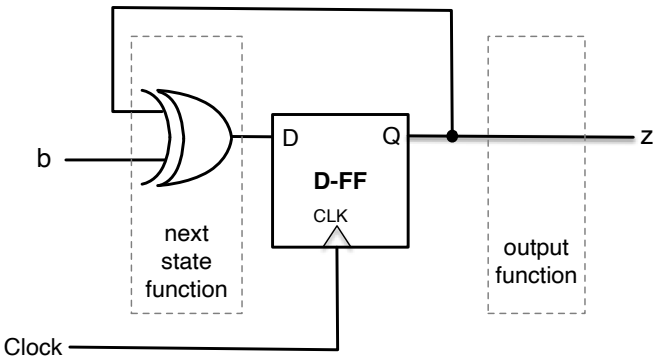
b	Q	D
0	0	0
0	1	1
1	0	1
1	1	0

State Transition Function

Q	z
0	0
1	1

Output Function

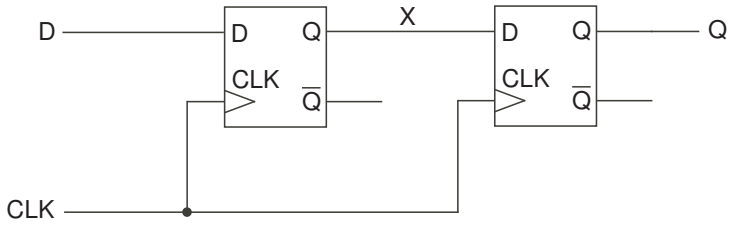
Com base nestas deduções, o circuito lógico resultante é o apresentado abaixo.



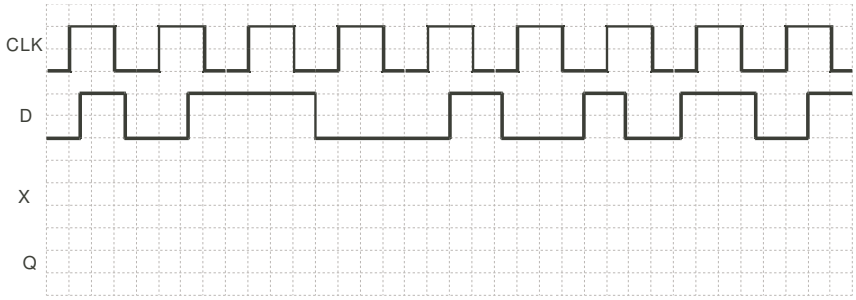
4.2 Exercícios Propostos

▸ **Exercício 8**

Considere o circuito apresentado abaixo, com *flip-flops* do tipo D ativados pelo flanco ascendente, com as saídas inicialmente a 0.

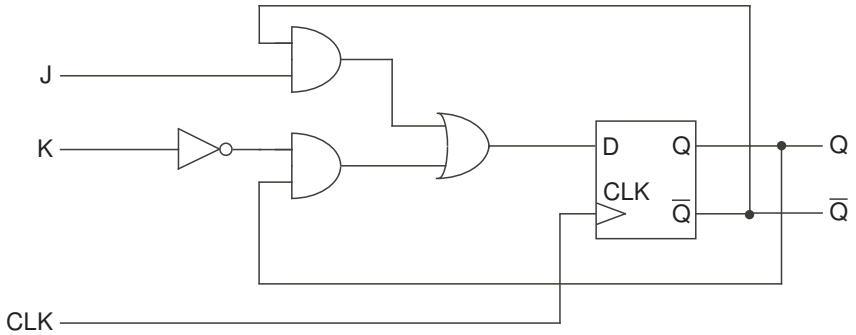


Desenhe a forma de onda da saída Q dada a entrada D.
Sugestão: comece por verificar o valor de X após cada transição do sinal de relógio.



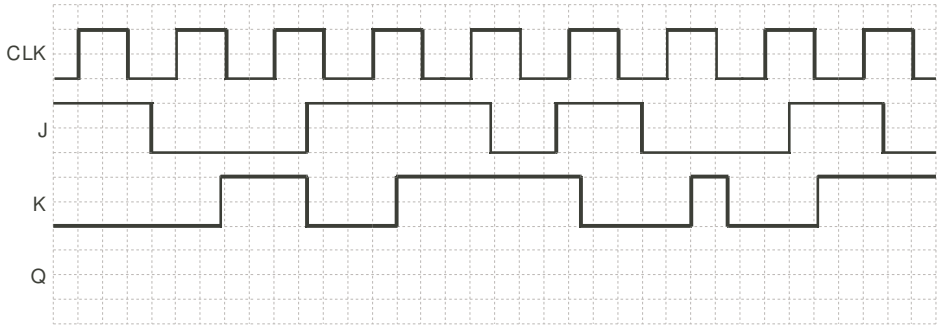
▸ **Exercício 9**

Considere o circuito sequencial abaixo, que inclui um *flip-flop* do tipo D ativado pelo flanco ascendente, inicialmente no estado com $Q = 0$.



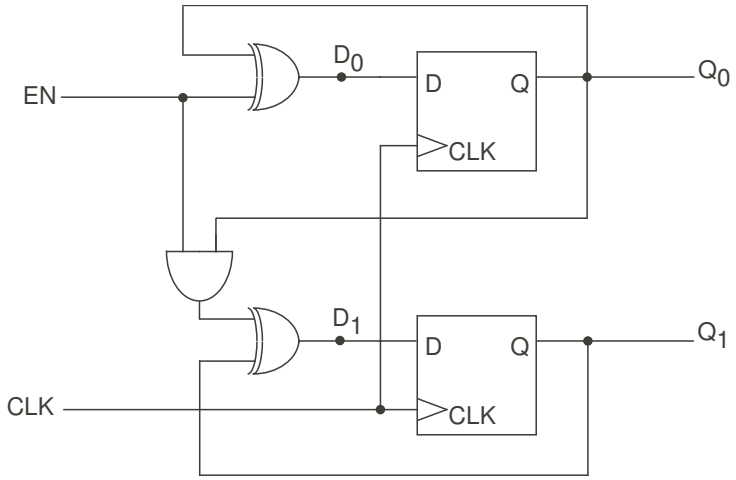
a) Obtenha a expressão lógica de $D(J, K, Q)$ na entrada D do *flip-flop*.

b) Considerando os valores das entradas J e K apresentados na figura abaixo, obtenha o valor lógico do sinal de saída Q do circuito.



▸ **Exercício 10**

Considere o circuito sequencial apresentado abaixo.



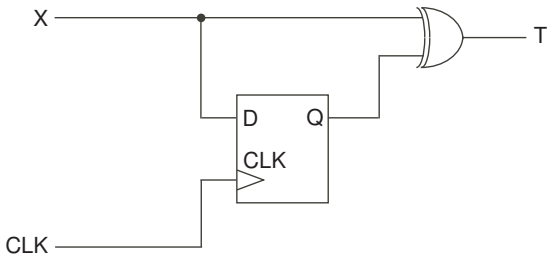
a) Obtenha as expressões lógicas das entradas D_0 e D_1 de ambos os *flip-flops* do tipo D.

b) Obtenha as formas de onda correspondentes aos sinais Q_0 e Q_1, assumindo que o estado inicial dos *flip-flops* é $Q_0, Q_1 = 00$ e que $EN = 1$.

c) Apresente o estado do circuito (em termos dos estados dos *flip-flops* e dos sinais relevantes) após 4 transições consecutivas do sinal de relógio (CLK).

▸ **Exercício 11**

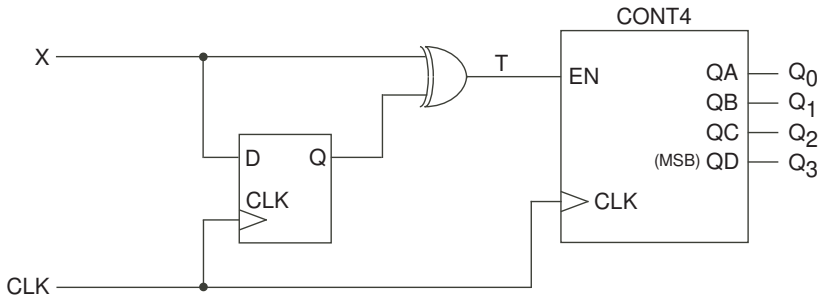
Considere o circuito abaixo, com um *flip-flop* do tipo D e uma porta XOR (OU exclusivo).



a) Assumindo que o estado inicial do *flip-flop* é 0, determine a sequência de valores lógicos da saída T se a entrada X exibir a sequência 0110001001111100 (um bit por cada ciclo de relógio).

b) Descreva a função deste circuito, relacionando os sinais T e X.

c) A este circuito acrescentamos um contador síncrono de 4 bits, como mostrado na figura abaixo. Descreva a relação entre as saídas do contador, $Q_3Q_2Q_1Q_0$, e a entrada X do circuito.

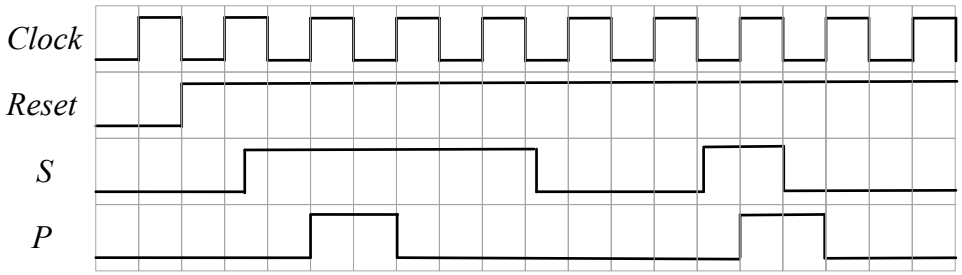


▸ **Exercício 12**

Em muitas aplicações de controlo digital, pode ser vantajoso adotar uma codificação de estados *one-hot*, em que uma máquina de estados de N bits usa N *flip-flops* e cada estado individual é representado tendo apenas um dos N *flip-flops* no estado 1 e os restantes *flip-flops* no estado 0. A transição entre estados consegue-se "passando"o 1 entre *flip-flops*, de tal forma que apenas um dos N *flip-flops* da "cadeia"retém o estado 1. Embora isto possa ser visto como um desperdício de estados (pois com N *flip-flops* poderiam codificar-se até 2^N estados distintos), oferece outros benefícios, particularmente em termos de síntese.

Um aspeto fundamental desta abordagem de projeto de controlo é a fase de inicialização. Aqui, um bit 1 tem de ser "injetado"na máquina de estados, de tal forma que apenas o primeiro *flip-flop* da "cadeia"de *flip-flops* seja colocado a 1. Isto é particularmente delicado quando esta inicialização é feita por um dispositivo externo com velocidades de reação muito diferentes e possivelmente assíncronas em relação à máquina de estados.

Neste exercício, pede-se que projete um circuito sequencial de *inicialização* que recebe como entrada um sinal S, operado por um circuito sequencial "lento", e "injeta"um bit 1 num circuito sequencial com codificação *one-hot*. A única saída desta máquina sequencial é o sinal P. Obviamente, sempre que S é 0, a máquina de estados principal deve estar toda no estado $0 \ldots 0$. Depois, sempre que S transita para 1, o primeiro *flip-flop* da "cadeia"será inicializado a 1 (de acordo com o seu relógio interno), mas apenas um único 1 deve ser gerado, independentemente de quanto tempo o sinal de entrada S permanecer a 1. De facto, para que um segundo 1 seja gerado, o sinal de entrada S terá de voltar ao estado 0. O diagrama temporal abaixo apresenta o comportamento temporal esperado da saída P deste circuito sequencial de inicialização em relação à sua entrada S.



▸ **Exercício 13**

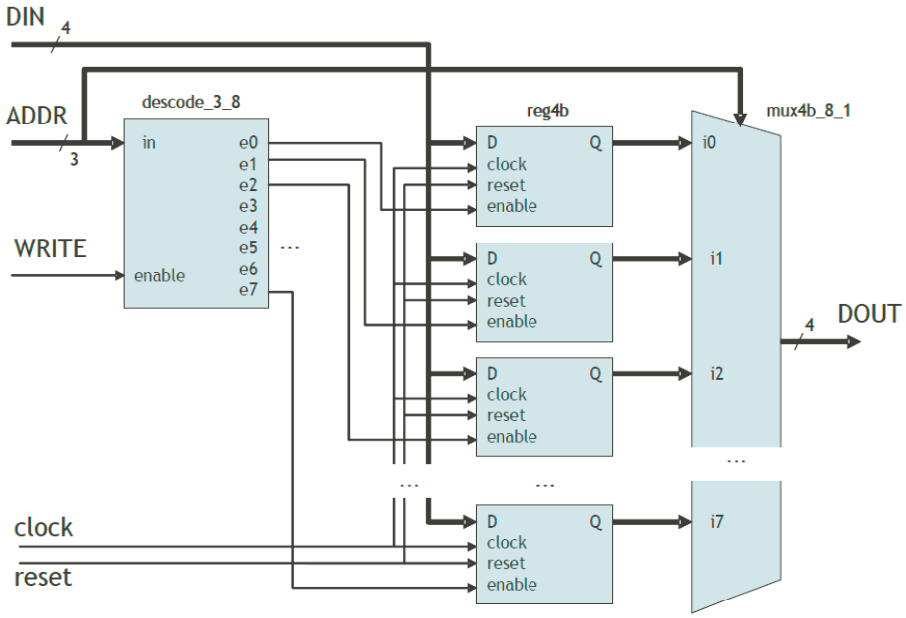
Conceba uma máquina de estados finitos (FSM) para uma máquina sequencial síncrona com uma única entrada e uma única saída. A máquina recebe sequências de entrada de 3 bits, um por ciclo de relógio, e gera, no quarto ciclo de relógio, uma saída que torna *par* o número de 1s da palavra completa de 4 bits (também chamada *nibble*). Durante os três primeiros ciclos de relógio, a saída da máquina deve ser 0. Como exemplo, quando a entrada é 010, no quarto ciclo de relógio a saída da máquina sequencial deve ser 1, enquanto quando a entrada é 101, a saída da máquina sequencial deve ser 0.

▸ **Exercício 14**

Conceba uma máquina de estados finitos (FSM) para uma máquina sequencial síncrona com uma única entrada X e duas saídas, respetivamente $Z1$ e $Z2$, que se comporte da seguinte forma. A saída $Z0$ deve ser 1 apenas quando, na entrada X, tiver sido observada, ao longo dos últimos 4 ciclos de relógio, a sequência binária 1101, e deve ser 0 caso contrário. A saída $Z1$ deve ser 1 apenas quando, na entrada X, tiver sido observada, ao longo dos últimos 4 ciclos de relógio, a sequência binária 1011, e deve ser 0 caso contrário. Note que a máquina deve detetar os mesmos padrões de entrada continuamente, *i.e.*, pode haver um padrão 1101 sobreposto a duas sequências consecutivas de 4 bits de entrada, como por exemplo no caso de a entrada ser 101101.

▸ **Exercício 15**

A figura abaixo apresenta a estrutura de um banco de registos.



Para além dos 8 registos de 4 bits, o circuito inclui um descodificador de 3 para 8 e um multiplexador de 8 para 1 (com sinais de 4 bits de largura). A entrada DIN (Data INput) contém os dados de 4 bits a armazenar num dos 8 registos, cujo endereço é definido pelas linhas do barramento $ADDR$. A operação de escrita no registo só ocorre quando $WRITE = 1$ e é síncrona em relação ao sinal de relógio. A leitura de registos, no entanto, é assíncrona e puramente combinatória: dado o endereço $ADDR$ e $WRITE = 0$, o circuito apresenta na saída $DOUT$ (Data OUTput) os valores do registo correspondentemente endereçado.

a) Determine a capacidade de armazenamento deste banco de registos em número de bytes.

b) Descreva que mecanismo de hardware garante a escrita dos dados num único registo.

c) Assumindo que o conteúdo inicial de todos os registos é 0000 e que os valores dos sinais de entrada são, respetivamente, $DIN = 1111$, $ADDR = 001$ e $WRITE = 1$, descreva as operações do circuito após uma transição do sinal de relógio.

d) Descreva a função dos sinais *enable* e *reset*.

e) Descreva o papel do descodificador e do multiplexador no funcionamento deste banco de registos.

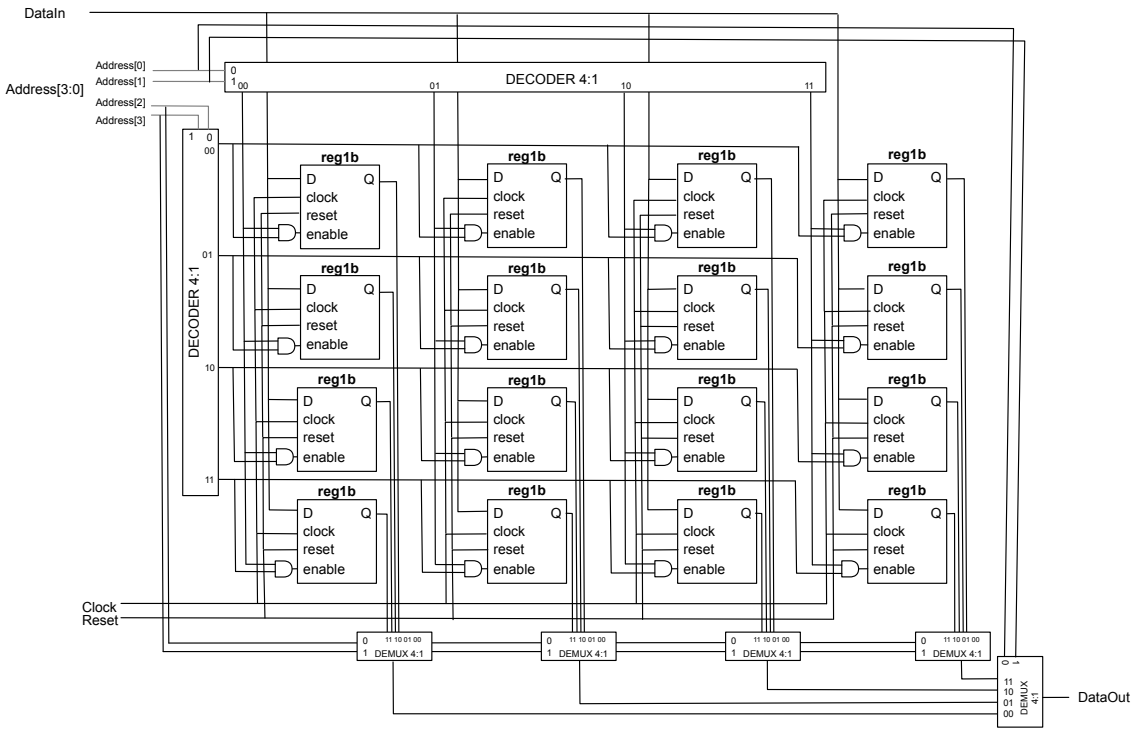
f) Sugira as modificações e/ou adições necessárias ao circuito para permitir a leitura simultânea de dois registos distintos, usando como saídas dois sinais de 4 bits, nomeadamente $DOUT1$ e $DOUT2$.

5 Circuitos e Organização de Memória

5.1 Exercícios Resolvidos

▸ **Exercício 1**

A figura abaixo apresenta a estrutura de um banco de registos que inclui registos de 1 bit e vários circuitos lógicos combinatórios.



a) Determine a capacidade de memória (em bits) de todo o conjunto.

b) Descreva o seu funcionamento, em particular o papel das linhas de endereço e dos descodificadores e demultiplexadores associados.

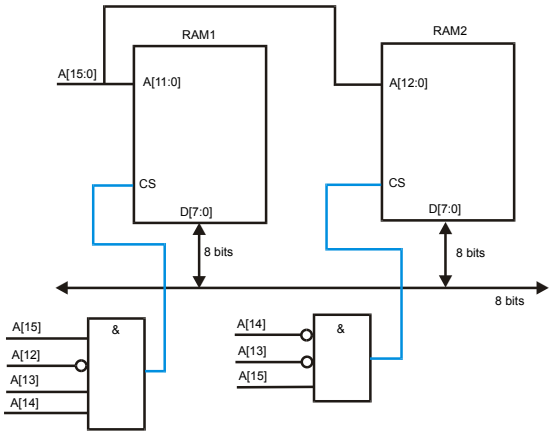
Solução

a) Trata-se de um banco de registos simples (e possivelmente pouco eficiente) de 4 por 4 registos de 1 bit, que permite operações de *leitura* e *escrita* em bits individuais. Cada bit está disposto numa organização matricial 4×4 e é acedido através de um endereço de 4 bits.

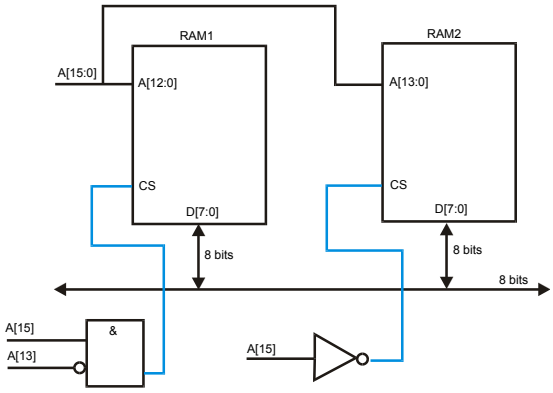
b) Para aceder a um bit específico deste conjunto, o endereço de 4 bits é dividido em dois conjuntos de 2 bits. Os bits mais significativos determinam a linha do registo de 1 bit selecionado e os menos significativos, a coluna. A operação de *leitura* é totalmente assíncrona em relação ao sinal de relógio, pois a seleção do endereço seleciona imediatamente o registo a ler, usando a ”cascata”de demultiplexadores. Uma operação de *escrita* usa o sinal de *relógio* para escrever no registo de 1 bit selecionado. A seleção é feita através da combinação única de coluna e linha, usando uma porta *AND* associada a cada registo de 1 bit.

▸ **Exercício 2**

Considere os dois sistemas de memória externa apresentados abaixo.



(a)



(b)

a) Determine a capacidade de memória (em bytes) de cada sistema.

b) Obtenha o mapa de memória de cada sistema, justificando se a descodificação de endereços é total ou parcial.

c) Para cada um dos seguintes endereços, indique o dispositivo de memória que é selecionado: B7FF_H, 1000_H, E7AA_H e 8000_H.

Solução

a) Podemos obter a capacidade de memória multiplicando o número de posições endereçáveis pelo número de bits armazenados em cada posição. O número de bits por posição endereçável é igual ao número de bits de dados do porto de dados `D[]` de cada memória; neste problema em particular, são 8 bits (ou 1 byte). O número de posições é determinado pelo número de bits de endereço N e, para N bits de endereço, há um máximo de 2^N posições de memória endereçáveis.

Assim, na figura (a) temos:

- RAM1: $2^{12} \times 8\,\text{bits} = 2^2 \times 2^{10} \times 8\,\text{bits} = 4 \times 2^{10}\,\text{byte} = 4\,\text{KiB}$
- RAM2: $2^{13} \times 8\,\text{bits} = 8\,\text{KiB}$

enquanto na figura (b) temos:

- RAM1: $2^{13} \times 8$ bits = 8 KiB
- RAM2: $2^{14} \times 8$ bits = 16 KiB

b) Um mapa de memória indica, para cada endereço, que circuito de memória armazenará os dados correspondentes. Em ambas as figuras, o barramento de endereços do CPU tem 16 bits de largura, (A[15:0]), pelo que o espaço de endereçamento compreende 2^{16} endereços distintos, sendo assim capaz de referenciar individualmente o mesmo número de posições de memória. Cada uma destas posições tem 1 byte de largura e, como tal, temos um total de 64 KBytes. A gama de endereços está definida entre os valores 0000_H e $FFFF_H$.

Os endereços que correspondem a cada circuito de memória, *i.e.*, que o ativarão através do sinal de *chip select (cs)* correspondente (quando $cs = 1$), podem ser encontrados pela análise da lógica de descodificação, da seguinte forma.

Na figura (a) temos:

- RAM1: CS = $A_{15} \cdot A_{14} \cdot A_{13} \cdot \overline{A_{12}} = 1$.
  Esta condição é satisfeita sempre que $A_{15} = 1, A_{14} = 1, A_{13} = 1, A_{12} = 0$.
  Portanto, a gama de endereços que corresponde (ou seleciona) a memória RAM1 tem o formato genérico
  1110 XXXX XXXX XXXX
  onde X denota um bit que pode ser 0 ou 1. Os endereços de memória nesta gama são, portanto:
  1110 0000 0000 0000 a 1110 1111 1111 1111.
  que em hexadecimal se denota por $E000_H$ - $EFFF_H$.
  Como todos os bits de endereçamento são usados, $A_{15} - A_{12}$ na definição do sinal CS e os restantes 12 bits estão explicitamente ligados aos portos de endereço do circuito de memória RAM1, trata-se de facto de um esquema de *descodificação total*.
- RAM2: CS = $A_{15} \cdot \overline{A_{14}} \cdot \overline{A_{13}} = 1$.
  Esta condição é satisfeita sempre que $A_{15} = 1, A_{14} = 0, A_{13} = 0$.
  Portanto, a gama de endereços que corresponde (ou seleciona) a memória RAM2 tem o formato genérico
  100X XXXX XXXX XXXX.
  Os endereços de memória nesta gama são, portanto:
  1000 0000 0000 0000 a 1001 1111 1111 1111.
  que em hexadecimal se denota por 8000_H - $9FFF_H$.
  Tal como no exemplo anterior, trata-se de um esquema de *descodificação total*.

Em resumo, o mapa de memória do sistema de memória da figura (a) é o seguinte:

Gama (hex)	Dispositivo selecionado
0000--7FFF	---
8000--9FFF	RAM2
A000--DFFF	---
E000--EFFF	RAM1
F000--FFFF	---

A análise do circuito da figura (b) faz-se de forma análoga.

- RAM1: CS = $A_{15} \cdot \overline{A_{13}}$ = 1. Esta condição é satisfeita sempre que A_{15} = 1 e A_{13} = 0 e, como tal, os endereços correspondentes ao dispositivo de memória RAM1 têm o formato:

  1?0X XXXX XXXX XXXX.

  Aqui, o símbolo **?** indica que o bit correspondente (que substitui) não é usado na descodificação do endereço. Trata-se, portanto, de um esquema de *descodificação parcial*.

  Neste esquema de descodificação em particular, dois endereços que difiram apenas no bit A_{14} resultarão na seleção do mesmo dispositivo de memória, pois correspondem à mesma posição no espaço de endereçamento. Temos assim duas gamas de endereços equivalentes, que apenas diferem no valor do bit A_{14}.

  Para A_{14} = 0, a gama é:

  1000 0000 0000 0000 a 1001 1111 1111 1111.

  Em hexadecimal, a gama é 8000_H - $9FFF_H$.

  Para A_{14} = 1, a gama é dada por:

  1100 0000 0000 0000 a 1101 1111 1111 1111.

  Em hexadecimal, a gama é $C000_H$ - $DFFF_H$.

  As duas gamas de endereços estão, com efeito, sobrepostas no dispositivo de memória RAM1. Como exemplo, os endereços 8000_H e $C000_H$ referem-se ambos à mesma posição física de memória na RAM1. Note que a quantidade de memória disponível não aumenta em resultado do uso de uma descodificação parcial.

- Também usamos descodificação parcial para o dispositivo de memória RAM2. Aqui, os endereços têm o formato apresentado abaixo:

  0?XX XXXX XXXX XXXX,

  ao qual correspondem as seguintes gamas de endereços: 0000_H – $3FFF_H$ e 4000_H – $7FFF_H$.

O mapa de memória do sistema da figura (b) é o apresentado abaixo, onde os asteriscos ('*') indicam gamas de endereços sobrepostas.

Gama de endereços (hex)	Dispositivo de memória
0000--3FFF	RAM2 (*)
4000--7FFF	RAM2 (*)
8000--9FFF	RAM1 (**)
A000--BFFF	---
C000--DFFF	RAM1 (**)
E000--FFFF	---

c) Para determinar os dispositivos de memória selecionados para cada um dos endereços fornecidos, basta inspecionar os mapas de memória obtidos acima, resultando na tabela abaixo.

Endereço	Figura (a)	Figura (b)
B7FF	---	---
1000	---	RAM2
E7AA	RAM1	---
8000	RAM2	RAM1

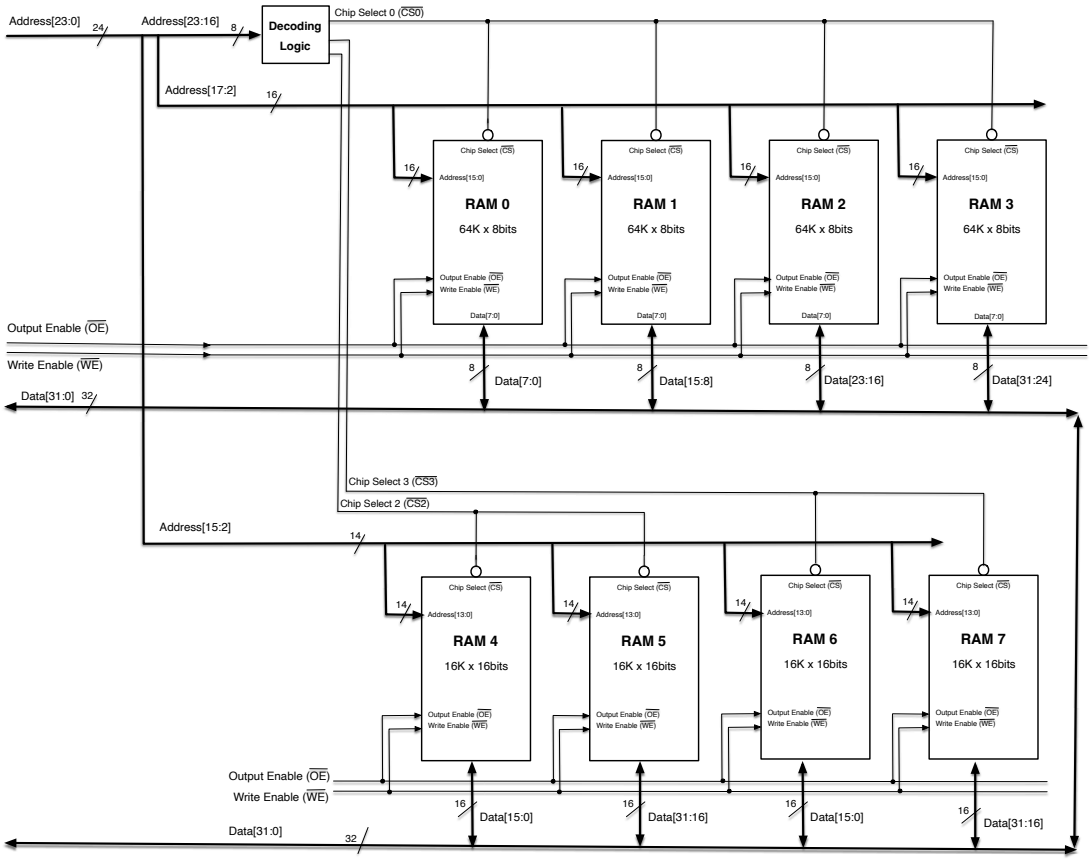
▸ **Exercício 3**

Considere um sistema de computação com um único processador (CPU) e um barramento de dados de 32 bits. Internamente, o processador tem capacidade de endereçamento de 24 bits, *i.e.*, pode endereçar 16 Mbytes. No entanto, como cada palavra endereçável tem 32 bits de largura, *i.e.*, 4 bytes, os endereços do CPU usados no barramento de endereços consistem nos 22 bits superiores dos endereços internos do CPU. Por outras palavras, sempre que o CPU quer aceder a um byte da memória, lê 4 bytes. Os 2 bits menos significativos do endereço são, portanto, ignorados quando um endereço é colocado no barramento de endereços, sendo usados internamente para selecionar qual dos 4 bytes lidos as instruções do processador querem manipular.

Pede-se que projete o circuito lógico de descodificação que usa 4 módulos de memória RAM de 64 Kbytes cada, com saídas de dados de 8 bits, e 4 módulos de memória RAM de 32 Kbytes cada, com saídas de dados de 16 bits. O circuito deve atribuir 256 Kbytes aos primeiros endereços do mapa de endereçamento do CPU e 128 Kbytes aos últimos endereços (endereços altos) do mapa de endereçamento do CPU. Terá de organizar os módulos de memória de modo a fornecer 32 bits no barramento de dados do sistema.

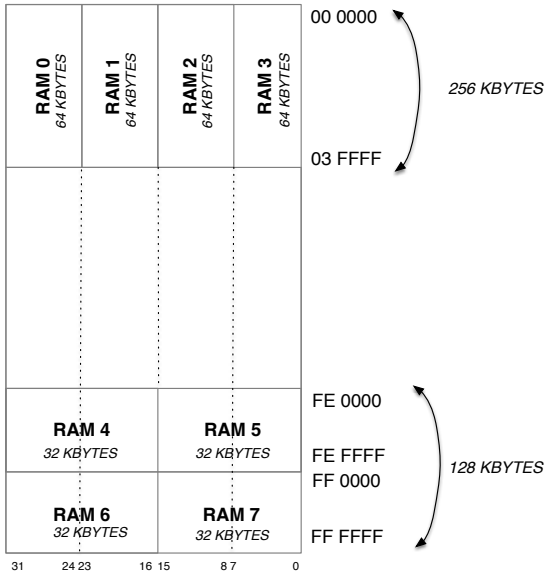
Solução

O aspeto possivelmente complicado deste projeto é perceber que os dois bits menos significativos do endereço não são usados fora do processador. A isto chama-se **alinhamento de endereços**, neste caso em fronteiras de 4 bytes. O processador nunca endereça explicitamente um byte na memória, mas sim uma palavra completa de 32 bits. Em resultado, os 2 bits menos significativos de qualquer endereço nunca são usados no barramento de endereços. A figura abaixo apresenta uma possível organização dos 8 módulos de RAM a usar, onde omitimos, por agora, os detalhes da lógica de descodificação.

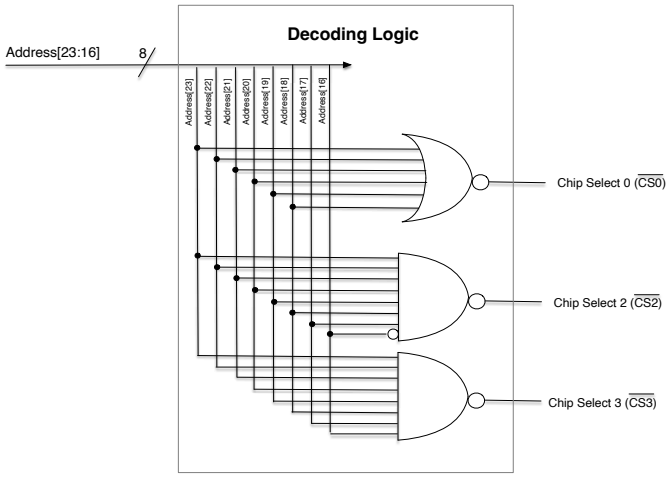


Nesta figura, RAM0 a RAM3 estão todas ligadas à mesma linha de chip select e às mesmas linhas de endereço, com uma capacidade agregada de 256 Kbytes; mas, como estão organizadas para fornecer palavras de 32 bits, a gama de endereços coberta é de apenas 64 K palavras. Assim, temos de ter os 16 bits de endereço inferiores (acima dos bits 0 e 1) ligados a estes 4 módulos de memória em paralelo. Isto cobre os endereços do espaço de endereçamento do processador de 00 0000 a 03 FFFF.

Quanto à parte alta do espaço de endereçamento, usamos os 4 módulos de memória de 32 Kbytes, cada um dos quais fornece dados de 16 bits. Neste caso, agrupamos RAM4 a RAM7 em pares, como indicado na figura. Aqui, temos duas linhas de chip select, uma para cada um dos pares de RAMs. A capacidade agregada destes módulos de memória é de 128 Kbytes, cobrindo 32K endereços, portanto apenas de FE 0000 a FF FFFF, como mostrado no mapa de memória abaixo.



Uma possível implementação discreta da lógica de descodificação é a apresentada no diagrama abaixo. A linha de Chip Select 0 (ou $\overline{CS0}$) ativa as 4 RAMs da primeira gama de endereços do espaço de endereçamento, enquanto $\overline{CS2}$ e $\overline{CS3}$ ativam as gamas de FE 0000 a FE FFFF e de FF 0000 a FF FFFF, respetivamente.



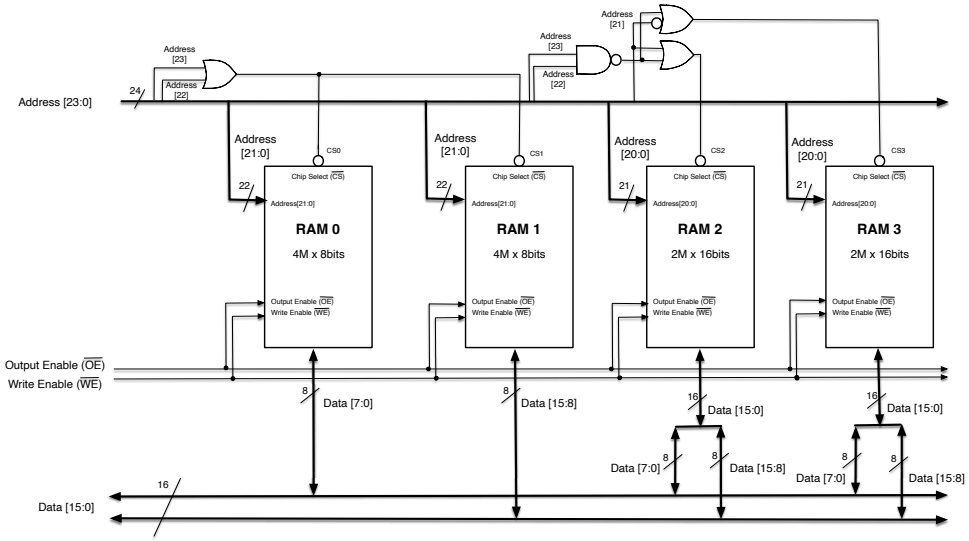
▸ **Exercício 4**

Projete um sistema de memória com palavras indexáveis de 16 bits usando 4 dispositivos de memória. Dois dos dispositivos têm capacidade de armazenamento de 4M palavras de 8 bits e os restantes 2 dispositivos têm capacidade de armazenamento de 2M palavras de 16 bits.

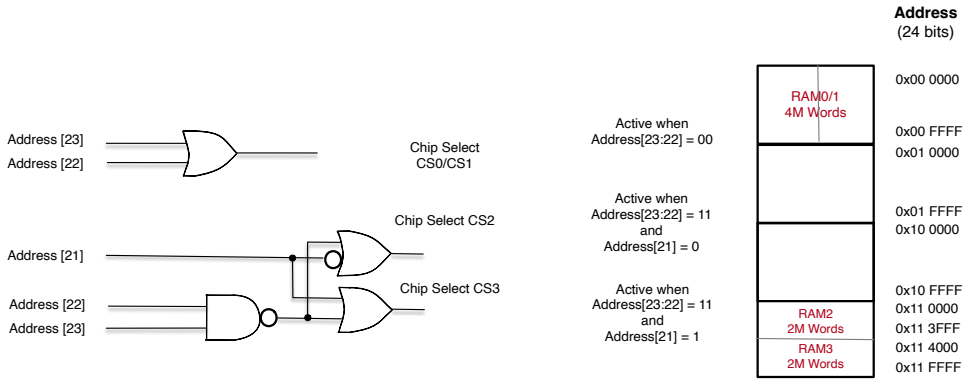
Para o seu projeto, indique a capacidade global em bytes e apresente um diagrama lógico com o endereçamento de cada dispositivo de memória individual. Sugira um possível mapa de endereçamento para um sistema de computação com 24 bits de endereçamento.

Solução

Claramente, a capacidade total é de $4 + 2 \times 2$ Mbytes, ou seja, 8 Mbytes. O aspeto possivelmente complicado deste projeto são os vários mapeamentos possíveis de endereços para armazenamento. O diagrama abaixo apresenta uma disposição possível, onde os dois dispositivos de armazenamento de 8 bits de largura estão mapeados nos endereços inferiores do espaço de endereçamento e os dois dispositivos de armazenamento de 16 bits de largura estão mapeados nos endereços superiores.



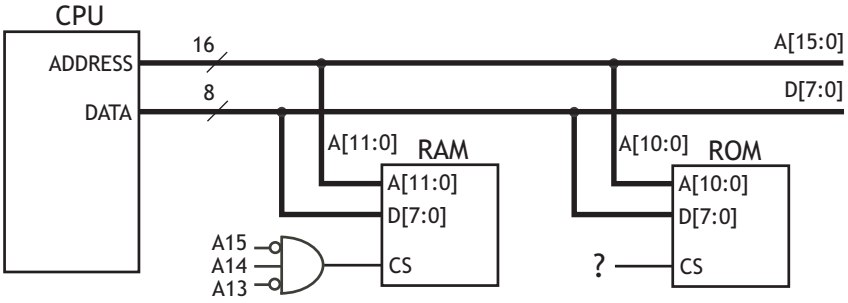
Para compreender este mapeamento, mostramos abaixo a lógica discreta simples, indicando para que combinação dos 24 bits de endereço a saída correspondente está ativa (neste caso, com um valor lógico baixo).



5.2 Exercícios Propostos

▸ **Exercício 5**

O sistema de memória apresentado na figura abaixo é composto por um dispositivo de memória RAM (Random-Access-Memory) e um de memória ROM (Read-Only-Memory). O barramento de endereços tem 16 bits de largura e o barramento de dados tem 8 bits de largura. O sinal *CS* (Chip-Select) de cada dispositivo de memória ativa o dispositivo correspondente.



a) Determine a gama de endereços associada à memória RAM e indique se corresponde a uma descodificação total ou parcial.

b) Considerando que se espera que a primeira posição da memória ROM esteja no endereço 0xC800 e que o endereço de cada posição é único, determine o endereço da última posição da ROM.

c) Dado o endereçamento descrito na alínea anterior, defina e desenhe a lógica de descodificação correspondente a ligar à entrada *CS* da ROM.

▸ **Exercício 6**

Um dado processador define um espaço de endereçamento de 64 KiB e tem um barramento de dados de 8 bits de largura. O seu mapa de memória é o seguinte:

Gama de endereços	Dispositivo selecionado
0x0000-0x3FFF	ROM1
0x4000-0x7FFF	RAM1
0x8000-0xCFFF	-
0xD000-0xDFFF	RAM2
0xE000-0xFFFF	-

a) Determine o tamanho (em bytes) de cada dispositivo de armazenamento.

b) Determine as expressões lógicas dos circuitos lógicos de descodificação correspondentes, assumindo uma descodificação total.

c) Desenhe um diagrama de blocos do sistema de memória que inclua a lógica de descodificação de endereços.

▸ **Exercício 7**

Um computador usa um processador com endereços de 14 bits, manipulando quantidades de 8 bits. Assuma que só tem à sua disposição dispositivos de memória RAM com $2^{10} \times 4$ bits.

Neste exercício, pede-se que projete um sistema de memória com 2048 endereços de memória, cada um designando uma posição de memória com 8 bits. Os endereços devem começar no endereço 0, empregar descodificação total, e os endereços ímpares e pares devem selecionar dispositivos RAM distintos.

a) Desenhe o esquema do sistema, mostrando a lógica de descodificação e de interligação entre os vários dispositivos de memória.

b) O valor `0x9A` deve ser associado ao endereço 250_{H}. Em que dispositivos de memória e em que endereço desses dispositivos ficam os dados armazenados?

Parte III

Linguagem Assembly

6 Linguagem Assembly

Este capítulo contém exercícios de linguagem assembly centrados exclusivamente na arquitetura do conjunto de instruções (ISA) RISC-V RV32IMFD. As primeiras páginas resumem as instruções mais importantes e o seu efeito no estado do processador RISC-V. O resumo inclui também uma breve descrição da codificação das instruções e do seu formato, bem como (quando aplicável) considerações de nível de sistema, tais como o mapa típico de endereços de memória e a convenção de chamada de subrotinas (*procedure linkage*). Para uma fonte definitiva, consulte `https://riscv.org/technical/specifications/`.

Conjunto de Instruções RISC-V

Formatos base das instruções

31----------25	24---20	19---15	14---12	11-------7	6-----0	
funct7	rs2	rs1	funct3	rd	opcode	**Tipo R**
imm[11:0]		rs1	funct3	rd	opcode	**Tipo I**
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	**Tipo S**
imm[12][10:5]	rs2	rs1	funct3	imm[4:1][11]	opcode	**Tipo B**
imm[31:12]				rd	opcode	**Tipo U**
imm[20][10:1][11][19:12]				rd	opcode	**Tipo J**

Instruções inteiras base RV32I

Inst	Nome	FMT	Opcode	funct3	funct7	Descrição
add	ADD	R	0110011	000	0000000	rd = rs1 + rs2
sub	SUB	R	0110011	000	0100000	rd = rs1 - rs2
xor	XOR	R	0110011	100	0000000	rd = rs1 ∧ rs2
or	OR	R	0110011	110	0000000	rd = rs1 | rs2
and	AND	R	0110011	111	0000000	rd = rs1 & rs2
sll	Shift Left Logical	R	0110011	001	0000000	rd = rs1 << rs2
srl	Shift Right Logical	R	0110011	101	0000000	rd = rs1 >> rs2
sra	Shift Right Arith.	R	0110011	101	0100000	rd = rs1 >> rs2
slt	Set Less Than	R	0110011	010	0000000	rd = (rs1 < rs2)?1:0
sltu	Set Less Than (U)	R	0110011	011	0000000	rd = (rs1 < rs2)?1:0
addi	ADD Immediate	I	0010011	000		rd = rs1 + imm
xori	XOR Immediate	I	0010011	100		rd = rs1 ∧ imm
ori	OR Immediate	I	0010011	110		rd = rs1 | imm
andi	AND Immediate	I	0010011	111		rd = rs1 & imm
slli	Shift Left Log. Imm	I	0010011	001	0000000	rd = rs1 << imm[0:4]
srli	Shift Right Log. Imm	I	0010011	101	0000000	rd = rs1 >> imm[0:4]
srai	Shift Right Arith. Imm	I	0010011	101	0100000	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	010		rd = (rs1 <imm)?1:0
sltiu	Set Less Than (U) Imm	I	0010011	011		rd = (rs1 <imm)?1:0
lb	Load Byte	I	0000011	000		rd = M[rs1+imm][0:7]
lh	Load Half	I	0000011	001		rd = M[rs1+imm][0:15]
lw	Load Word	I	0000011	010		rd = M[rs1+imm][0:31]
lbu	Load Byte (U)	I	0000011	100		rd = M[rs1+imm][0:7]
lhu	Load Half (U)	I	0000011	101		rd = M[rs1+imm][0:15]
sb	Store Byte	S	0100011	000		M[rs1+imm][0:7] = rs2[0:7]
sh	Store Half	S	0100011	001		M[rs1+imm][0:15] = rs2[0:15]
sw	Store Word	S	0100011	010		M[rs1+imm][0:31] = rs2[0:31]
beq	Branch ==	B	1100011	000		if(rs1 == rs2) PC += imm
bne	Branch ≠	B	1100011	001		if(rs1 != rs2) PC += imm
blt	Branch <	B	1100011	100		if(rs1 < rs2) PC += imm
bge	Branch ≥	B	1100011	101		if(rs1 >= rs2) PC += imm
bltu	Branch < (U)	B	1100011	110		if(rs1 < rs2) PC += imm
bgeu	Branch ≥ (U)	B	1100011	111		if(rs1 >= rs2) PC += imm
jal	Jump And Link	J	1101111	-		rd = PC+4; PC += imm
jalr	Jump And Link Reg	I	1100111	000		rd = PC+4; PC = rs1 + imm
lui	Load Upper Imm	U	0110111	-		rd = imm << 12
auipc	Add Upper Imm to PC	U	0010111	-		rd = PC + (imm << 12)

Extensão de multiplicação RV32M

Inst	Nome	FMT	Opcode	funct3	funct7	Descrição
mul	MUL	R	0110011	000	0000001	rd = (rs1 × rs2)[31:0]
mulh	MUL High	R	0110011	001	0000001	rd = (rs1 × rs2)[63:32]
mulsu	MUL High (S) (U)	R	0110011	010	0000001	rd = (rs1 × rs2)[63:32]
mulu	MUL High (U)	R	0110011	011	0000001	rd = (rs1 × rs2)[63:32]
div	DIV	R	0110011	100	0000001	rd = rs1 / rs2
divu	DIV (U)	R	0110011	101	0000001	rd = rs1 / rs2
rem	Remainder	R	0110011	110	0000001	rd = rs1 % rs2
remu	Remainder (U)	R	0110011	111	0000001	rd = rs1 % rs2

Pseudoinstruções

As pseudoinstruções são aceites pelo assemblador e traduzidas para instruções nativas (i.e., instruções diretamente suportadas pelo CPU). Existem por conveniência do programador.

Pseudoinstrução	**Instrução(ões) nativa(s)**	**Significado**
`la rd, symbol`	`addi rd, rd, symbol[11:0]`	Carregar endereço de símbolo
`l{b|h|w|d} rd, symbol`	`l{b|h|w|d} rd, symbol[11:0](rd)`	Carregar global
`s{b|h|w|d} rd, symbol, rt`	`s{b|h|w|d} rd, symbol[11:0](rt)`	Armazenar global
`nop`	`addi x0, x0, 0`	Nenhuma operação
`li rd, immediate`	várias sequências	Carregar imediato
`mv rd, rs`	`addi rd, rs, 0`	Copiar registo
`not rd, rs`	`xori rd, rs, -1`	Complemento para 1
`neg rd, rs`	`sub rd, x0, rs`	Complemento para 2
`negw rd, rs`	`subw rd, x0, rs`	Complemento para 2 (word)
`sext.w rd, rs`	`addiw rd, rs, 0`	Extensão de sinal (word)
`seqz rd, rs`	`sltiu rd, rs, 1`	Ativar se = zero
`snez rd, rs`	`sltu rd, x0, rs`	Ativar se ≠ zero
`sltz rd, rs`	`slt rd, rs, x0`	Ativar se < zero
`sgtz rd, rs`	`slt rd, x0, rs`	Ativar se > zero
`beqz rs, offset`	`beq rs, x0, offset`	Saltar se = zero
`bnez rs, offset`	`bne rs, x0, offset`	Saltar se ≠ zero
`blez rs, offset`	`bge x0, rs, offset`	Saltar se ≤ zero
`bgez rs, offset`	`bge rs, x0, offset`	Saltar se ≥ zero
`bltz rs, offset`	`blt rs, x0, offset`	Saltar se < zero
`bgtz rs, offset`	`blt x0, rs, offset`	Saltar se > zero
`bgt rs, rt, offset`	`blt rt, rs, offset`	Saltar se >
`ble rs, rt, offset`	`bge rt, rs, offset`	Saltar se ≤
`bgtu rs, rt, offset`	`bltu rt, rs, offset`	Saltar se >, sem sinal
`bleu rs, rt, offset`	`bgeu rt, rs, offset`	Saltar se ≤, sem sinal
`j offset`	`jal x0, offset`	Salto
`jal offset`	`jal x1, offset`	Salto com ligação
`jr rs`	`jalr x0, rs, 0`	Salto por registo
`jalr rs`	`jalr x1, rs, 0`	Salto com ligação por registo
`ret jalr`	`x0, x1, 0`	Retorno de subrotina
`call offset`	`auipc x1, offset[31:12]` `jalr x1, x1, offset[11:0]`	Chamada de subrotina distante

Convenção de uso dos registos

Registo	Nome ABI	Descrição	Preservado por
x0	zero	Constante zero	N/A
x1	ra	Endereço de retorno	Chamado
x2	sp	Apontador de pilha	Chamado
x3	gp	Apontador global	N/A
x4	tp	Apontador de thread	N/A
x5-x7	t0-t2	Temporários	Chamador
x8	s0/fp	Preservado/apontador de frame	Chamado
x9	s1	Registo preservado	Chamado
x10-x11	a0-a1	Argumentos/retorno de função	Chamador
x12-x17	a2-a7	Argumentos de função	Chamador
x18-x27	s2-s11	Registos preservados	Chamado
x28-x31	t3-t6	Temporários	Chamador

6.1 Exercícios Resolvidos

▸ **Exercício 1**

Nas expressões de atribuição e aritméticas abaixo, os símbolos denotam variáveis de um programa de alto nível que contêm valores inteiros (usando representações de complemento para 2 com 32 bits). Para cada atribuição e expressão, especifique um mapeamento entre variáveis e registos do RISC-V e apresente um exemplo de código assembly que efetue a avaliação das expressões e a correspondente atribuição de valores. Aqui, a referência a *A* deve entender-se como uma variável do tipo vetor unidimensional, em que cada posição do vetor é também um valor inteiro de 32 bits.

a) `f = g - (f + 5)`

b) `f = A[12] + 17`

Solução

O primeiro passo neste tipo de problemas é determinar a atribuição de variáveis a registos.[1] Aqui, por simplicidade, fazemos uma atribuição arbitrária e assumimos que a atribuição se mantém durante toda a execução do código. Como a arquitetura RISC-V tem 31 registos de uso geral (dado que `x0` é sempre 0), neste caso particular pode usar-se um registo diferente para cada variável.

a) Uma possível atribuição de registos a variáveis é a seguinte (escolha arbitrária):

`x5`: `f` `x6`: `g`

O fragmento de código que efetua os cálculos pretendidos é:

```
addi    x5, x5, 5    # calcula f = f + 5
sub     x5, x6, x5   # calcula f = g - f
```

Após esta sequência de duas instruções, `x5` contém o novo valor associado a `f`. O cálculo da primeira parte da expressão (instrução `addi`) pode guardar o resultado intermédio no registo `x5`, porque a segunda instrução estabelece o valor final correto.

b) Uma possível atribuição de registos a variáveis é a seguinte:

`x5`: `f` `x6`: endereço base de `A`

Como cada elemento de uma sequência de inteiros tem 4 bytes, o elemento de índice 12 do vetor `A` está armazenado em memória no endereço:

`endereço base de A + 12 × 4 = 48`

A primeira instrução vai buscar o valor armazenado nessa posição.

```
lw      x5, 48(x6)  # carrega uma word do endereco x6+48
addi    x5, x5, 17  # soma 17 a esse valor
```

[1] Na vida real, esta tarefa é feita pelo compilador e é, em geral, um processo muito complexo. No final, cada variável fica associada a um registo específico num dado ponto da execução do código.

O código soma depois trivialmente o valor 17 a este valor, para obter o valor final da expressão que deve ser atribuído à variável f e, portanto, ao registo x5.

▸ **Exercício 2**

Assumindo as seguintes condições iniciais dos registos do RISC-V:

x5 = 0xBEADFEED x6 = 0xDEADFADE

Determine o valor de x7 após a execução das seguintes sequências de instruções:

a)
```
slli    x7, x5, 4
or      x7, x7, x6
```

b)
```
srli  x7, x5, 3
andi  x7, x7, 0xFFFFFFEF
```

Solução

Em binário, os valores iniciais dos registos são:

$$\texttt{x5} = \texttt{1011 1110 1010 1101 1111 1110 1110 1101}_2$$
$$\texttt{x6} = \texttt{1101 1110 1010 1101 1111 1010 1101 1110}_2$$

a) A primeira instrução desloca o valor de x5 quatro bits para a esquerda, colocando a zero os 4 bits menos significativos. O resultado da operação é armazenado em x7; o registo x5 fica inalterado. O valor de x7 após a execução da primeira instrução é:

$$\texttt{x7} = \texttt{1110 1010 1101 1111 1110 1110 1101 0000}_2$$

A instrução or calcula a função OU inclusivo, or, de cada bit de x7 com o bit correspondente de x6[2]. O resultado é guardado em x7. Assim,

$$\texttt{x7} = \texttt{1111 1110 1111 1111 1111 1110 1101 1110}_2 = \texttt{0xFEFFFEDE}$$

b) A instrução srli desloca o valor de x5 três posições para a direita, introduzindo zeros a partir da esquerda; os 3 bits menos significativos perdem-se. O valor de x7 após a execução da primeira instrução é, portanto:

$$\texttt{x7} = \texttt{000}1\ \texttt{0111 1101 0101 1011 1111 1101 1101}_2$$

A instrução andi calcula a função and [3] de cada bit de x7 com o bit correspondente da constante de 32 bits 0xFFFFFFEF.

$$\texttt{FFFFFFEF}_{16} = \texttt{1111 1111 1111 1111 1111 1111 1110 1111}_2$$

[2] Recorde que o resultado da operação or de dois bits é 1 sempre que pelo menos um dos bits operandos é 1.
[3] O and de dois bits é 1 apenas se ambos os bits operandos forem 1.

O resultado é armazenado em `x7`. Neste caso, os operandos são dados pelo conteúdo de `x7` e pela constante indicada. O valor final de `x7` é:

$$\texttt{x7} = 0001\ 0111\ 1101\ 0101\ 1011\ 1111\ 1100\ 1101_2 = \texttt{0x17D5BFCD}$$

▸ **Exercício 3**

Assumindo os valores iniciais dos registos `x5` e `x6` indicados abaixo, determine o valor de `x6` após a execução do fragmento de código *assembly* RISC-V também apresentado abaixo.

`x5` = FFFF FFFF$_{16}$
`x6` = 0011 1111 1111 1000 0000 0000 0000 0000$_2$

```
        bge  x5, x6, ELSE
        beq  x0, x0, DONE
ELSE:   addi x6, x6, 2
DONE:   ...
```

Solução

A primeira instrução, um salto condicional, compara o conteúdo de `x5` e `x6`. Se `x5` for maior ou igual a `x6` (a instrução `bge` interpreta os valores dos registos como números em complemento para 2), o controlo é transferido para a instrução na etiqueta `ELSE`. Como, neste caso, o conteúdo de `x5` é negativo e o conteúdo de `x6` é positivo, o salto não será tomado. Em resultado, a segunda instrução a ser executada é a instrução de salto incondicional (a instrução `beq` com uma condição que é sempre verdadeira). Esta instrução faz com que o controlo seja transferido para a instrução na etiqueta `DONE`. A instrução `addi` não é executada, pelo que `x6` mantém o seu valor inicial.

▸ **Exercício 4**

Implemente a subrotina `sumSquare`, que recebe um inteiro n e devolve o valor da seguinte expressão se $n > 0$, ou 0 se $n \leq 0$.

$$n^2 + (n-1)^2 + (n-2)^2 + \cdots + 1^2$$

A subrotina `sumSquare` deve chamar uma subrotina `square` que calcula o quadrado de um inteiro.

Solução

```
sumSquare:
      addi  sp, sp, -16   # Reserva espaco para 4 words na pilha
      sw    ra, 12(sp)    # Guarda o endereco de retorno
      sw    s0, 8(sp)     # Guarda s0 e s1
      sw    s1, 4(sp)     #
      add   s0, a0, x0    # Copia N para s0
      add   s1, x0, x0    # Inicializa s1 (acumulara a soma)
loop:bge    x0, s0, end   # Salta se s0 nao for positivo
      add   a0, s0, x0    # Copia s0 para a0 para chamar 'square'
      jal   ra, square    # Chama a subrotina 'square'
      add   s1, s1, a0    # Atualiza s1 com o valor devolvido por 'square'
      addi  s0, s0, -1    # Decrementa o contador
      jal   x0, loop      # Salta para o inicio do ciclo
end:  add   a0, s1, x0    # Copia o valor acumulado de s1 para a0
      lw    ra, 12(sp)    # Recupera o valor de ra
      lw    s0, 8(sp)     # Recupera o valor de s0
      lw    s1, 4(sp)     # Recupera o valor de s1
      addi  sp, sp, 16    # Liberta o espaco reservado na entrada
      jr    ra            # Retorna ao chamador

 square:
      mul   a0, a0, a0
      ret
```

▸ **Exercício 5**

Escreva e teste um programa que use uma subrotina para contar o número de bits a 1 na representação binária de um valor do tipo `word`.

Solução

Esta é uma solução possível:

```
.data
num: .word 38     # Exemplo: Res=3

.text
    # testar e imprimir (usando o RARS)
    la   t0, num
    lw   a0, 0(t0)
    jal  ra, nbits1
    li   a7, 10
    ecall

# A subrotina
nbits1:
      mv   t0, zero
loop:
      beq  a0, zero, end
      andi t1, a0, 1
      add  t0, t0, t1
      srli a0, a0, 1
      j    loop
end:  mv   a0, t0
      ret
```

▸ **Exercício 6**

A Figura 6.1 mostra o estado de execução de uma subrotina no RARS. No teste apresentado, a subrotina processa uma sequência de cinco inteiros (32 bits) armazenados em memória a partir do endereço `0x10010000`.

a) Determine quantos bytes de memória são ocupados pela sequência de elementos.

b) Determine o valor decimal do quarto elemento da sequência.

c) Determine o endereço de memória do terceiro elemento da sequência.

d) Determine o endereço de memória da instrução `j cycle`.

e) Tendo em conta os valores dos registos, determine quantas vezes a instrução `lw t1, 0(a0)` já foi executada.

f) Determine o valor de `a0` depois de o código terminar a execução.

g) Descreva o que a subrotina faz.

Solução

Respostas muito curtas:

a) 20

b) 19

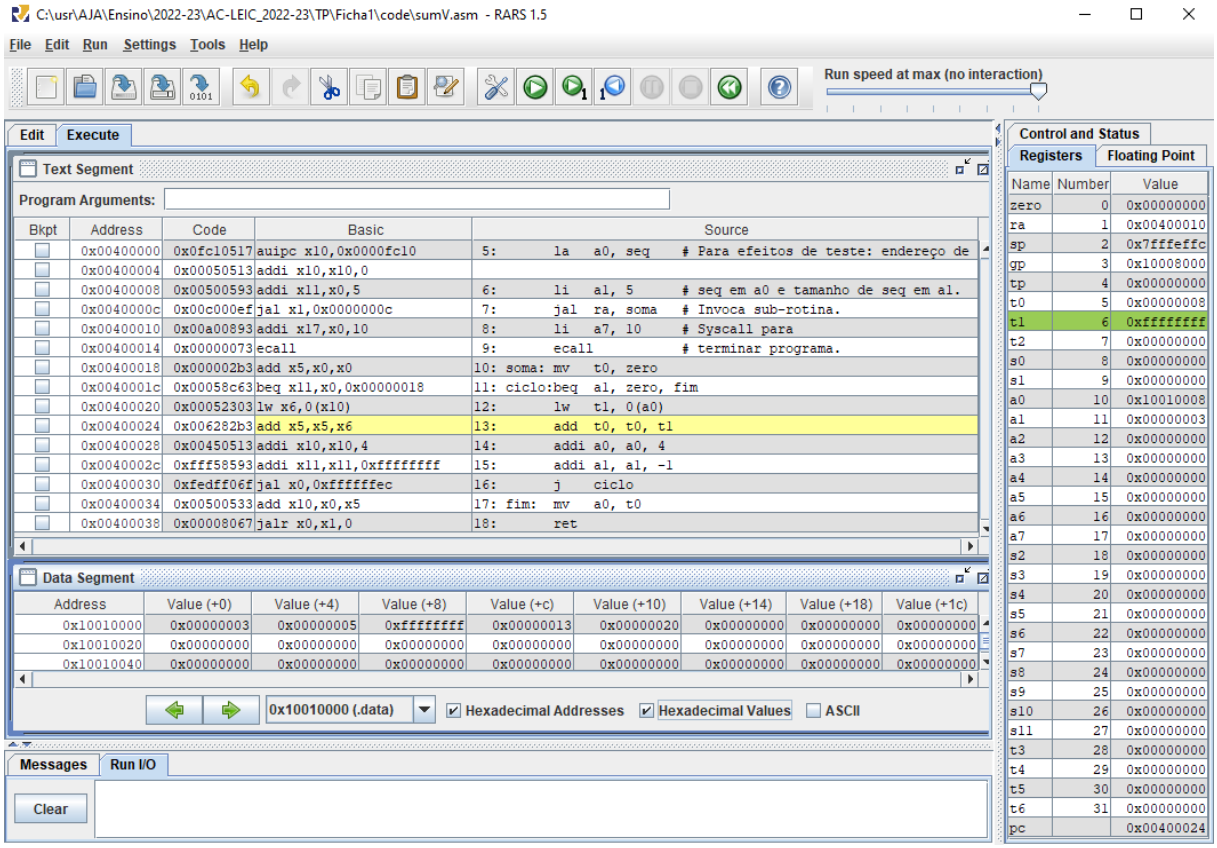


Figura 6.1: Captura de ecrã do RARS

c) `0x10010008`

d) `0x00400030`

e) 3

f) 58

g) A subrotina calcula a soma dos elementos de uma sequência.

6.2 Exercícios Propostos

▸ **Exercício 7**

Para as seguintes expressões aritméticas (inteiros de 32 bits), especifique um mapeamento entre variáveis e registos e obtenha um possível fragmento de código *assembly* RISC-V que as implemente.

a) `f = g + (j + 2)`

b) `k = a + b - f + d - 30`

c) `f = g + h + A[4]`

d) `f = g - A[B[10]]`

e) `f = k - g + A[h + 9]`

f) `f = g - A[B[2] + 4]`

▸ **Exercício 8**

Para os seguintes fragmentos de código *assembly* RISC-V, indique um mapeamento entre registos e variáveis e obtenha as expressões simbólicas implícitas. Assuma que os acessos com deslocamento correspondem a acessos a variáveis do tipo vetor, em que um registo armazena o endereço base do vetor. Por exemplo, o acesso com a instrução assembly `lw x5, 4(x6)` vai buscar o segundo elemento de um vetor cujo endereço base está armazenado no registo `x6`.

a)
```
add     x5,x5,x6
add     x5,x5,x7
add     x5,x5,x28
add     x5,x5,x29
```

b)
```
lw      x5, 4(x6)
```

c) Assuma que `x6` contém o endereço base da sequência `A[]`.

```
addi    x6, x6, -20
slli    x30, x5, 3
add     x6, x6, x30
lw      x5, 8(x6)
```

▸ **Exercício 9**

Considerando os seguintes valores iniciais dos registos da arquitetura RISC-V:

`x5 = 0x55555555` `x6 = 0x89ABCDEF`

determine o valor de `x7` após a execução das seguintes instruções:

a)
```
slli    x7, x5, 4
or      x7, x7, x6
```

b)
```
slli    x7, x5, 4
andi    x7, x7, -1
```

c)
```
srli    x7, x5, 3
andi    x7, x7, 0x0EF
```

▸ **Exercício 10**

As arquiteturas de processadores RISC-V RV32I implementam apenas instruções inteiras muito simples. Este exercício aborda exemplos de instruções hipotéticas mais complexas, que podem ser encapsuladas como rotinas assembly.

a) Considere a instrução hipotética `abs`, que coloca num registo o valor absoluto de outro.

`abs x5, x6` é equivalente a `x5 ← |x6|`

Apresente uma sequência de instruções RISC-V que efetue esta operação.

b) Faça o mesmo para a instrução `sgt`, onde `sgt x5, x6, x7` é equivalente a `se x6 > x7 então x5 ← 1, caso contrário x5 ← 0`.

▸ **Exercício 11**

Considere o fragmento de código assembly RISC-V apresentado abaixo:

```
L1:     sw    x28,0(x29)
        slli  x28,x28,4
        addi  x29,x29,4
        bne   x28, x0, L1
```

Assuma os seguintes valores iniciais dos registos `x28` e `x29`:

`x28 = 0x12345678` `x29 = 0x000007D0`

a) Explique que posições de memória são modificadas durante a execução deste fragmento de código.

b) Assumindo que o código começa no endereço `0x00400000`, apresente o código-máquina (em formato hexadecimal) deste fragmento de código.

▸ **Exercício 12**

Numa região de memória (cujo endereço inicial é definido pelo conteúdo do registo `x29`) encontra-se uma sequência de inteiros não nulos (cada um codificado em 32 bits). A sequência termina com um valor nulo (*i.e.*, `0x00000000`).

Escreva um fragmento de código *assembly* RISC-V que determine o número de valores da sequência. O valor terminador da sequência não é incluído nesta contagem. O resultado deve ser armazenado no registo `x5`.

▸ **Exercício 13**

Considere os seguintes fragmentos de código *assembly* RISC-V.

Fragmento 1:

```
L1:   beq   x5, x0, L2
      addi  x7, x7, 2
      addi  x5, x5, -1
      beq   x0, x0, L1
L2:   # ...
```

Fragmento 2:

```
L1:   addi  x3, x3, 0xA
L2:   addi  x7, x7, 2
      addi  x3, x3, -1
      bne   x3, x0, L2
      addi  x5, x5, -1
      bne   x0, x5, L1
L3: # ...
```

a) Assumindo os valores iniciais dos registos: `x5 = 0x0000000A`, `x7 = 0x00000000` e `x3 = 0x00000000`, determine, para cada fragmento de código, o valor final do registo `x7`.

b) Assuma agora que `x5 = N` (com `N > 0`). Determine, para cada fragmento, o número de instruções executadas em função de `N`.

▸ **Exercício 14**

Escreva um fragmento de código para determinar se duas sequências de inteiros (32 bits) têm o mesmo conteúdo. As sequências têm 10 elementos. Use a seguinte atribuição de registos:

```
endereço base da sequência A → x10
endereço base da sequência B → x11
```

O resultado, a armazenar em `x10`, deve ser $0x00000001$ se as duas sequências forem idênticas, ou $0x00000000$ caso contrário.

▸ **Exercício 15**

Escreva um fragmento de código assembly RISC-V para determinar quantos números ímpares estão contidos numa sequência de 50 inteiros (números de 32 bits). Assuma que o endereço base da sequência está armazenado no registo `x10`. O resultado deve ser guardado no registo `x10`.

▸ **Exercício 16**

Escreva um programa que efetue várias tarefas sobre sequências (vetores) de inteiros de 32 bits (words) armazenados em memória. O programa deve ser construído em torno de um conjunto de funções (subrotinas), cada uma das quais efetuando uma tarefa específica. Assuma que o programa recebe como argumentos o endereço da sequência em memória no registo `x10` e o número de elementos da sequência, ou o seu comprimento, no registo `x11`.

Note que o uso dos registos deve seguir as convenções habituais quanto aos registos preservados e restaurados nas chamadas de funções. Teste as suas soluções com o simulador `RARS`.

a) Somar todos os elementos de uma sequência.

b) Determinar o número de elementos ímpares de uma sequência.

c) Determinar quantos elementos de uma sequência são iguais ou maiores do que um dado valor. Aqui, use o valor no registo `x12` como valor limiar.

d) Determinar se duas sequências do mesmo comprimento são iguais.

e) Alterar todos os elementos de uma sequência que estejam fora do intervalo $[a; b]$ para o valor a (se o elemento for menor do que a) ou para o valor b (se o elemento for maior do que b).

▸ **Exercício 17**

A subrotina `setzero` coloca a zero todos os elementos ímpares de uma sequência de números sem sinal e devolve a soma desses elementos. Os parâmetros da subrotina são, por ordem, os seguintes: 1) endereço base da sequência; 2) número de elementos da sequência.

Complete o código da subrotina tendo em conta as convenções habituais sobre o uso dos registos.

```
setzero:
        mv      t0, zero
L1:     beq     a1, zero, final         # parar?
        _______ t1, 0(_______)          # obter elemento
        andi    t2, t1, 1
        beq     _______, zero, L2
        _______ zero, 0(_______)        # colocar a zero
        add     t0, t0, t1
L2:     addi    a0, a0, _______         # preparar a proxima iteração
        addi    a1, _______, _______
        j       L1
final:  mv      _______,t0
        ret
```

▸ **Exercício 18**

O conjunto de instruções base do RISC-V não inclui instruções de rotação. Assim, é necessário implementar a rotação (também conhecida como deslocamento cíclico) com múltiplas instruções básicas. Neste exercício, pede-se então que desenvolva duas subrotinas que recebem dois argumentos, nos registos `a0` e `a1`, respetivamente. No argumento `a0` está o valor a rodar e no argumento `a1` a quantidade numérica de rotação, que se assume ser menor ou igual a 32. O resultado deve ser guardado no mesmo registo do argumento de entrada, i.e., `a0`. Se o argumento `a1` for maior do que 32, a função não efetua qualquer rotação.

▸ **Exercício 19**

Considere o seguinte programa, que chama duas subrotinas (`POLI` e `QUAD`).

```
.data
num: .word 7

.text
la   t0, num
lw   a0, 0(t0)
call POLI
li   a7, 10
ecall       # parar
```

```
POLI: ...           # ***
    mv    s0, a0    # <1>
    call  QUAD
    mv    t0, a0
    li    a0, 3
    mul   s0, s0, a0
    add   a0, t0, s0
    addi  a0, a0, 1
    ...           # ***
    ret           # <2>

QUAD: mul  a0, a0, a0
      ret
```

a) Analise o programa e descreva o que a subrotina `POLI` calcula.

b) Complete o código com as instruções que considera necessárias nos locais marcados com ***.

c) Indique o conteúdo da pilha do sistema imediatamente antes da execução das instruções marcadas com <1> e <2>.

▸ **Exercício 20**

Escreva e teste subrotinas envolvendo vetores com um ou mais elementos, de acordo com as especificações abaixo. Assuma que o endereço base do vetor e o número de elementos são passados à subrotina por esta ordem (se houver mais argumentos, devem ser passados a seguir aos anteriores).

a) Determinar o valor máximo de um vetor com dados do tipo `byte`.

b) Determinar o valor máximo de um vetor com dados do tipo `unsigned word`.

c) Determinar o valor mínimo de um vetor com dados do tipo `halfword`.

d) Determinar o número de valores de um vetor que pertencem ao intervalo $[a; b]$, assumindo dados do tipo `word`.

e) Inverter a ordem dos elementos de um vetor de words com sinal.

▸ **Exercício 21**

Em `C/C++`, as strings são representadas por vetores de carateres terminados por um carácter nulo. Isto significa que uma string é um vetor de char em que o último char é o carácter especial chamado "nulo" (um byte com o valor `0x00`, representado como \0). Por exemplo, a string "FSC"seria armazenada em memória (a partir do endereço de exemplo `0x00001000`) como:

	Byte			
	0 (LSB)	1	2	3 (MSB)
0x00001000	'F'	'S'	'C'	\0

Todos os carateres representáveis (números, letras, símbolos, etc.) são codificados segundo uma norma (ver `https://www.asciitable.com/`). Por exemplo, "1" é armazenado como o byte `0x31`.

Neste problema, pede-se que escreva duas subrotinas RISC-V, nomeadamente "atoi"(ASCII to integer) e "strlen"(string length). A subrotina "atoi"converte uma string ASCII de dígitos decimais, terminada por nulo, no seu valor inteiro. Por exemplo, a string "29567225"ocupa 9 bytes em memória, se cada dígito decimal for representado como um char de 1 byte, mas pode em vez disso ser armazenada como um inteiro de 4 bytes. A subrotina "strlen"calcula o comprimento de uma string, devolvendo um inteiro não negativo com o número de carateres da string.

Assuma que a1 contém o endereço inicial da string terminada por nulo. No caso da subrotina "atoi", pode assumir que a string pode conter o primeiro carácter opcional '-' (i.e., o valor é negativo) ou '+' (indicando que o valor é positivo), seguido apenas de carateres 0 a 9 (códigos ASCII `0x30` a `0x39`). O número convertido deve ser armazenado no registo de retorno `a0`. Se aparecer na string um carácter que não seja um dígito, a função deve devolver -1 em `a0`.

▸ **Exercício 22**

Linguagens mais antigas, como o FORTRAN77, incluíam a construção do *GOTO calculado*. Aqui, uma lista de etiquetas de código é associada a uma expressão de valor inteiro. Com base no valor dessa expressão, o controlo é transferido para o código que corresponde a uma das etiquetas, aqui ilustrado pela presença de uma instrução `CONTINUE`. A expressão tem de avaliar para um valor inteiro entre 1 e o número de etiquetas na instrução, digamos K. Se o valor estiver fora desta gama, *i.e.*, se for menor ou igual a zero ou maior do que K, a instrução não tem efeito e o controlo é transferido para a instrução a seguir ao `GOTO`. O fragmento de código apresentado abaixo mostra um exemplo genérico com 4 etiquetas e uma expressão simples `N`.

```
      ...
      GO TO (10, 20, 30, 40), N
      ...
10    CONTINUE
      ...
20    CONTINUE
      ...
40    CONTINUE
      ...
```

Address	Contents
0x0100 1000	0x0000 0004
0x0100 1004	address of label 10
0x0100 1008	address of label 20
0x0100 100C	address of label 30
0x0100 1010	address of label 40
0x0100 1014	address of next instr.

Pede-se que conceba uma estratégia de geração de código em assembly para esta construção de `GOTO` calculado, assumindo que o compilador cria um vetor de endereços correspondentes às várias etiquetas. Este vetor, a guardar em memória, terá as etiquetas de código correspondentes às etiquetas do código na construção `GOTO`, bem como a etiqueta da instrução a seguir ao `GOTO`, que tem de ser usada no caso de a expressão avaliar para um valor fora da gama de valores. A estrutura deste vetor em memória, tal como gerada pelo compilador e guardada na fase de carregamento do programa executável, é a mostrada acima, à direita. Aqui, na posição 0 (índice 0 do vetor) temos o comprimento do vetor e, na última posição, temos a etiqueta correspondente à próxima instrução do código a executar se a expressão na instrução `GOTO` não estiver na gama de 1 a K (4 neste exemplo). Para efeitos deste exercício, pode assumir que o endereço do vetor da tabela do `GOTO` está armazenado em memória com um deslocamento de 64 bytes em relação ao valor do registo apontador de pilha.

▸ **Exercício 23**

Muitas linguagens de alto nível permitem o uso de vetores de tamanho variável e multidimensionais. A figura abaixo, uma função C99 simples, permite a declaração de uma variável local `vals`, que é um vetor unidimensional com n elementos, mas em que o valor de n só é conhecido em tempo de execução.

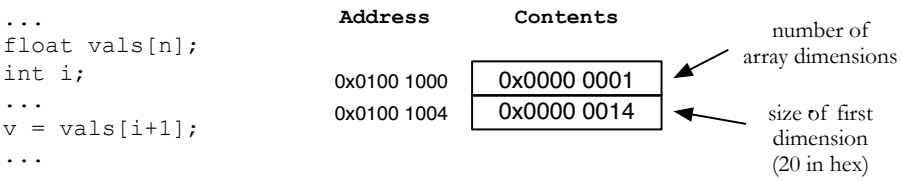
```
float read_and_process(int n){
    float vals[n];

    for (int i = 0; i < n; ++i)
        vals[i] = read_val();

    return process(n, vals);
}
```

Esta funcionalidade, embora muito conveniente para o programador, torna a geração de código para o acesso aos elementos do vetor mais elaborada, pois o número e o tamanho de cada dimensão

dos vetores só são conhecidos em tempo de execução. Para suportar esta funcionalidade da linguagem, os compiladores recorrem à noção de *dope vector*, que é um vetor de tempo de execução que inclui o número e o tamanho de cada dimensão do vetor, bem como o seu endereço base. Desta forma, sempre que o programador inclui um acesso ao vetor, o compilador traduz o acesso para uma sequência de acessos a este *dope vector*, que calcula em tempo de execução o endereço do elemento pretendido e o carrega ou armazena. No exemplo abaixo, mostramos o dope vector para a variável `vals`, assumindo que a função `read_and_process` foi invocada com o valor inteiro 20.

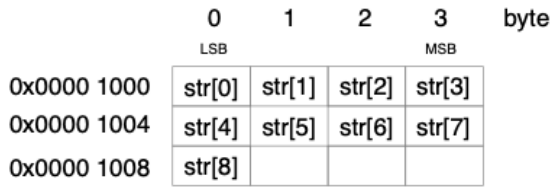


Neste exercício, pede-se que conceba e mostre como gerar código assembly para a simples instrução de atribuição `v = vals[i+1]` mostrada acima (à esquerda), assumindo que o compilador usa um dope vector para o acesso ao vetor `vals`, como descrito acima (à direita). Pode assumir que o endereço do dope vector está armazenado em memória com um deslocamento de 64 bytes em relação ao valor do registo apontador de pilha.

▸ **Exercício 24**

O *hashing* é uma operação fundamental para a aleatorização de acessos a estruturas de dados de alto nível (*e.g.*, tabelas de dispersão) e na ordenação (*e.g.*, determinação de um elemento pivô no quicksort). Uma função de hashing boa e rápida é, portanto, de grande importância. Neste exercício, pede-se que desenvolva uma implementação em linguagem assembly de um algoritmo de hashing simples mas eficaz. Este algoritmo, a implementar como uma função, recebe como entrada o endereço de uma string de carateres, a string `str` (armazenada em memória), e o seu comprimento `len`, como o número de carateres desta string. O algoritmo pode ser descrito pelo código C abaixo, onde o resultado do hashing é o valor da variável `h`. Aqui, usamos uma variável auxiliar `i` para indexar os elementos da string `str` e o operador ^, que denota um OU exclusivo (XOR) bit a bit.

```
1 h = 5381;
2 i = 0;
3 while (len > 0) {
4     h = h + (h << 5);
5     h = h ^ s[i];
6       i = i + 1
7     len = len - 1;
8 }
```



Note que, na sua implementação, tem de garantir que, quando faz o XOR, ambos os argumentos são inteiros de 32 bits, pelo que tem de converter o carácter de uma quantidade de 8 bits para um valor de 32 bits. Além disso, como a string está armazenada em memória de forma compacta, cada acesso à memória vai efetivamente buscar 4 carateres consecutivos da string. Não tão simples é o facto de a string estar armazenada num formato big-endian. Aqui, `str[0]` está armazenado nos 8 bits mais

significativos da primeira word de 32 bits, correspondendo assim ao primeiro endereço de byte. A figura abaixo apresenta a disposição de uma string de 9 elementos.

```
lb   t,off(b)  # $t <- sign-extended byte
               # from memory address b+off
               # b is a base register.
               # off is 16-bit two's complement.

lbu  t,off(b)  # $t <- zero-extended byte
               # from memory address b+off
               # b is a base register.
               # off is 16-bit two's complement.
```

Na sua implementação, pode assumir que ambos os argumentos (o endereço de str e o valor de `len`) estão carregados nos registos do processador `a0` e `a1`, respetivamente, e que o resultado da função deve ser armazenado no registo `t0`. Sugerimos o uso de duas instruções que permitem carregar um byte específico de um endereço de memória. O endereço do byte é calculado em tempo de execução, somando um deslocamento a um registo base (tal como com as instruções load word e store word). As instruções diferem na forma como o byte de 8 bits é colocado no registo de 32 bits.

A instrução `lb` carrega o byte da memória para os oito bits de ordem inferior do registo. Estes são os bits 0-7 do registo. Depois, copia o bit 7 para os bits 8-31 do registo (todos os bits à esquerda do bit 7). Outra forma de dizer isto é que a instrução lb carrega o registo com uma versão de 32 bits com extensão de sinal do byte no endereço designado. Use a instrução lb quando o byte for considerado um inteiro com sinal de 8 bits na gama −128 ... + 127 e quiser uma versão de 32 bits do mesmo inteiro.

A instrução `lbu` preenche os bits 8-31 do registo com zeros. Use esta instrução quando o byte for considerado um carácter ASCII ou um inteiro sem sinal de 8 bits.

▸ **Exercício 25**

Desenvolva uma rotina assembly RISC-V que recebe duas entradas nos registos `a0` e `a1`, a interpretar como valores de vírgula flutuante IEEE-754 de precisão simples, e determina o sinal do resultado da multiplicação (sem calcular explicitamente a multiplicação) destes dois números. Se o resultado da multiplicação for negativo, a subrotina deve devolver o valor `0x00000001` no registo `a0`; caso contrário (quando o resultado é positivo ou nulo), o resultado no registo a0 deve ser `0x00000000`.

▸ **Exercício 26**

Desenvolva uma rotina assembly RISC-V que verifica se a sua entrada no registo `a0`, a interpretar como um valor de vírgula flutuante IEEE-754 de precisão simples, é um NaN (not-a-number). Se for o caso, a rotina deve devolver o valor `0x00000001` no registo `a0`; caso contrário, o resultado no registo `a0` deve ser `0x00000000`.

▸ **Exercício 27**

Desenvolva uma rotina assembly RISC-V que, dado um valor de vírgula flutuante IEEE-754 de precisão simples normalizado no registo `a0`, devolve no mesmo registo o menor número normalizado maior do que o original; se esse número não existir (*i.e.*, não tiver representação normalizada), a rotina devolve o argumento original inalterado.

▸ **Exercício 28**

Escreva e teste um programa que use uma subrotina que efetua a adição de vetores ($R \leftarrow A + B$). Se ocorrer overflow ao somar dois valores, use como resultado o maior ou o menor valor representável (saturação), consoante a soma seja positiva ou negativa.

Parte IV

Arquitetura de Processadores

7 Organização do Processador

Resumo dos sinais de controlo da ALU

A ALU usada nas implementações do CPU tem 4 linhas de controlo, duas entradas de dados de 32 bits (A e B) e uma saída de resultado de 32 bits. As operações suportadas são apresentadas na tabela seguinte.

Linhas de controlo da ALU	Função
0000	AND
0001	OR
0010	adição
0110	subtração
0111	se $A < B$ então 1 senão 0
1100	NOR

Para as implementações discutidas nas aulas, as linhas de controlo da ALU são definidas pela unidade de controlo da ALU (`ALU control unit`), que tem as seguintes entradas:

- bits 30 e 14–12 da instrução;
- o barramento de dois bits `ALUOp`.

Instrução	ALUOp[1:0]	Ação da ALU	Controlo da ALU
lw	00	adição	0010
sw	00	adição	0010
beq	01	subtração	0110
add	10	adição	0010
sub	10	subtração	0110
and	10	AND	0000
or	10	OR	0001

7.1 Exercícios Resolvidos

▸ **Exercício 1**

Considere a implementação de ciclo único do CPU apresentada na figura 7.1. Determine o valor de todos os sinais de controlo durante a execução da seguinte instrução:

```
add     x3, x8, x11
```

Que componentes desempenham funções úteis?

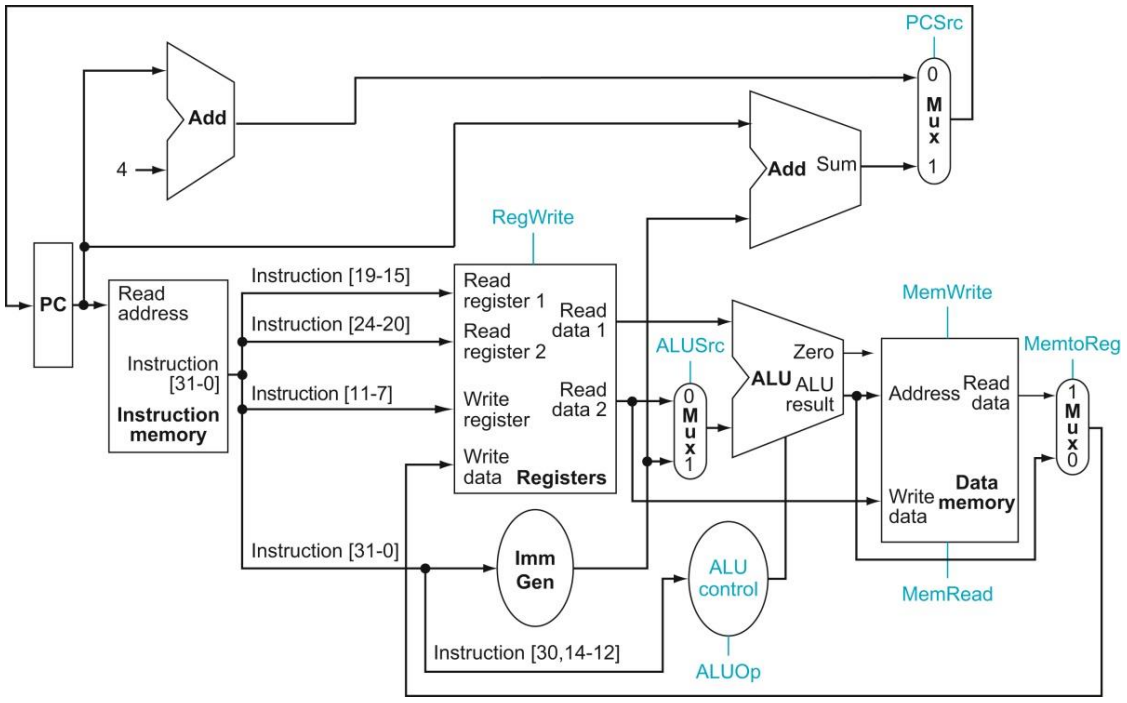


Figura 7.1: CPU de ciclo único com multiplexadores e sinais de controlo

Solução

Os valores dos sinais de controlo são determinados pela operação realizada, que estabelece a forma como os recursos do CPU são usados.

A instrução `add` usa dois registos como fonte dos operandos e um como destino do resultado. Como a instrução altera o conteúdo do registo (para guardar o resultado da adição), temos de ter `RegWrite=1`. Como o valor que é armazenado no registo vem da ALU e não da memória, temos `MemtoReg=0`.

Esta instrução não acede à memória, nem para leitura nem para escrita, pelo que `MemRead=0` e `MemWrite=0`. Esta instrução não afeta o fluxo de instruções, pelo que `PCSrc=0`.

A instrução `add` usa a ALU para efetuar a adição de dois operandos provenientes do banco de registos. Um operando da ALU vem sempre do banco de registos; a origem do segundo é determinada pelo sinal `ALUSrc`. Como, neste caso, o segundo operando também vem do banco de registos, temos de ter `ALUSrc=0`.

A operação da ALU é controlada pelo sinal de 2 bits `ALUOp`. Como se trata de uma instrução do tipo R, temos $\texttt{ALUOp=10}_2$.

Todos os módulos desempenham funções úteis, exceto a memória de dados, o módulo `ImmGen` e o

somador usado para calcular o destino dos saltos relativos (módulo `Add` acima da ALU).

▸ **Exercício 2**

Suponha que a unidade de controlo usada com o CPU da figura 7.1 tem um defeito de fabrico que faz com que um determinado sinal de controlo fique fixo em 0 (uma falha do tipo *stuck-at-0*). Indique, justificando, que instruções deixam de funcionar corretamente, considerando que o defeito ocorre em:

a) `RegWrite` b) `ALUOp[1]`

Solução

Para que uma instrução não funcione como esperado, é necessário que o sinal, em funcionamento correto, assuma um valor diferente do imposto pelo defeito de fabrico.

a) Como o defeito de fabrico impõe sempre `RegWrite=0`, todas as instruções que requerem `RegWrite=1` funcionarão incorretamente. As instruções com `RegWrite=1` são as que alteram o conteúdo de um registo: as instruções do tipo R e a instrução `lw`.

b) O sinal `ALUOp[1]` é igual a 1 apenas para as instruções lógico-aritméticas. Como o defeito impõe `ALUOp[1]=0`, a ALU comporta-se, nestas instruções, como se estivesse a efetuar um cálculo de endereços, i.e., a efetuar uma adição. Portanto, o defeito de fabrico impede que todas as instruções do tipo R, exceto a `add`, funcionem corretamente: `sub`, `and` e `or`.

▸ **Exercício 3**

Como referido anteriormente, os processadores podem ter defeitos de fabrico. Suponha que pretende detetar se um CPU tem um determinado defeito, procedendo da seguinte forma:

1. preencher o PC, os registos e a memória de instruções com valores devidamente selecionados;
2. executar *uma única* instrução;
3. ler os valores do PC, dos registos e da memória de dados.

Os valores lidos no terceiro passo são examinados para determinar se uma dada falha está presente ou não.

a) Apresente um teste para determinar se o bit 12 da saída da memória de instruções está sempre a 0 (*stuck-at-0*).

b) Repita a alínea anterior para uma falha *stuck-at-1* (saída sempre a 1). É possível usar um único teste para ambos os casos? Porquê?

Solução

Para detetar um defeito, é necessário provocar uma situação em que o sinal afetado assumiria normalmente o valor complementar do imposto pelo defeito.

a) Este defeito afeta o bit 12 das instruções lidas da memória. Este é o bit menos significativo do campo `funct3`. É usado como entrada dos módulos `ImmGen` e `ALU control`. Por exemplo, as instruções `and` e `or` diferem apenas neste bit. Assim, uma solução possível é:

```
and x6, x5, zero
```

Coloque em `x5` qualquer valor diferente de zero. Na ausência do defeito, o valor de `x6` será zero depois de o CPU executar a instrução. O defeito transformará a instrução `and` numa instrução `or`; portanto, o valor de `x6` será igual ao valor de `x5`.

b) Usando um raciocínio análogo, podemos usar: `or x6, x5, zero`.

Na ausência do defeito, o valor de `x6` será igual ao valor de `x5` depois de o CPU executar a instrução. O defeito transformará a instrução `or` numa instrução `and`; portanto, o valor de `x6` será zero.

Não é possível testar simultaneamente a presença de ambos os defeitos, porque isso exigiria usar uma única instrução para impor condições opostas no local da falha.

▸ **Exercício 4**

Assuma que o processador da figura 7.1 lê da memória a seguinte instrução:

```
0000 0000 1100 0011 0010 0010 1000 0011
```

a) Determine o valor da saída do módulo `ImmGen`.

b) Determine os valores das entradas da unidade de controlo da ALU.

c) Qual é o valor do PC depois de executar a instrução?

Assuma agora que o conteúdo dos registos é o seguinte:

x0	x1	x2	x3	x4	x5	x6	x7	x12
0	1	4	3	-4	5	192	8	1

d) Determine os valores das saídas de cada multiplexador.

e) Determine os valores das entradas da ALU e dos dois somadores.

f) Determine os valores de todas as entradas do banco de registos.

Solução

A solução abaixo usa a figura 7.1 como referência.

a) O componente `ImmGen` recebe parte da instrução como entrada e produz um valor de 32 bits, obtido por extensão de sinal de um valor construído a partir de campos da instrução.

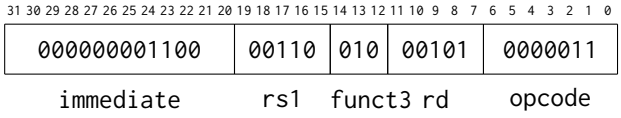
Neste caso, `Instr[6:0]=0000011`, pelo que temos uma instrução de `load` (não é necessário saber exatamente qual). Este tipo de instruções tem o valor imediato codificado nos bits 31–20. O valor do campo é 000000001100_2 ($=12_{10}$), pelo que o valor da saída do `ImmGen` é:

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100_2$ ou `0x0000000C`

b) Como a instrução é um `load`, a ALU tem de ser usada para calcular o endereço efetivo (somando o conteúdo de um registo ao valor imediato codificado na instrução), pelo que `ALUOp=00`.

c) Como uma instrução `load` não altera o fluxo de execução, `PC ← PC + 4`.

d) Para responder às próximas questões, é necessário descodificar a instrução completa:



Assim, `rs1=6` e `rd=5`; como `funct3=010`$_2$, a instrução é `lw`. A instrução completa é: `lw x5, 12(x6)`.

Como a instrução usa a ALU para calcular a soma `x6+12`, o multiplexador na entrada da ALU seleciona a constante (após extensão de sinal): a saída deste multiplexador tem o valor 12.

A instrução `lw` transfere um valor da memória para um registo. Portanto, a saída do multiplexador controlado por `MemtoReg` tem o valor proveniente da posição de memória lida (valor armazenado no endereço 204, porque `x6`=192).

Esta instrução não altera o fluxo de instruções. Portanto, a saída do multiplexador controlado por `PCSrc` é igual a `PC+4`.

e) A entrada superior da ALU tem o valor do endereço base (registo `x6`): 192. A entrada inferior da ALU tem a constante 12 (em 32 bits).

As entradas do somador da esquerda são: `PC` e 4.

As entradas do somador da direita são `PC` e o valor imediato 12 (em 32 bits).

f) Os valores nas entradas do banco de registos são:

- `Read register 1`: valor de `rs1`: 6;
- `Read register 2`: valor de `Instr[24:20]`, que é `01100`$_2$. Note que este valor é *irrelevante*, porque o valor da saída `Read Data 2` não é usado nesta instrução;
- `Write register`: valor de `rd`: 5;
- `Write data`: valor lido da memória (endereço 204);
- `RegWrite`: 1 (sinal de controlo que ativa a escrita do novo valor no registo 5).

7.2 Exercícios Propostos

▸ **Exercício 5**

Para as seguintes instruções em código-máquina RISC-V, determine as instruções assembly correspondentes. Indique o tipo de codificação, bem como os valores de todos os campos.

a) `0xFCA2A023`

b) `0xFEC58EE3`

▸ **Exercício 6**

Considere o CPU da figura 7.1. Determine o valor de todos os sinais de controlo durante a execução das instruções indicadas abaixo. Indique também que componentes desempenham funções úteis.

a) `lw x3,64(x7)`

b) `sw x8,128(x2)`

c) `beq x9, x0, -8`

▸ **Exercício 7**

Suponha que a unidade de controlo usada com o CPU da figura 7.1 tem um defeito de fabrico que faz com que um determinado sinal de controlo tenha o valor fixo 0. Que instruções deixam de funcionar corretamente, assumindo que a falha ocorre em:

a) `PCSrc`

b) `MemWrite`

▸ **Exercício 8**

Considere a instrução `lw x8, 40(x17)`. Consulte a Fig. 7.1 ao responder às seguintes questões.

a) Represente a instrução em código-máquina.

b) Qual é o número de registo presente na entrada `Read register 1`? O valor deste registo é usado?

c) Qual é o número de registo presente na entrada `Read register 2`? O valor deste registo é usado?

d) Qual é o número de registo presente na entrada `Write register`? O valor deste registo é realmente alterado?

e) Qual é o valor dos sinais de controlo `MemRead` e `MemWrite`?

f) Repita todas as alíneas anteriores para a instrução:

```
L1: beq x11,x0, L1
```

▸ **Exercício 9**

A implementação de ciclo único da Fig. 7.1 não foi projetada para instruções do tipo I, como `addi` ou `andi`.

a) Que blocos lógicos adicionais, se alguns, são necessários para acrescentar as instruções do tipo I ao CPU apresentado na Fig. 7.1? Acrescente os blocos lógicos necessários à Fig. 7.1 e explique a sua finalidade.

b) Indique os valores dos sinais gerados pela unidade de controlo de ciclo único para a instrução `addi`. Explique o raciocínio para quaisquer sinais de controlo "indiferentes" (*don't care*).

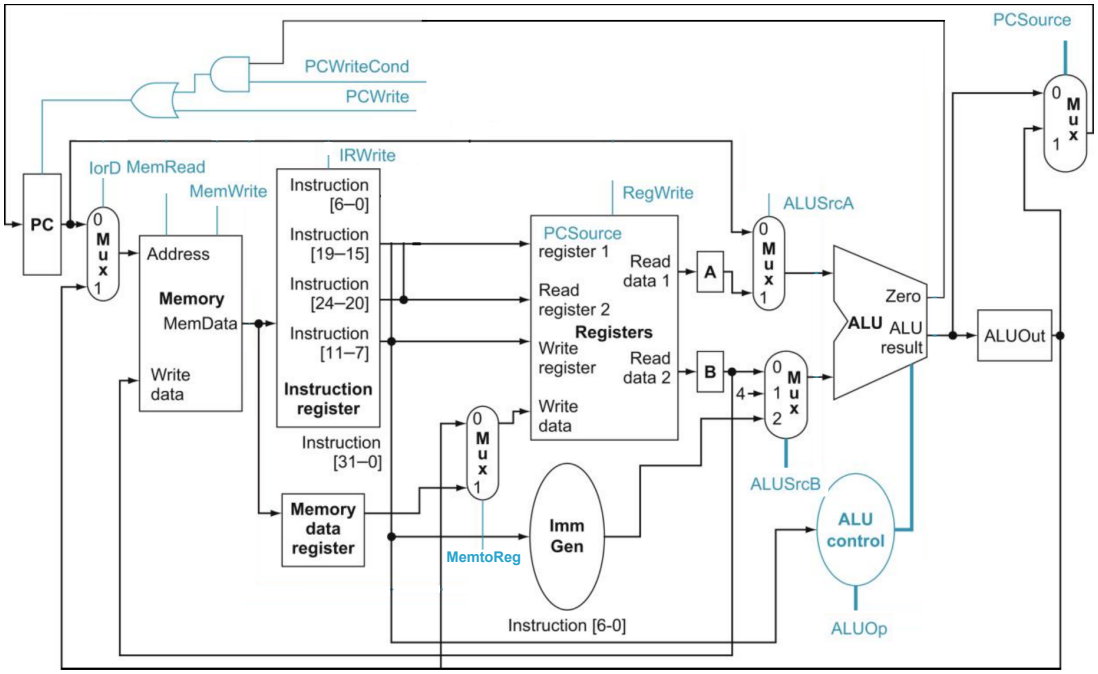


Figura 7.2: Caminho de dados para a implementação multiciclo do CPU

▸ **Exercício 10**

Considere o CPU multiciclo cujo caminho de dados é apresentado na Fig. 7.2; a máquina de estados de controlo correspondente está na Fig. 7.3. Mostre a sequência de estados que a unidade de controlo percorre ao executar o seguinte fragmento de código:

```
add  a0, a1, a0
lw   a3, 0(t1)
beq  a3, a0, L1
```

▸ **Exercício 11**

Considere o CPU multiciclo cujo caminho de dados é apresentado na Fig. 7.2 e a máquina de estados de controlo correspondente da Fig. 7.3. Assuma que o período de relógio é de 5 ns e que um programa executa 10^{12} instruções, distribuídas da seguinte forma:

ALU/Lógica	Salto	Load	Store
45 %	20 %	20 %	15 %

a) Qual é o tempo de execução do programa? Compare com o tempo de execução do mesmo programa quando é usado o CPU de ciclo único com um período de relógio de 20 ns.

b) Durante que percentagem do tempo de execução está a ALU a realizar trabalho útil?

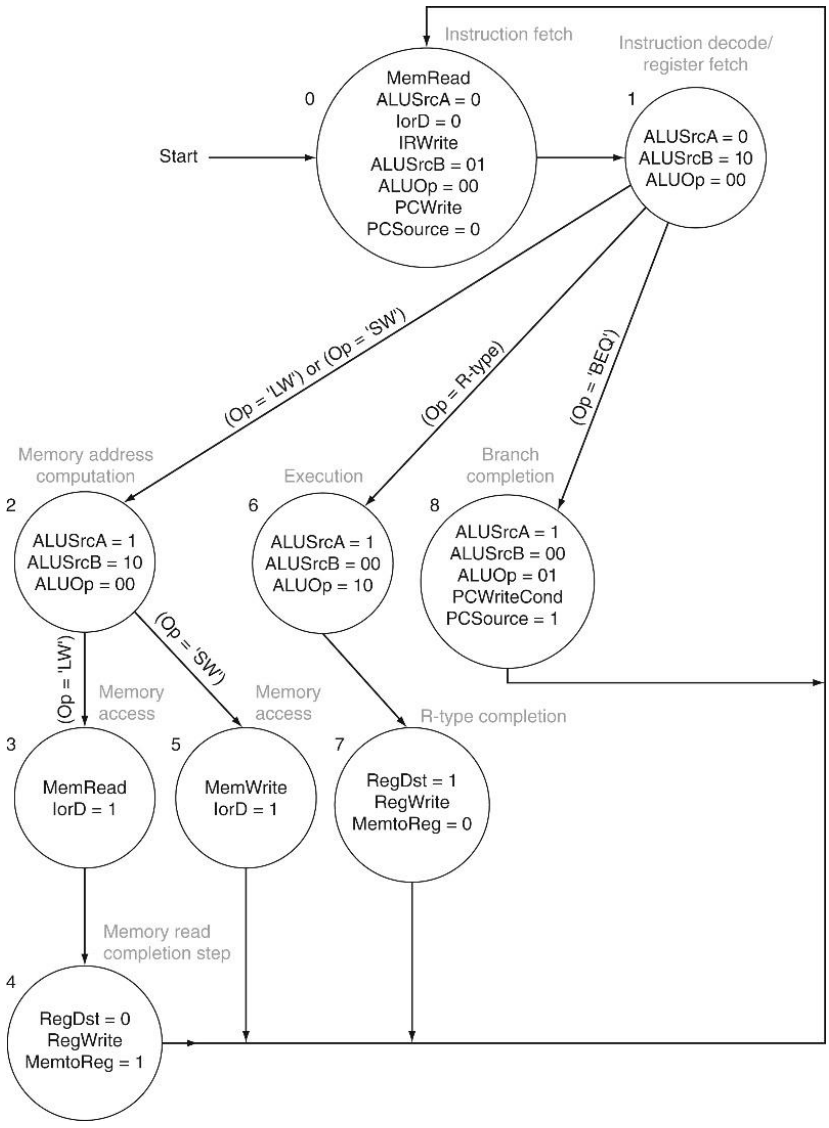


Figura 7.3: Máquina de estados do CPU multiciclo

8 Desempenho do Processador

Alguns destes exercícios foram retirados ou adaptados do livro "Computer Organization and Design – The Hardware/Software Interface", D. Hennessy e J. Patterson, 4.ª edição.

8.1 Exercícios Resolvidos

▸ **Exercício 1**

Um computador tem três classes de instruções, *A*, *B* e *C*. O CPI de cada uma delas é 1, 2 e 3, respetivamente. Um projetista está a implementar um compilador e precisa de escolher uma de duas sequências de instruções a usar nesse computador. Destas sequências conhece-se o número de instruções de cada classe, como mostrado na tabela abaixo.

Sequência de instruções	N.º de instruções por classe		
	A	*B*	*C*
1	2	1	2
2	4	1	1

a) Determine que sequência usa mais instruções.

b) Determine que sequência executa mais depressa.

c) Calcule o CPI de cada sequência.

Solução

a) Na sequência 1 são executadas 2 + 1 + 2 = 5 instruções e na sequência 2 são executadas 4 + 1 + 1 = 6 instruções.

b) O número de ciclos é dado por:

$$N_{\text{ciclos}} = \sum_i N_i \times \text{CPI}_i$$

onde N_i é o número de instruções da classe i e CPI_i é o número de ciclos por instrução dessa classe.

Sequência 1: $N_{\text{ciclos}} = 2 \times 1 + 1 \times 2 + 2 \times 3 = 10$

Sequência 2: $N_{\text{ciclos}} = 4 \times 1 + 1 \times 2 + 1 \times 3 = 9$

A sequência 2 é mais rápida, porque executa em menos ciclos de relógio.

c)

$$\text{CPI} = \frac{N_{\text{ciclos}}}{N_{\text{inst}}}$$

Sequência 1: CPI = 10/5 = 2

Sequência 2: CPI = 9/6 = 1,5

▸ **Exercício 2**

Considere um computador com três classes de instruções, *A*, *B* e *C*, para as quais o valor do CPI é 1, 2 e 3, respetivamente. Para executar neste computador, um programa é obtido através de dois compiladores diferentes, resultando no número de instruções de cada classe indicado na tabela.

Sequência de instruções	N.º de instruções por classe		
	A	*B*	*C*
Compilador 1	5×10^9	1×10^9	1×10^9
Compilador 2	10×10^9	1×10^9	1×10^9

O computador tem um relógio de 500 MHz.

a) Que sequência é mais rápida?

b) Que sequência atinge um valor de MIPS (milhões de instruções por segundo) mais elevado?

Solução

a) O tempo de execução é dado por:

$$t_{\text{exec}} = \frac{N_{\text{ciclos}}}{f_{\text{CLK}}}$$

com o número de ciclos de relógio expresso por:

$$N_{\text{ciclos}} = \sum_i N_i \times \text{CPI}_i$$

O tempo de execução da sequência produzida por cada compilador é:

$$t_{\text{execC1}} = \frac{(5 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9}{500 \times 10^6} = \frac{10 \times 10^9}{500 \times 10^6} = 20\,\text{s}$$

e

$$t_{\text{execC2}} = \frac{(10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9}{500 \times 10^6} = \frac{15 \times 10^9}{500 \times 10^6} = 30\,\text{s}$$

Dados os tempos de execução calculados, pode concluir-se que o compilador 1 gera o programa mais rápido.

b)

$$\text{MIPS} = \frac{N_{\text{instr}}}{t_{\text{exec}} \times 10^6}$$

Assim,

$$\text{MIPS}_1 = \frac{(5 + 1 + 1) \times 10^9}{20 \times 10^6} = 350$$

e

$$\text{MIPS}_2 = \frac{(10 + 1 + 1) \times 10^9}{30 \times 10^6} = 400$$

O compilador 2 atinge uma taxa de execução de instruções mais elevada.

▸ **Exercício 3**

A execução de um programa no computador A, que opera com um sinal de relógio de 2 GHz, demora 10 s. Está a ser desenvolvido um computador B, capaz de executar o mesmo programa em 6 s. Este computador B pode atingir uma frequência de relógio mais elevada do que o computador A, mas requer 1,2 vezes mais ciclos de relógio do que o computador A para executar esse programa. Determine a frequência do sinal de relógio a que B pode funcionar.

Solução

O tempo de execução de um programa com N_{instr} instruções é dado por:

$$t_{\text{exec}} = N_{\text{instr}} \times \text{CPI} \times T_{\text{CLK}}$$

A razão dos tempos de execução nos computadores A e B é:

$$\frac{t_{\text{execA}}}{t_{\text{execB}}} = \frac{N_{\text{instr}} \times \text{CPI}_A \times f_{\text{CLKB}}}{N_{\text{instr}} \times \text{CPI}_B \times f_{\text{CLKA}}} = \frac{10}{6}$$

Como $\text{CPI}_B = 1{,}2 \times \text{CPI}_A$, obtemos $f_{\text{CLKB}} = f_{\text{CLKA}} \times 1{,}2 \times \frac{10}{6} = 2 \times f_{\text{CLKA}}$, pelo que $f_{\text{CLKB}} = 4\,\text{GHz}$.

▸ **Exercício 4**

Considere dois computadores, A e B, que suportam o mesmo conjunto de instruções. O período do sinal de relógio dos computadores A e B é de 250 ps e 500 ps, respetivamente, e o número de ciclos de relógio consumidos por instrução (CPI) é 2 no computador A e 1,2 no computador B.

Qual dos computadores é mais rápido?

Solução

$$t_{exec_A} = N_{instr} \times 2 \times 250 = 500 \times N_{instr}\ \text{ps}$$

e

$$t_{exec_B} = N_{\text{instr}} \times 1{,}2 \times 500 = 600 \times N_{\text{instr}}\text{ps}$$

Assim,

$$\frac{t_{\text{execB}}}{t_{\text{execA}}} = \frac{6}{5} = 1{,}2$$

O computador A é o mais rápido.

▸ **Exercício 5**

Um dado programa é executado em dois computadores diferentes, *A* e *B*, resultando nos seguintes valores medidos:

Parâmetro	A	B
N.º de instruções	10^7	8×10^6
Frequência	4 GHz	4 GHz
CPI	1	1,1

a) Que computador tem o valor de MIPS mais elevado?

b) Que computador é mais rápido?

Solução

a)

$$\text{MIPS} = \frac{N_{instr}}{t_{exec} \times 10^6} = \frac{N_{instr}}{N_{instr} \times \text{CPI} \times T_{CLK} \times 10^6} = \frac{f_{CLK}}{\text{CPI} \times 10^6}$$

Assim,

$$\text{MIPS}_A = \frac{4 \times 10^9}{1 \times 10^6} = 4 \times 10^3$$

e

$$\text{MIPS}_B = \frac{4 \times 10^9}{1{,}1 \times 10^6} \approx 3636$$

O computador A tem o valor de MIPS mais elevado.

b)

$$t_{exec} = N_{instr} \times \text{CPI} \times T_{CLK} = \frac{N_{instr}}{\text{MIPS} \times 10^6}$$

$$t_{exec_A} = \frac{10^7}{4 \times 10^3 \times 10^6} = 2{,}5\,\text{ms}$$

$$t_{exec_B} = \frac{8 \times 10^6}{3636 \times 10^6} \approx 2{,}2\,\text{ms}$$

O computador B é o mais rápido.

▸ **Exercício 6**

Foram medidos os seguintes valores para um dado programa:

- Percentagem de instruções de vírgula flutuante (FP), exceto raiz quadrada: 25 %;
- Percentagem de operações de raiz quadrada: 2 %;
- CPI médio das instruções FP (exceto raiz quadrada): 4,0;
- CPI da instrução de raiz quadrada: 20;
- CPI médio das restantes instruções: 1,33.

Considere duas alternativas para melhorar o desempenho: reduzir o CPI da operação de raiz quadrada para 2, ou baixar o CPI de todas as instruções FP para 2,5. Compare as duas alternativas usando a equação de desempenho do CPU.

Solução

O número de instruções executadas e a frequência de relógio mantêm-se em ambos os casos. Portanto, basta comparar os valores de CPI.

$$\text{CPI}_1 = 0{,}25 \times 4 + 0{,}02 \times 2 + (1 - 0{,}25 - 0{,}02) \times 1{,}33 \approx 2{,}01$$

$$\text{CPI}_2 = 0{,}25 \times 2{,}5 + 0{,}02 \times 20 + (1 - 0{,}25 - 0{,}02) \times 1{,}33 \approx 2{,}00$$

Os valores são muito semelhantes, mas a segunda alternativa tem um tempo de execução mais curto e, portanto, um melhor desempenho.

▸ **Exercício 7**

Um programa gasta 30 % do tempo de execução em adições de vírgula flutuante, 25 % em multiplicações de vírgula flutuante e 10 % em divisões de vírgula flutuante.

Para a nova geração deste processador, a equipa propõe três possíveis melhorias, mutuamente exclusivas, à unidade de vírgula flutuante, que tornam:

1. o somador duas vezes mais rápido;
2. o multiplicador três vezes mais rápido;
3. o divisor dez vezes mais rápido.

Que alternativa proporciona a maior melhoria de desempenho?

Solução

A lei de Amdahl pode ser expressa por:

$$S_{\text{total}}(f) = \frac{1}{(1 - f) + \frac{f}{s}}$$

onde f é a fração de tempo originalmente gasta nas operações melhoradas e s é o seu ganho (*speedup*).

Aplicando a lei de Amdahl às três situações descritas, podemos então quantificar o aumento de desempenho:

1. $f = 0{,}3$ e $s = 2 \quad \rightarrow \quad s_{\text{total}}(f) = \frac{1}{0{,}7+0{,}3/2} = 1{,}18$
2. $f = 0{,}25$ e $s = 3 \quad \rightarrow \quad s_{\text{total}}(f) = \frac{1}{0{,}75+0{,}25/3} = 1{,}20$
3. $f = 0{,}1$ e $s = 10 \quad \rightarrow \quad s_{\text{total}}(f) = \frac{1}{0{,}9+0{,}1/10} = 1{,}10$

O maior ganho de desempenho é obtido melhorando o multiplicador (alternativa 2). Pode também concluir-se que tornar o divisor muito mais rápido conduz ao menor aumento de desempenho, devido à baixa ocorrência de divisões de vírgula flutuante (10 %). De facto, mesmo tornando o divisor infinitamente mais rápido, o ganho global seria de 1,11.

▸ **Exercício 8**

A execução de um programa num computador demora 1000 s, dos quais 75 % são gastos em multiplicações e divisões. Pretende-se reprojetar o computador com hardware mais rápido para efetuar as operações de multiplicação e divisão.

Calcule quanto mais rápidas têm de ser as operações de multiplicação e divisão para que o programa seja:

a) 3× mais rápido;

b) 4× mais rápido.

Solução

a) De acordo com a lei de Amdahl, o ganho global é dado por:

$$s_{total}(f) = \frac{1}{(1-f) + \frac{f}{s}}$$

onde f é a fração de tempo originalmente gasta nas operações melhoradas e s é o seu ganho. Assim,

$$3 = \frac{1}{(1-0{,}75) + \frac{0{,}75}{s}} = \frac{1}{\frac{1}{4} + \frac{3}{4 \times s}} = \frac{4 \times s}{s+3}$$

o que dá

$$3 \times s + 9 = 4 \times s \Leftrightarrow s = 9$$

i.e., as operações devem ser 9× mais rápidas.

b) Procedendo da mesma forma que em (a):

$$4 = \frac{1}{(1-0{,}75) + \frac{0{,}75}{s}} = \frac{1}{\frac{1}{4} + \frac{3}{4 \times s}} = \frac{4 \times s}{s+3}$$

e

$$4 \times s + 12 = 4 \times s$$

que não tem solução: é impossível executar o programa 4× mais rápido melhorando apenas estas operações.

8.2 Exercícios Propostos

▸ **Exercício 9**

A tabela abaixo mostra a frequência de funcionamento de 3 processadores, o número de instruções executadas por cada um deles e o respetivo tempo de execução.

Processador	F_{CLK} (GHz)	N.º de instruções	Tempo (s)
P_1	2	20×10^9	7
P_2	1,5	30×10^9	10
P_3	3	90×10^9	9

a) Calcule o número de instruções executadas por ciclo, para cada processador.

b) Calcule a frequência a que o processador P_2 deve funcionar para que o tempo de execução das instruções seja igual ao apresentado para P_1.

c) Calcule o número de instruções a executar em P_2 de modo que o seu tempo de execução seja igual ao de P_3.

▸ **Exercício 10**

Considere que dois processadores diferentes, P_1 e P_2, implementam o mesmo conjunto de instruções e que estas se podem agrupar em quatro classes diferentes (A, B, C e D). A frequência do CPU e o valor do CPI por classe de instruções de cada implementação são apresentados na tabela.

		CPI			
Processador	F_{CLK} (GHz)	A	B	C	D
P_1	1,5	1	2	3	4
P_2	2	2	2	2	2

a) Considere um programa com 1 milhão de instruções, divididas pelas quatro classes da seguinte forma: 10 % são da classe A, 20 % são da classe B, 50 % são da classe C e 20 % são da classe D. Verifique que processador é mais rápido na execução do programa.

b) Calcule o valor do CPI médio de cada processador.

c) Calcule o número de ciclos de relógio necessários para executar o programa em cada processador.

▸ **Exercício 11**

Dois processadores, P_1 e P_2, implementam o mesmo conjunto de instruções de formas diferentes. P_1 funciona com um relógio de 4 GHz e P_2 funciona a 6 GHz. O número médio de ciclos de relógio de cada classe de instruções é dado pela tabela abaixo.

Classe	CPI de P_1	CPI de P_2
A	1	2
B	2	2
C	3	2
D	4	4
E	3	4

O desempenho de pico é definido como a taxa máxima a que um computador pode executar a sequência de instruções mais favorável (para uma dada tarefa). Determine o desempenho de pico de P_1 e P_2, expresso em instruções por segundo. Comente o resultado.

▸ **Exercício 12**

A tabela seguinte mostra o número de operações de vírgula flutuante (FP) executadas em três programas diferentes e o tempo de execução em três computadores diferentes, *A*, *B* e *C*.

Programa	N.º de operações FP	Tempo de execução (s)		
		Comp. *A*	Comp. *B*	Comp. *C*
1	5×10^9	2	5	10
2	20×10^9	20	20	20
3	40×10^9	200	50	15

a) Indique que computador é o mais rápido em termos de tempo total de execução.

b) Determine quantas vezes esse computador é mais rápido do que os outros dois.

▸ **Exercício 13**

A tabela mostra o número total de instruções executadas por um programa, bem como a sua distribuição por tipo.

Total	Aritméticas	Store	Load	Salto
700	500	50	100	50

a) Assuma que a execução de instruções aritméticas demora 1 ciclo de relógio, que os loads e stores demoram 5 ciclos de relógio e que as instruções de salto demoram 2 ciclos de relógio. Calcule o tempo de execução com o processador a 2 GHz.

b) Calcule o valor do CPI do programa.

c) Se o número de instruções load for reduzido para metade, quanto mais rápido fica o programa e qual é o valor do CPI?

▸ **Exercício 14**

Um processador executa um programa constituído por vários tipos de instruções, incluindo instruções de transferência de dados. Assumindo que é possível melhorar o tempo de execução das instruções de transferência de dados por um fator de 11, calcule a percentagem do tempo de execução gasta por estas instruções de modo a melhorar o desempenho do processador por um fator de 5, i.e., de modo que o ganho global seja 5.

▸ **Exercício 15**

Ao medir o tempo de acesso ao disco, verificou-se que 80 % do tempo de execução de um dado programa num computador era gasto em acessos ao disco. Ao substituir o disco desse computador, verificou-se que o novo disco era 2× mais rápido.

a) Calcule quanto melhorou o desempenho do computador.

b) Analise qual teria sido a melhoria de desempenho se o disco tivesse um tempo de acesso desprezável.

▸ **Exercício 16**

Suponha que pretende melhorar um computador acrescentando-lhe uma unidade vetorial, que é uma unidade que implementa eficientemente instruções específicas para o processamento de vetores. Um cálculo efetuado em modo vetorial é 10 vezes mais rápido do que usando as instruções normais, não vetoriais.

a) Desenhe um gráfico que mostre o ganho global em função da percentagem do tempo de execução gasto em modo vetorial (percentagem do tempo de execução gasto na execução de instruções vetoriais, ou percentagem de vetorização).

b) Que percentagem de vetorização é necessária para atingir um ganho de 2?

c) Para a execução que corresponde a um ganho de 2 sobre o tempo de execução original, que percentagem deste novo tempo (mais curto) é consumida pela execução usando a unidade vetorial?

d) Que percentagem de vetorização é necessária para atingir metade do ganho máximo possível?

e) Assuma que se determinou empiricamente que a percentagem de vetorização é de 70 %. Partindo do princípio de que já existe uma unidade vetorial na arquitetura (i.e., já existe uma unidade vetorial que executa as instruções vetoriais 10 vezes mais depressa), os projetistas de hardware pensam que é possível aumentar ainda mais o desempenho da unidade vetorial por um fator de 2, i.e., duplicar o desempenho da unidade vetorial.

   Por outro lado, o grupo do compilador poderia tentar aumentar o uso da vetorização para aumentar o desempenho. Que aumento na percentagem de vetorização (em relação à situação atual) seria necessário para atingir o mesmo resultado que a duplicação do desempenho do hardware? Qual das alternativas é mais aconselhável?

▸ **Exercício 17**

Um processador suporta 4 classes de instruções. Durante a execução de um dado programa, a percentagem de tempo gasto em instruções de cada classe foi a seguinte:

Classe	A	B	C	D
Tempo (%)	30	20	40	10

Uma nova versão do processador executa as instruções da classe A três vezes mais depressa e as instruções da classe C quatro vezes mais depressa. Determine o ganho obtido pela nova versão.

Parte V

Soluções

Soluções dos Exercícios

1 Representação de Informação

Exercício 12

a) 100000000_2; 100_{16}

b) 11111111111_2; $7FF_{16}$

c) $11000{,}01_2$; $18{,}4_{16}$

d) A parte fracionária não tem representação finita. Usando 4 bits para a parte fracionária, a resposta é $100{,}0011_2$; $4{,}3_{16}$.

e) 16_{10}; 10_{16}

f) $4{,}125_{10}$; $4{,}2_{16}$

g) 11110_2; 30_{10}

h) 43981_{10}; 1010101111001101_2

i) $171{,}75_{10}$; $10101011{,}11_2$

j) 456_{16}

Exercício 13

a) $5{,}04_{10}$

b) A parte fracionária não tem representação finita. Usando 3 dígitos para a parte fracionária, a resposta é $16{,}667_{10}$.

c) 100111_3

d) 2341_5

e) $102{,}1_4$

f) A parte fracionária não tem representação finita. Usando 3 dígitos para a parte fracionária, a resposta é $22{,}111_6$.

Exercício 14

a) Representação inválida.

b) A parte fracionária não tem representação finita. Usando 3 dígitos para a parte fracionária, a resposta é 277,167.

c) Representação inválida.

d) Representação inválida.

e) 307_{10}

f) $12{,}25_{10}$

Exercício 15

a) 1001110_2
b) $1010{,}001_2$
c) 1001_2
d) 10101_2
e) 1111010_2
f) $11000100{,}01_2$
g) 101100_2

Exercício 16

a)

+	0	1	2
0	0	1	2
1	1	2	10
2	2	10	11

b)

×	0	1	2
0	0	0	0
1	0	1	2
2	0	2	11

c) 1102_3
d) 1022_3
e) 10120_3

Exercício 17

a) $3 = 0011_2$; $2 = 0010_2$; $-3 = 1011_2$

b) $3 + 2 = 0101_2$; $2 + (-3) = 1001_2$

c) O resultado correto (9_{10}) não pode ser representado em sinal e grandeza usando apenas 4 bits, já que a gama representável é $[-7; +7]$.

Exercício 18

Sinal e grandeza: $[-7, 7]$ e, em geral, $[-(2^{n-1} - 1); 2^{n-1} - 1]$;
complemento para 2: $[-8, 7]$ e, em geral, $[-2^{n-1}; 2^{n-1} - 1]$.

Exercício 19

a) sinal e grandeza: 00010010_2; compl. para 2: 00010010_2
b) sinal e grandeza: 00110001_2; compl. para 2: 00110001_2
c) sinal e grandeza: 10110001_2; compl. para 2: 11001111_2
d) sinal e grandeza: 10000011_2; compl. para 2: 11111101_2
e) sinal e grandeza: 11100100_2; compl. para 2: 10011100_2
f) sinal e grandeza: 01110011_2; compl. para 2: 01110011_2
g) sinal e grandeza: 11111111_2; compl. para 2: 10000001_2
h) sinal e grandeza: impossível; compl. para 2: 10000000_2

Exercício 20

a) 1010100_2

b) Não há overflow. Assumindo que os números estão ambos representados em notação de complemento para 2, o facto de a sua diferença ser menor em módulo do que qualquer um deles significa que tem sempre uma representação válida com o mesmo tamanho dos valores originais.

Exercício 21

a) 10000001_2; resultado correto (não há *overflow*).

b) 01011111_2; resultado inválido (há *overflow*).

c) 00000000_2; resultado correto (não há *overflow*).

Exercício 22

a) X sinal e grandeza: −99; compl. para 2: −29
Y sinal e grandeza: 72; compl. para 2: 72.

b) Sinal e grandeza: $\underline{1}0011011$; resultado correto (não há *overflow*).
Compl. para 2: 00101011; resultado correto (não há *overflow*).

Exercício 23

a) P_{16} =7A; Q_{10} =34

b) i) 0011100_2; resultado inválido, pois há *overflow*, uma vez que a adição requer 8 bits.

ii) 0011100_2; resultado correto, pois não há *overflow*.

Exercício 24

a) $A + B = 1000100_2 = 68_{10}$.

b) $A + B = 0100000_2 = 32_{10}$.

c) $A + B = 0100100_2 = 36_{10}$.

Exercício 25

a) $M = 01001010_2$ e $N = -92$

b) $M - N = 10100110_2$; resultado inválido, pois há *overflow*.
$N - M = 01011010_2$; resultado inválido, pois há *overflow*.

Exercício 26

a) $S = -56$; $T = 17$

b) $S = 10111000_2$; $T = 00010001_2$

c) $S + T = 10100111_2$; resultado válido, pois não há *overflow*.

Exercício 27

a) $Y = 49_{16}$; $Z = 10011110_2$

b) 01011010_2; o resultado é inválido porque há *overflow*, pois a adição de números negativos não pode ser um valor positivo.

c) 01000011_2

Exercício 28

a) $x \in [0; 31]$

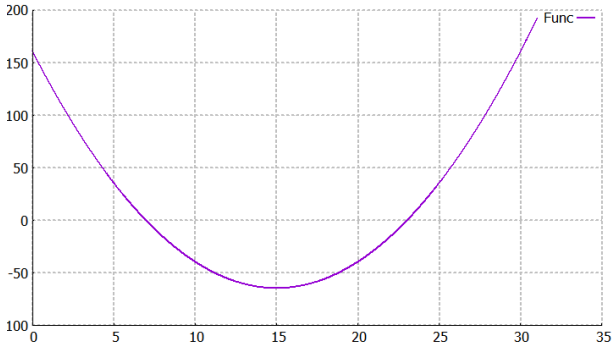
b) 192

c) -64 (nota: é necessário calcular a derivada e igualá-la a 0)

d) 9 bits

e) $192 : 011000000_2$ e $-64 : 111000000_2$ (complemento para 2, 9 bits)

Nota: apenas para ilustração, apresenta-se o gráfico da função:



Exercício 29

a) $X = -15$ e $Y = 15$

b) $X = 1111111111110001_2$ e $Y = 0000000000001111_2$.

c) $X + Y = \not{1}00000000_2$ usando a representação de 8 bits e $X + Y = \not{1}0000000000000000_2$ usando a representação de 16 bits. Ambos os resultados são válidos, pois não há *overflow*. Como esperado, estender o sinal para 16 bits não faz diferença neste cálculo.

Exercício 30

O número mínimo de dígitos do número x em base b é dado por $\lfloor \log_b(x) \rfloor + 1$.

a) 8 dígitos, 11111001_2

b) 6 dígitos, 100020_3

c) 4 dígitos, 1444_5

d) 3 dígitos, 371_8

2 Aritmética de Vírgula Flutuante

Exercício 7

a) $1{,}01100000000000000000000_2$

b) 00000100_2

c) $-22{,}0$

Exercício 8

a) $41FA0000_{16}$

b) $BF200000_{16}$

c) 00000000_{16}

d) 44805000_{16}

Exercício 9

a) $(0\ 00000001\ 00000000000000000000000)_2 \rightarrow 1{,}17549435 \times 10^{-38}$

b) $(0\ 11111110\ 11111111111111111111111)_2 \rightarrow 3{,}4028235 \times 10^{38}$

c) $(1\ 00000001\ 00000000000000000000000)_2 \rightarrow -1{,}17549435 \times 10^{-38}$

d) $(1\ 11111110\ 11111111111111111111111)_2 \rightarrow -3{,}4028235 \times 10^{38}$

Exercício 10

a) Dado que o bit de sinal é 1, esta codificação representa um número negativo. O valor do expoente verdadeiro é dado por $1 - 3 = -2$, conduzindo a um valor numérico de $-2^{-2} \times 1{,}1101_2$, ou $-2^0 \times 0{,}011101_2$. O valor decimal desta codificação binária é, portanto, $-(2^{-2} + 2^{-3} + 2^{-4} + 2^{-6}) = -2^{-6} \times (2^4 + 2^3 + 2^2 + 1) = -\dfrac{16 + 8 + 4 + 1}{64} = -\dfrac{29}{64}$.

b) Esta codificação denota um número positivo. O valor do expoente verdadeiro é $5-3 = 2$, conduzindo a um valor numérico $2^2 \times 1{,}1001_2$, ou $2^0 \times 110{,}01_2$. O valor decimal desta codificação binária é, portanto, $6 + 2^{-2} = 6{,}25$.

c) Sendo um número positivo, o bit de sinal tem de ser 0. Para determinar o valor do expoente e a mantissa, representamos o valor do número em binário e determinamos depois o expoente da representação. Assim, $\dfrac{3}{8} = \dfrac{1+2}{8} = 2^{-3} \times 11_2 = 2^{-2} \times 1{,}1_2$. Na representação em excesso de 3, o expoente é $-2 + 3 = 1 = 001_2$ e a mantissa (com o 1 à cabeça omitido, por se tratar de uma representação normalizada) é $0{,}1000_2$. A codificação completa de 3/8 neste formato é, portanto, $(0\ 001\ 1000)_2$.

Exercício 11

a) A: 42040000_{16}; B: $C0380000_{16}$

b) i) $A + B$: $41F10000_{16}$

ii) $B - A$: $C20F8000_{16}$

iii) $3 \times B$: $C10A0000_{16}$

c) $A + B = 30.125$ $B - A = -35.875$ $3 \times B = -8.625$.

Exercício 12

a) 40000000_{16} b) 42150000_{16} c) $C2250000_{16}$ d) $C29D0000_{16}$

Exercício 13

a) $01000001001110100000000000000000_2$ ou $413A0000_{16}$

b) Ambos os números são positivos, e o resultado da adição também. Assim, $Sinal_{X+Y} = 0$. Quanto às mantissas, é necessário alinhá-las de acordo com o mesmo expoente. Neste caso $Exp_X = 126$, ou seja, um expoente verdadeiro $Exp_X^{\text{verd}} = -1$, e $Exp_Y = 130$, ou seja, um expoente verdadeiro $Exp_Y^{\text{verd}} = 3$. Consequentemente, o expoente do resultado será 130, o que significa que temos de ajustar a mantissa de X (deslocando-a na direção do LSB) 4 bits antes de a somar à mantissa de Y. O diagrama abaixo apresenta a adição das duas mantissas, mostrando o 1 à cabeça, que no caso de X é deslocado. Neste caso não há transbordo no MSB, pelo que não é necessário mais nenhum ajuste do expoente do resultado.

$$\begin{array}{rr} \text{Mantissa}_X: & 0{,}000\ 1100\ 0000\ 0000\ 0000\ 0000 \\ \text{Mantissa}_Y: & \underline{+1{,}011\ 1010\ 0000\ 0000\ 0000\ 0000} \\ \text{Mantissa}_{X+Y}: & 1{,}100\ 0110\ 0000\ 0000\ 0000\ 0000 \end{array}$$

Em conclusão, a codificação hexadecimal do valor resultante é, portanto, 41460000_{16}.

c) O valor decimal de X é 0,75, pelo que $X + Y$ é 12,375, que é denotado no formato IEEE 754 de vírgula flutuante de precisão simples, em hexadecimal, como 41460000_{16}.

Exercício 14

a) $11000001001001000000000000000000_2$ ou $C1240000_{16}$

b) Esta operação é um pouco mais elaborada. Aqui X é positivo e Y é negativo. Como o módulo de Y é maior do que o de X, o resultado de $X + Y$ será negativo. Para efetuar a subtração das mantissas, os expoentes têm de ser idênticos. Para o conseguir, aumentamos o expoente de X para 130, i.e., aumentamo-lo de 2, deslocando a mantissa correspondente para a direita, na direção do LSB, 2 bits. Assim, a mantissa deslocada de X é $0011\ 0000\ 00\cdots0_2$. A mantissa de Y permanece inalterada. No entanto, para subtrair a mantissa de Y à mantissa de X, temos de obter o seu complemento para 2, para as podermos somar. O complemento para 2 da mantissa de Y é, portanto, $0101\ 1100\ 0000\ 00\cdots0_2$.

$$\begin{array}{rr} \text{Mantissa}_X: & 0011\ 0000\ 0000\ 00\cdots0 \\ \text{Mantissa}_Y: & \underline{+0101\ 1100\ 0000\ 00\cdots0} \\ \text{Mantissa}_{X+Y} & 1000\ 1100\ 0000\ 00\cdots0 \end{array}$$

Como sabemos que este é um número negativo, temos de complementar a mantissa do resultado, obtendo a codificação $0111\ 0100\ 00\cdots0_2$. Notamos agora que esta não está normalizada, i.e., não tem um bit 1 à cabeça, pelo que temos de a deslocar na direção do MSB, diminuindo assim o valor do expoente na representação para 129. As componentes da representação são, portanto, $Sinal = 1$, $Exp = 10000001_2 = 129_{10}$ e Mantissa $= 1101\ 0000\ 00\cdots0_2$, conduzindo à codificação hexadecimal $C0E80000_{16}$.

c) O valor decimal de X é 3,0, pelo que $X + Y$ é −7,25, que é denotado no formato IEEE 754 de vírgula flutuante de precisão simples, em hexadecimal, como C0E80000_{16}.

Exercício 15

a) $X = \text{C1400000}_{16}$ e $Y = \text{C1B00000}_{16}$. Ambos os números são negativos, pelo que $Sinal_X = Sinal_Y = 1$. $Exp_X = 130$ e $Exp_Y = 131$, e $\text{Significando}_X = 1{,}1000\ 00\cdots0_2 = 1.5_{10}$ e $\text{Significando}_Y = 1{,}0110\ 00\cdots0_2 = 1{,}375_{10}$.

b) $X = -12{,}0_{10}$ e $Y = -22_{10}$.

c) Neste caso particular, ambos os números são negativos, e também o será o resultado da sua adição: Sinal = 1. Deslocamos a mantissa de X para a direita para ajustar o seu expoente a 131 e efetuamos a adição das mantissas. Esta adição resulta numa codificação da mantissa $10{,}00100\cdots0_2$, que agora tem de ser ajustada para ficar normalizada, conduzindo a um valor final do expoente de $10000100_2 = 132_{10}$. A codificação final da mantissa é, assim, $1{,}000100\cdots0_2$.

d) O valor da soma de X com Y é -34_{10}, que é codificado como C2080000_{16} no formato IEEE 754 de vírgula flutuante de precisão simples, com Sinal=1, $\text{Exp} = 1000\ 0100_2 = 132_{10}$ e $\text{Significando} = 1.001\ 00\cdots0_2 = 1.0625_{10}$.

Exercício 16

$X = -256_{10}$ e $Y = 1.0_{10}$, $3F80000_{16}$. Como $X < Y$, $X - Y$ é negativo, eliminando assim as opções B e D. A opção C também é falsa, uma vez que Y é positivo. Apenas a opção A é verdadeira, pois $\text{Exp}_X = 10000111_2 = 135_{10}$ e, portanto, $\text{Exp}_X^{\text{verd}} = 8$.

Exercício 17

a) $3{,}3895314 \times 10^{38}$

b) $1{,}17549435 \times 10^{-38}$

c) A representação de $2{,}6 \times 10^6$ é $\texttt{4A1E}_{16}$ e a representação de $3{,}1 \times 10^4$ é $\texttt{46F2}_{16}$.

O resultado da multiplicação neste formato é $5195_{16} \rightarrow 79993765888$, mas o resultado do cálculo exato seria 80600000000. Assim, o erro absoluto é 606234112 e o erro relativo é 0.752%. Embora esta seja uma representação de tamanho reduzido, que induz erros em operações aritméticas simples, o erro relativo é ainda assim pequeno.

Exercício 18

a) 2^{23} valores; todos os intervalos têm o mesmo tamanho: 2^{-23}.

b) 2^{22} valores; todos os intervalos têm o mesmo tamanho: 2^{-22}.

Exercício 19

a) x: 00000001_2, $x = \frac{1}{1024} = 0{,}0009765625_{10}$; y: 01110111_2, $y = 120_{10}$; z: 00001000_2, $z = \frac{1}{128} = 0{,}0078125_{10}$.

b) W: 01000000_2, correspondente a $1{,}0_{10}$; erro de representação $|1{,}01 - 1{,}00| = 0{,}1$ (decimal).

c) Resultado em «binary8p4»: 01001000_2, correspondendo a $2{,}0_{10}$; erro é $|2{,}02 - 2{,}00| = 0{,}02$ (decimal).

3 Circuitos Combinatórios

Exercício 15

a) $F(A, B, C, D, E) = A \cdot B + \overline{C} \cdot E$

b) $F(A, B, C) = \overline{A} \cdot \overline{C}$

c) $G(A, B, C) = A \cdot \overline{B} + A \cdot \overline{C}$

d) $F(A, B, C, D) = \overline{B} \cdot C + A \cdot C \cdot \overline{D}$

e) $F(W, X, Y, Z) = W + X \cdot \overline{Z} + X \cdot \overline{Y}$

f) $F(A, B, C, D) = \overline{A} \cdot C + C \cdot D$

Exercício 16

a)

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

b)

X	Y	Z	G
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

c)

W	X	Y	Z	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Exercício 17

a) Soma de produtos: $F(X, Y, Z) = X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} + Y \cdot Z$ ou $F(X, Y, Z) = X \cdot \ Y + \overline{Y} \cdot \overline{Z} + Y \cdot Z$.

Produto de somas: $F(X, Y, Z) = (Y + \overline{Z}) \cdot (X + \overline{Y} + Z)$.

b) Soma de produtos: $F(W, X, Y, Z) = W \cdot X \cdot \overline{Y} \cdot Z + W \cdot \overline{X} \cdot Y + W \cdot \overline{X} \cdot \overline{Z} + \overline{W} \cdot X \cdot Y + \overline{W} \cdot X \cdot \overline{Z} + Y \cdot \overline{Z}$.

Produto de somas: $F(W, X, Y, Z) = (W + X + Y) \cdot (W + X + \overline{Z}) \cdot (W + Y + \overline{Z}) \cdot (X + Y + \overline{Z}) \cdot (\overline{W} + \overline{X} + Y + Z) \cdot (\overline{W} + \overline{X} + \overline{Y} + \overline{Z})$.

Exercício 18

a) $F(X, Y, Z) = X \cdot Y + \overline{Y} \cdot Z$.

b) $F(X, Y, Z) = \overline{X} \cdot \overline{Z} + X \cdot Z$.

c) $F(W, X, Y, Z) = \overline{W} \cdot Y \cdot \overline{Z} + X \cdot \overline{Z} + W \cdot Z$.

d) $F(W, X, Y, Z) = \overline{Z} + \overline{W} \cdot X + W \cdot \overline{X} \cdot Y$.

Exercício 19

a) $F(X, Y, Z) = (Y + Z) \cdot (X + \overline{Y})$.

b) $F(X, Y, Z) = (X + \overline{Z}) \cdot (\overline{X} + Z)$.

c) $F(W, X, Y, Z) = (W + X + Y) \cdot (W + \overline{Z}) \cdot (\overline{W} + X + Z)$.

d) $F(W, X, Y, Z) = (W + X + \overline{Z}) \cdot (\overline{W} + \overline{X} + \overline{Z}) \cdot (X + Y + \overline{Z})$.

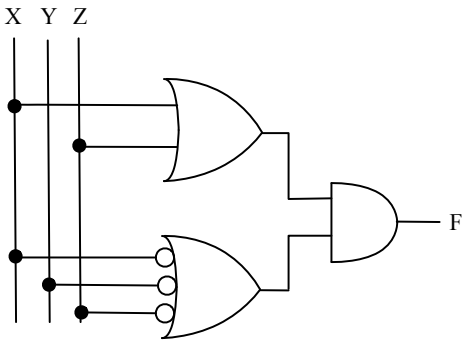
Exercício 20

a)

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

b) $F = (X + Z)\cdot(\overline{X} + \overline{Y} + \overline{Z})$.

c)



Exercício 21

a)
$$\begin{aligned} Y &= A + B + (B.C) \\ &= A + (B.1) + (B.C) \\ &= A + B.(1 + C) \\ &= A + B \end{aligned}$$

b)
$$\begin{aligned} Z &= \overline{(A + B).(B.C)} \\ &= \overline{(A + B)} + \overline{(B.C)} \\ &= \overline{A}.\overline{B} + \overline{B} + \overline{C} \\ &= (\overline{A} + 1).\overline{B} + \overline{C} \\ &= \overline{B} + \overline{C} \end{aligned}$$

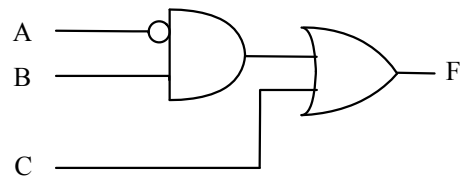
Exercício 22

a) $F = \overline{A}\cdot B + \overline{C}$

b)

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

c)



Exercício 23

a)

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

$F = (X + Y + Z)\cdot\overline{X}$

b)

A	B	C	D	G
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$G = A\cdot B + \overline{A}\cdot\overline{B}\cdot C\cdot\overline{D} + \overline{A}\cdot\overline{B}\cdot\overline{C}\cdot D + \overline{A}\cdot B\cdot\overline{C}\cdot\overline{D} + \overline{A}\cdot B\cdot C\cdot\overline{D} + A\cdot\overline{B}\cdot C\cdot\overline{D}$

Exercício 24

A	B	C	$A\cdot\overline{C}$	$A\cdot\overline{B}\cdot C$	$\overline{A}\cdot B\cdot C$	G
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	0

$G = (A + B)\cdot(A + C)\cdot(\overline{A} + \overline{B} + \overline{C})$

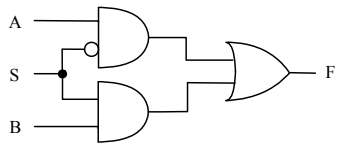
Exercício 25

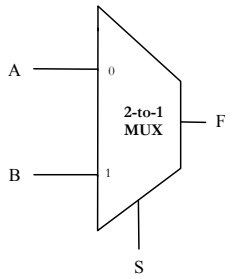
a)

A	B	S	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

b) $F(A, B, S) = A \cdot \overline{S} + B \cdot S$.

c)





d) $G(X, Y, S) = X \cdot Y \cdot + X \cdot S + Y \cdot S$.

e) Nesta alínea, pode deduzir a expressão da função implementada e simplificá-la algebricamente. Em alternativa, construa a tabela de verdade a partir da expressão obtida para o circuito e da obtida na alínea anterior.

Exercício 26

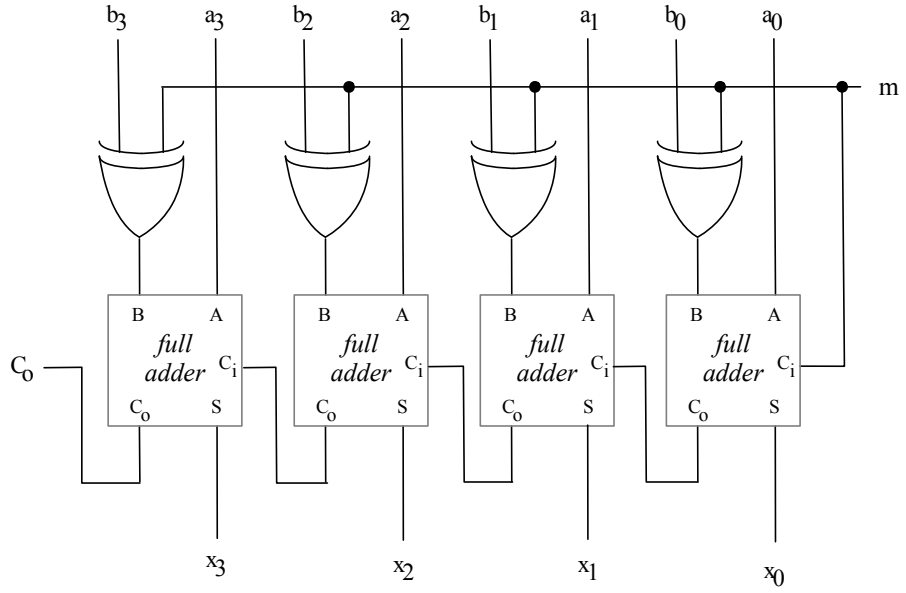
a)

a_3	a_2	a_1	a_0	S
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

b) $S = a_3 \cdot \overline{a_1} \cdot \overline{a_0}$

Exercício 27

Podemos simplesmente usar portas XOR para complementar a representação de B, controladas diretamente pela entrada m. Ou seja, quando $m = 1$ complementamos B e também somamos 1, injetando diretamente m como *carry in* do full-adder do LSB. Desta forma, sempre que $m = 1$, o circuito calcula efetivamente $A - B$. Quando $m = 0$, não há complemento de B nem *carry in* e, como tal, o circuito comporta-se como um simples somador bit a bit. O circuito lógico abaixo ilustra este projeto.



Exercício 28

a)

a_1	a_0	b_1	b_0	Maior
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

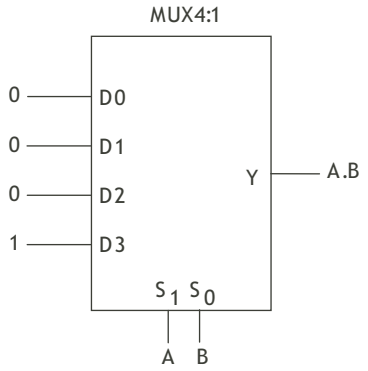
b) Maior = $a_1 \cdot \overline{b_1} + a_0 \cdot \overline{b_1} \cdot \overline{b_0} + a_1 \cdot a_0 \cdot \overline{b_0}$.

Exercício 29

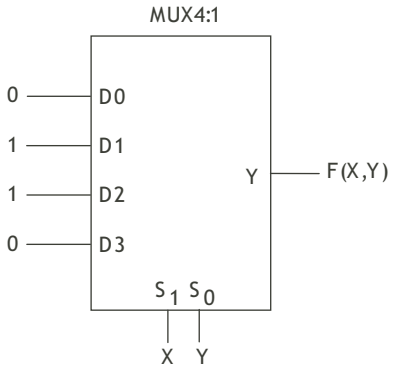
Observamos que os dois multiplexadores escolhem efetivamente entre A e B, pois a sua entrada de seleção é comum e as entradas de maior peso estão ligadas aos bits de maior peso tanto de A como de B. A seleção entre A e B na saída é feita através da saída do comparador, de tal forma que, quando $A > B = 1$, as entradas dos multiplexadores ligadas a A são ligadas à saída dos multiplexadores e, portanto, à saída do circuito. Quando $A > B = 0$, a saída do circuito global passa a ser igual à entrada B. O circuito produz assim o máximo de A e B.

Exercício 30

a)



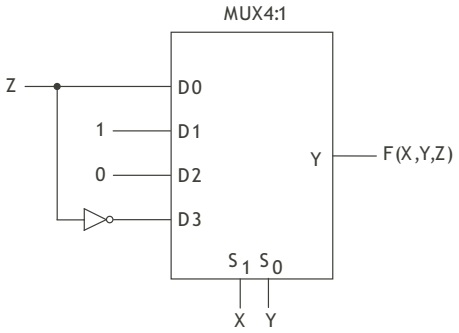
b)



c) Como exemplo, e dada a função F definida como:

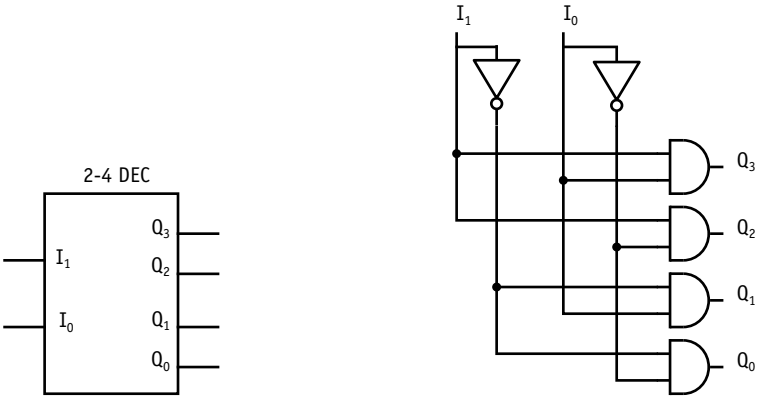
X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

pode deduzir-se o circuito

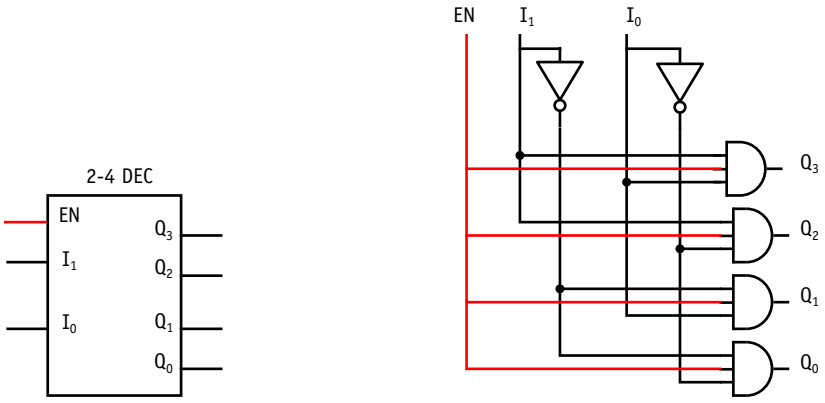


Exercício 31

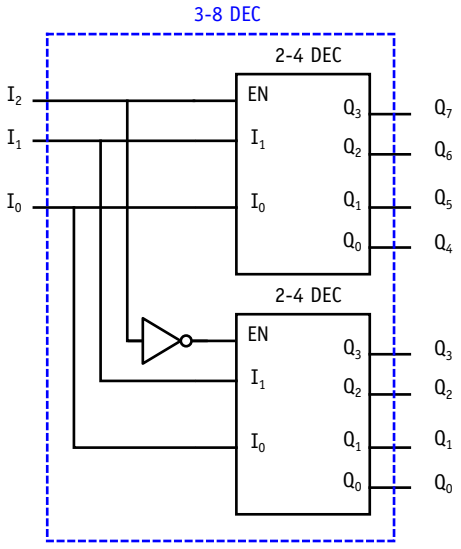
a) O circuito abaixo implementa um descodificador binário de 2 para 4.



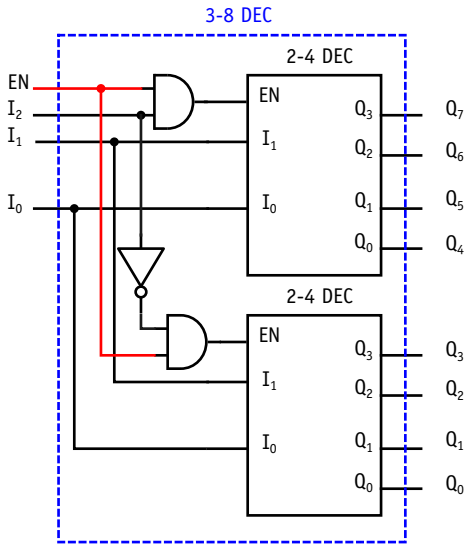
b) O circuito abaixo implementa um descodificador binário de 2 para 4 com *enable*.



c) O circuito abaixo implementa um descodificador binário de 3 para 8 sem *enable*. Note que a numeração das saídas depende da forma como a entrada I_2 está ligada.



d) O circuito abaixo implementa um descodificador binário de 3 para 8 com *enable*.



Exercício 32

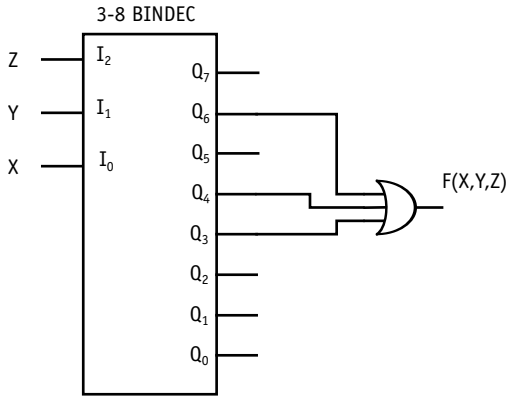
a) $Q_0 = \overline{I_2} \cdot \overline{I_1} \cdot \overline{I_0}$ e $Q_3 = \overline{I_2} \cdot I_1 \cdot I_0$.

b) Ligue X à entrada I_0, Y à entrada I_1 e Z à entrada I_2.

Para $X = 1, Y = 1, Z = 0$, apenas $Q_3 = 1$.

Com $X = 0, Z = 1$ podemos ter mais do que uma saída com o valor lógico 1, dependendo do valor de Y, nomeadamente Q_4 e Q_6. F deve ser 1 apenas nestes casos. Assim, $F(X, Y, Z) = Q_3 + Q_4 + Q_6$.

O circuito resultante é o seguinte:



Exercício 33

As saídas só estão especificadas quando $A \geq B$; para $A < B$, os valores são indiferentes (*don't care*).

a) A tabela de verdade é a seguinte. O símbolo x representa uma condição indiferente.

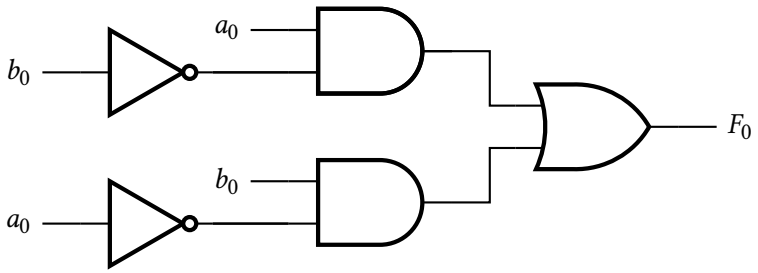
b_1	b_0	a_1	a_0	A	B	c_1	c_0
0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	1
0	0	1	0	2	0	1	0
0	0	1	1	3	0	1	1
0	1	0	0	0	1	x	x
0	1	0	1	1	1	0	0
0	1	1	0	2	1	0	1
0	1	1	1	3	1	1	0
1	0	0	0	0	2	x	x
1	0	0	1	1	2	x	x
1	0	1	0	2	2	0	0
1	0	1	1	3	2	0	1
1	1	0	0	0	3	x	x
1	1	0	1	1	3	x	x
1	1	1	0	2	3	x	x
1	1	1	1	3	3	0	0

b) Aproveitando as saídas não especificas e usando mapas de Karnaugh, obtêm-se as seguintes expressões mínimas:

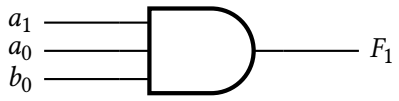
$$\begin{aligned} F_0 &= \overline{b_0} a_0 + b_0 \overline{a_0} \\ F_1 &= a_1 a_0 b_0 \end{aligned}$$

Estas expressões produzem os valores indicados na tabela para as combinações em que $A \geq B$. Para as restantes combinações, os valores da saída foram escolhidos de modo a simplificar os circuitos.

c) O circuito para F_0:



O circuito para F_1:



d) Para F_0, o percurso mais lento atravessa um inversor, uma porta AND de duas entradas e uma porta OR de duas entradas. Assim,

$$t_{F_0,\max} = 10 + 2 \times 10 + 2 \times 10 = 50\,\text{ns}.$$

Para F_1, o atraso da porta AND de três entradas é

$$t_{F_1,\max} = 3 \times 10 = 30\,\text{ns}.$$

O tempo de propagação máximo do módulo é o maior dos dois valores:

$$t_{\max} = 50\,\text{ns}.$$

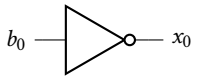
Exercício 34

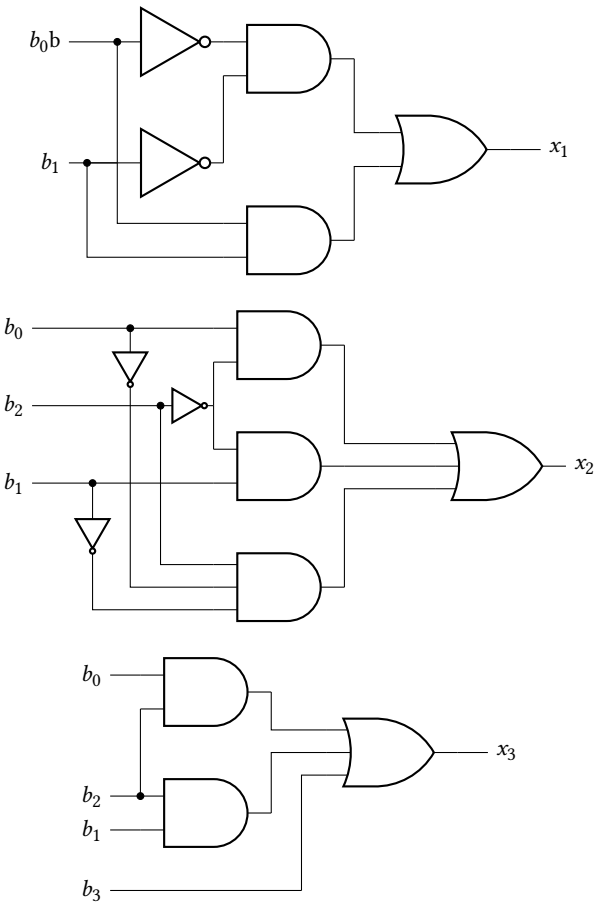
Constrói-se a tabela de verdade do codificador. As entradas de 1010 a 1111 não correspondem a algarismos decimais; por isso, as saídas nessas linhas são indiferentes, assinaladas por x.

Entradas				Saídas			
b_3	b_2	b_1	b_0	x_3	x_2	x_1	x_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

Simplificar as funções com mapas de Karnaugh aproveitando as linhas não especificadas permite obter:

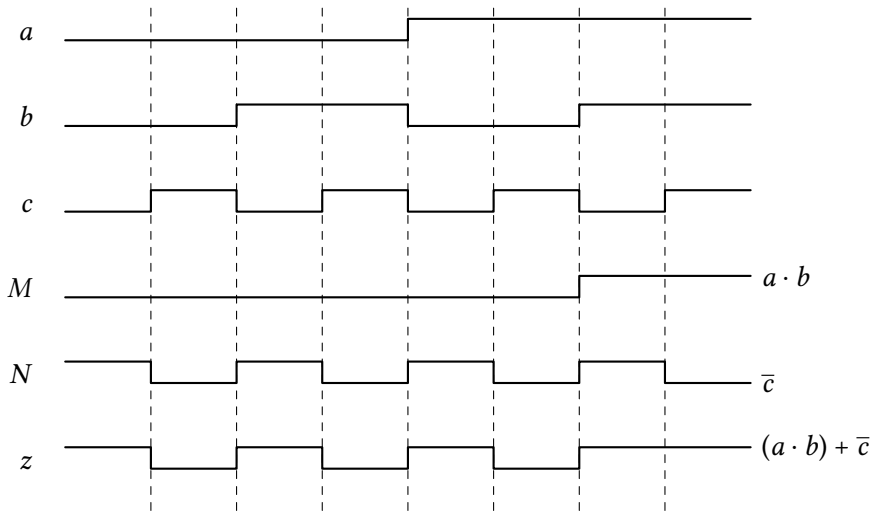
$$\begin{aligned}
x_0 &= \overline{b_0},\\
x_1 &= \overline{b_1}\,\overline{b_0} + b_1 b_0,\\
x_2 &= \overline{b_2} b_0 + \overline{b_2} b_1 + b_2 \overline{b_1}\,\overline{b_0},\\
x_3 &= b_3 + b_2 b_1 + b_2 b_0.
\end{aligned}$$



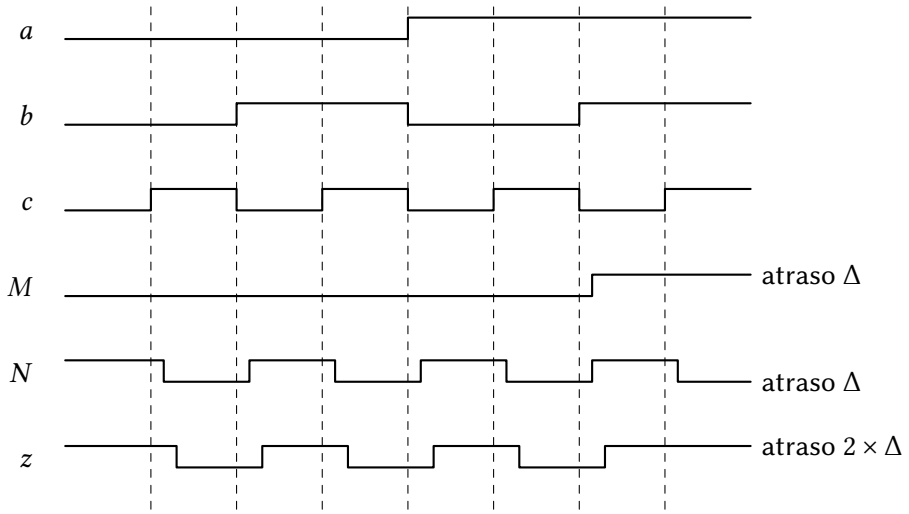


Exercício 35

a) Portas lógicas com tempo de propagação (atraso) nulo:

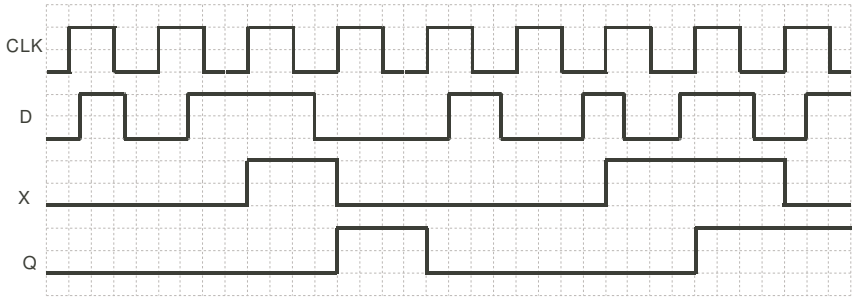


b) Portas lógicas com tempo de propagação fixo e pequeno Δ:



4 Circuitos Sequenciais

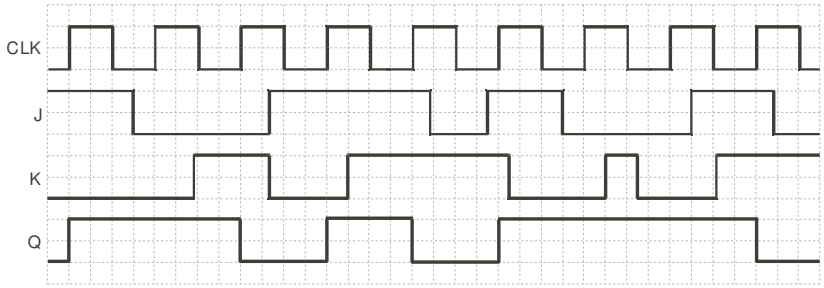
Exercício 8



Exercício 9

a) $D(J, K, Q) = J \cdot \overline{Q} + \overline{K} \cdot Q$

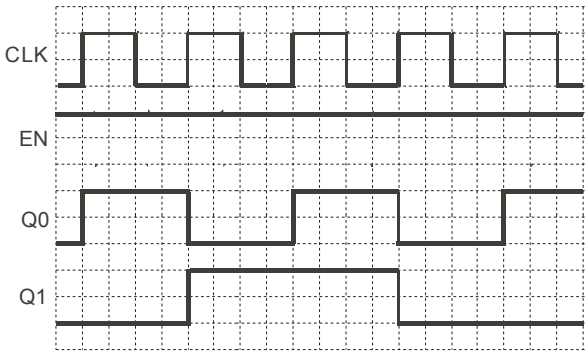
b)



Exercício 10

a) $D_0 = EN \oplus Q_0$ e $D_1 = EN \cdot Q_0 \oplus Q_1$

b) Ver as formas de onda abaixo.



c) Estado `00` (estado inicial).

Exercício 11

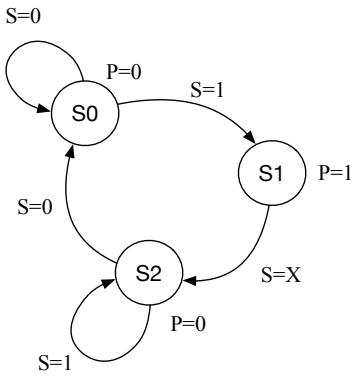
a) 0101001101000010.

b) A saída T assinala as mudanças de valor lógico da entrada X.

c) As saídas do contador indicam (em binário e até um valor máximo de 15) o número dessas transições.

Exercício 12

O circuito sequencial pretendido implementará uma máquina de estados que espera que o sinal de entrada S esteja ativo (*i.e.*, no nível lógico 1), após o que gera um único impulso na saída P e espera depois que a entrada S fique inativa. A máquina de estados finitos (FSM) correspondente é apresentada abaixo.



Como se pode ver, temos um total de 3 estados, para os quais precisamos de um mínimo de 2 *flip-flops*. Assumimos ainda que os *flip-flops* são do tipo D, com *reset* síncrono e ativados pelo flanco ascendente.

A parte não trivial deste problema é a *síntese* da máquina de estados finitos, em termos da definição das expressões lógicas das entradas D dos *flip-flops* em função do seu estado e das entradas.

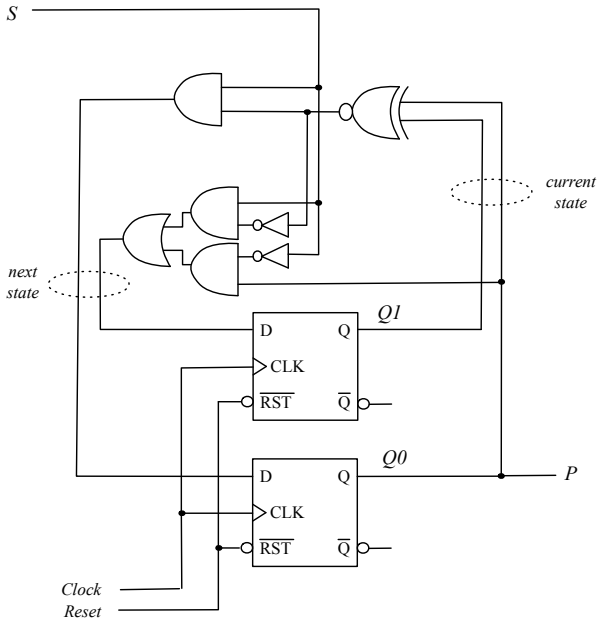
Começamos por fazer uma atribuição dos estados da FSM em termos dos valores das saídas dos 2 *flip-flops*, respetivamente Q_1 e Q_0. Uma atribuição possível é a seguinte: $S0 = 00$, $S1 = 01$ e $S2 = 10$. A combinação de valores 11 nunca é, portanto, usada, o que nos permite algum grau de liberdade em

termos de simplificação lógica.

O passo seguinte é observar as transições da FSM em relação ao estado atual da FSM e ao estado das entradas e obter aquilo a que se chama uma *tabela de excitação*. Esta tabela, apresentada abaixo, relaciona o *estado atual* e o valor das entradas com o valor dos *flip-flops* que constituirá o *estado seguinte* da FSM.

input	*current state*		*next state*		*output*
S	Q_1	Q_0	D_1	D_0	P
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	X	X	X
1	0	0	0	1	0
1	0	1	1	0	1
1	1	0	1	0	0
1	1	1	X	X	X

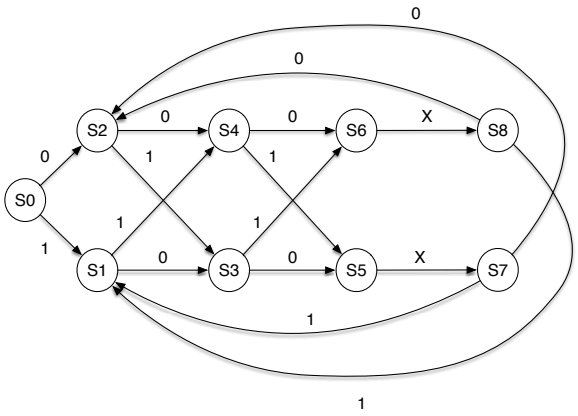
Com base nesta tabela, podemos obter (e simplificar logicamente) as expressões que definem os sinais D_0 e D_1, respetivamente, como $D_0 = (\overline{Q_1 \oplus Q_0}) \cdot S$ e $D_1 = ((Q_1 \oplus Q_0) \cdot S) + (\overline{S} \cdot Q_0)$. Por fim, o sinal de saída P pode simplesmente ser expresso como $P = Q_0$, pois corresponde a a FSM estar no estado S_1, e podemos ignorar com segurança a codificação de estado 11.



Exercício 13

A ideia geral da solução é a FSM "contar"o número de 0s e 1s observados à medida que os bits de entrada vão sendo revelados. Como não estamos realmente interessados no número de 0s nem no número de 1s, não precisamos de contar, mas apenas de acompanhar a paridade do número de 1s. Se, no final das entradas, isto é, depois de revelados os 3 bits de entrada, estivermos num estado que corresponde a um número ímpar de 1s, passamos para um estado que emite um 1 na saída. Caso contrário, passamos para um estado que emite um 0 na saída.

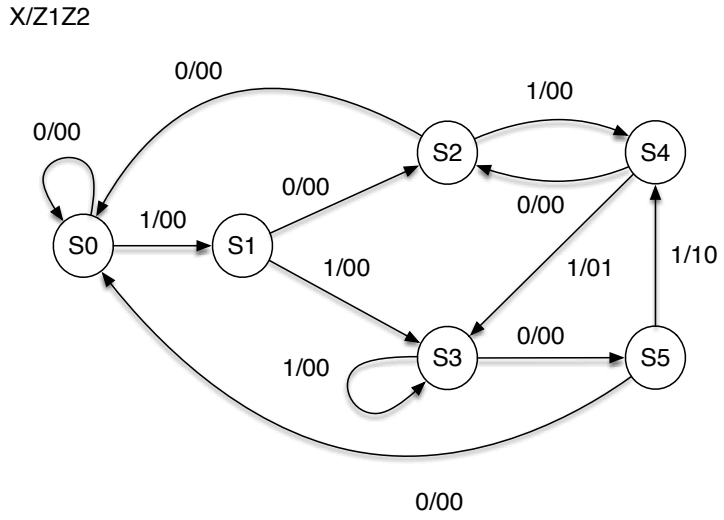
O diagrama de estados abaixo ilustra uma tal FSM com estado inicial $S0$. À direita da figura apresenta-se a função de saída, onde para cada estado se define o valor da saída.



State	Output
S0	0
S1	0
S2	0
S3	0
S4	0
S5	0
S6	0
S7	1
S8	0

Exercício 14

Neste caso particular, implementar a máquina sequencial como uma máquina de Mealy é muito vantajoso em termos do número de estados e, portanto, da implementação final em hardware. A figura abaixo apresenta uma possível realização da FSM como máquina de Mealy.



Exercício 15

a) 4 bytes.

b) O descodificador binário.

c) O conteúdo do registo com endereço 001 passará a ser 1111.

d) O sinal *enable* habilita a operação de escrita em qualquer registo. O sinal *reset* limpa o conteúdo de todos os registos.

e) O descodificador identifica (e ativa) um registo específico a usar na operação de escrita. O multiplexador, funcionando em conjunto com o descodificador, encaminha as saídas do registo endereçado para a saída do circuito.

f) Seria necessário incluir um segundo multiplexador de 8 para 1 (com sinais de 4 bits de largura), bem como um endereçamento adicional de 3 bits. Os únicos sinais partilhados seriam o barramento de dados de entrada, que estaria ligado simultaneamente às entradas de todos os registos. Note que teríamos dois barramentos de endereços, um para cada saída de leitura.

5 Organização de Memória

Exercício 5

a) [0x4000; 0x5FFF]

Descodificação parcial, pois há um bit (A_{12}) que não é explicitamente usado na descodificação que ativa o sinal CS da RAM.

b) O último endereço da ROM é 0xCFFF.

c) A lógica necessária é dada por $CS_{\text{ROM}} = A_{15} \cdot A_{14} \overline{A_{13}} \cdot \overline{A_{12}} \cdot A_{11}$ (porta lógica AND com duas das suas entradas complementadas).

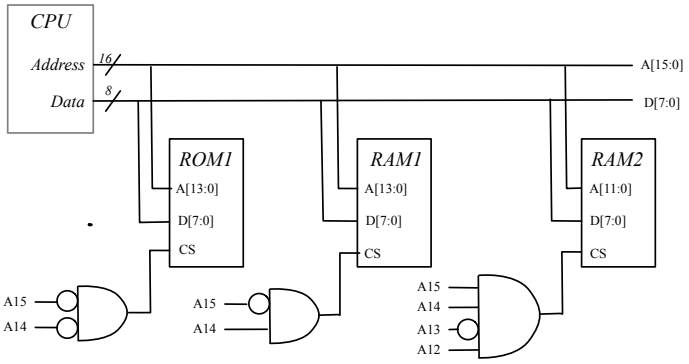
Exercício 6

a) ROM1: 16 KiB

RAM1: 16 KiB RAM2: 4 KiB

b) $CS_{\text{ROM1}} = \overline{A_{15}} \cdot \overline{A_{14}}$
$CS_{\text{RAM1}} = \overline{A_{15}} \cdot A_{14}$
$CS_{\text{RAM2}} = A_{15} \cdot A_{14} \cdot \overline{A_{13}} \cdot A_{12}$

c)



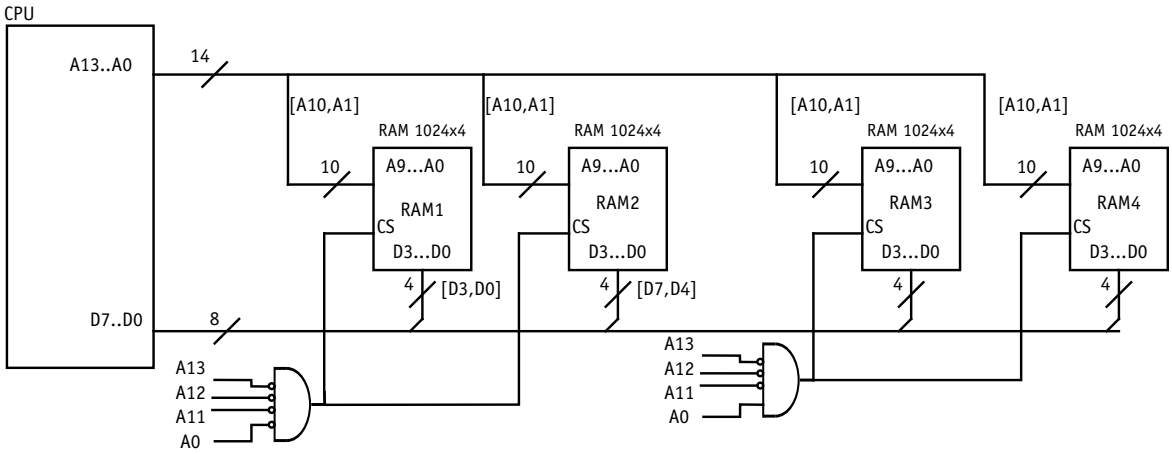
Exercício 7

Como cada dispositivo de memória só pode armazenar quantidades de dados de 4 bits, precisamos de ter 2 dispositivos de memória em paralelo (i.e., com lógica de endereçamento e seleção idêntica) para formar quantidades de dados de 8 bits. Um dispositivo armazena os bits `D[3:0]`, enquanto o segundo armazena os bits `D[7:4]`.

Assim, precisamos de 4 dispositivos RAM divididos em 2 grupos. Um grupo de 2 dispositivos armazena os dados correspondentes aos endereços pares entre 0 e 2047 (0, 2, 4, ..., 2046), enquanto o segundo grupo armazena os dados correspondentes aos endereços ímpares entre 1 e 2047 (1, 3, 5, ..., 2047).

O bit de endereço de menor peso, o bit `A0`, será usado para selecionar diretamente entre os dois grupos de dispositivos RAM.

a) A figura abaixo apresenta uma solução possível. Os dispositivos de memória `RAM1` e `RAM2` armazenam os dados correspondentes aos endereços pares.



b) O endereço é 250_{H}=00 0010 0101 0000_2. O bit menos significativo é usado para selecionar o grupo: neste caso, é '0', o que significa que é usado o grupo com os dispositivos de memória `RAM1` e `RAM2`. O endereço dentro do grupo é determinado pelos restantes bits:

$$0000100101000_2 = 128_{\text{H}}.$$

Assim, os dados serão armazenados na posição 128_{H} de ambos os dispositivos. Especificamente, o nibble `A` (bits `[D3:D0]`) é armazenado na `RAM1` e o nibble `9` (bits `D[7:4]`) é armazenado na `RAM2`.

6 Linguagem Assembly

Exercício 7

É necessário definir uma atribuição arbitrária de variáveis a registos.

a) Atribuição: `f` → `x5`, `g` → `x6`, `j` → `x7`.

```
addi      x5, x7, 2
add       x5, x6, x5
```

b) Atribuição: `a` → `x5`, `b` → `x6`, `d` → `x7`, `f` → `x28`, `k` → `x29`.

```
add       x29, x5, x6
sub       x29, x29, x28
add       x29, x29, x7
addi      x29, x29, -30
```

c) Atribuição: `f` → `x5`, `g` → `x6`, `h` → `x7`, `A` → `x28`. O registo `x8` é usado como auxiliar.

```
add       x5, x6, x7
lw        x8, 16(x28)
add       x5, x5, x8
```

d) Atribuição: `f` → `x5`, `g` → `x7`, `A` → `x28`, `B` → `x29`.

```
lw        x5, 40(x29)
slli      x5, x5, 2
add       x5, x5, x28
lw        x5, 0(x5)
sub       x5, x7, x5
```

e) Atribuição: `f` → `x5`, `g` → `x6`, `h` → `x7`, `k` → `x28`, `A` → `x29`.

```
addi      x30, x7, 9
slli      x30, x30, 2
add       x30, x29, x30
lw        x5, 0(x30)
add       x5, x5, x28
sub       x5, x5, x6
```

f) Atribuição: `f` → `x5`, `g` → `x6`, `A` → `x28`, `B` → `x29`.

```
lw      x7, 8(x29)
addi    x7, x7, 4
slli    x7, x7, 2
add     x7, x28, x7
lw      x7, 0(x7)
sub     x5, x6, x7
```

Exercício 8

a) Atribuição: `f` → `x5`, `g` → `x6`, `h` → `x7`, `i` → `x28`, `j` → `x29`

A expressão correspondente: `f = f + g + h + i + j`

b) Atribuição: `f` → `x5`, `A` → `x6`

A expressão correspondente é: `f = A[1]`

c) Atribuição: `f` → `x5`, `g` → `x6`, `A` → `x7`

A expressão correspondente é: `f = A[2g-3]`

Exercício 9

a) `0xDDFFDDFF`

b) `0x55555550`

c) `0x000000AA`

Exercício 10

a)
```
        add     x5, x6, x0
        bge     x6, x0, pos
        sub     x5, x0, x5
pos: ...
```

b)
```
        addi    x5, x0, 0
        bge     x7, x6, pos
        addi    x5, x0, 1
pos: ...
```

Exercício 11

a) Em cada iteração do ciclo `L1`, o valor do registo `x28` é armazenado em memória (no endereço dado pelo conteúdo do registo `x29`). Como em cada ciclo o valor do registo `x28` é deslocado 4 bits para a esquerda, o seu valor será zero após 8 iterações e, consequentemente, o salto não é tomado, terminando a execução do fragmento.

Tendo em conta os valores iniciais, a tabela com o conteúdo da memória é:

Endereço	Conteúdo da memória
0x000007D0	0x12345678
0x000007D4	0x23456780
0x000007D8	0x34567800
0x000007DC	0x45678000
0x000007E0	0x56780000
0x000007E4	0x67800000
0x000007E8	0x78000000
0x000007EC	0x80000000

b)

endereço	instrução
0x00400000	0x01CEA023
0x00400004	0x004E1E13
0x00400008	0x004E8E93
0x0040000C	0xFE0E1AE3

Exercício 12

Uma solução possível:

```
        add     x5, x0, x0
loop:   lw      x6, 0(x29)
        beq     x6, x0, end
        addi    x29, x29, 4
        addi    x5, x5, 1
        beq     x0, x0, loop
end:    # ...
```

Exercício 13

a) Fragmento 1: o valor de x7 é 20. Fragmento 2: o valor de x7 é 200.

b) Fragmento 1: $4 \times N + 1$ instruções; Fragmento 2: $(1 + 3 \times 10 + 2) \times N = 33 \times N$ instruções.

Exercício 14

Uma solução possível:

```
        addi    x5, x0, 1       # resultado: assumir sequencias iguais
        addi    x6, x0, 10      # tamanho da sequencia
loop:   lw      x7, 0(x10)      # obter um elemento de A
        lw      x28, 0(x11)     # obter um elemento de B
        beq     x7, x28, next   # sao iguais?
        add     x5, x0, x0      # terminar se forem diferentes
        beq     x0, x0, end     # salto incondicional
next:   addi    x6, x6, -1      # atualizar contagem de elementos restantes
        beq     x6, x0, end     # todos os elementos processados
        addi    x10, x10, 4     # ainda ha elementos restantes
        addi    x11, x11, 4     # atualizar enderecos
        beq     x0, x0, loop    # salto incondicional
end:    add     x10,x0,x5       # resultado em x10
```

Exercício 15

Uma solução possível:

```
          add     x5, x0, x0    # contador de numeros impares
          addi    x6, x0, 50    # tamanho da sequencia
loop:     lw      x7, 0(x10)    # obter um elemento da sequencia
          andi    x7, x7, 1     # isolar o LSB
          beq     x7, x0, L1    # o numero e par
          addi    x5, x5, 1     # o numero e impar
L1:       addi    x6, x6, -1    # atualizar contagem de elementos restantes
          beq     x6, x0, end   # todos os elementos foram processados
          addi    x10, x10, 4   # atualizar endereco do elemento
          beq     x0, x0,loop   # salto incondicional
end:      add     x10, x0, x5   # resultado em x10
```

Exercício 16

a) Parâmetros:

- `a0`: endereço base da sequência
- `a1`: número de elementos

```
addelems:
        li      t0, 0           # inicializar contador a 0
L1:     beq     a1, zero, L2    # verificar se ha elementos a processar
        lw      t1, 0(a0)       # obter um elemento
        add     t0, t0, t1      # acumular
        addi    a0, a0, 4       # endereco do proximo elemento
        addi    a1, a1, -1      # ajustar numero de elementos a processar
        j       L1              # repetir (proximo elemento)
L2:     mv      a0, t0          # devolver resultado em a0
        ret
```

b) Parâmetros:

- `a0`: endereço base da sequência
- `a1`: número de elementos

```
countodd:
      li      t0, 0           # inicializar contador a 0
L21:  beq     a1, zero, L23   # verificar se ha elementos a processar
      lw      t1, 0(a0)       # obter um elemento
      andi    t1, t1, 1       # t1 fica a 1 se o numero for impar, 0 caso contrario
      add     t0, t0, t1
L22:  addi    a0, a0, 4       # endereco do proximo elemento
      addi    a1, a1, -1      # ajustar numero de elementos a processar
      j       L21             # repetir (proximo elemento)
L23:  mv      a0, t0          # devolver resultado em a0
      ret
```

c) Parâmetros:

- `a0`: endereço base da sequência
- `a1`: número de elementos
- `a2`: limiar

```
# a0: endereco base, a1: numero de elementos, a2: limiar
countgeq:
        li      t0, 0
L31:    beq     a1, zero,L33
        lw      t1, 0(a0)
        blt     t1, a2, L32   # o elemento e menor que o limiar?
        addi    t0, t0, 1     # contar o elemento atual
L32:    addi    a0, a0, 4     # endereco do proximo elemento
        addi    a1, a1, -1
        j       L31
L33:    mv      a0,t0
        ret
```

d) Parâmetros:

- `a0`: endereço base da sequência 1
- `a1`: endereço base da sequência 2
- `a2`: número de elementos de cada sequência

```
# a0: endereco base seq 1, a1: endereco base seq 2,
# a2: numero de elementos
seqeq:
        addi    t0, zero, 1    # inicializar resultado a 1
L41:    beq     a2, zero, L43
        lw      t1, 0(a0)
        lw      t2, 0(a1)
        bne     t1, t2, L42    # se diferentes, terminar o varrimento
        addi    a0,a0,4
        addi    a1,a1,4
        addi    a2,a2,-1
        j       L41
L42:    mv      t0, zero       # o resultado final e 0
L43:    mv      a0,t0          # 1 ou 0
        ret
```

e) Parâmetros:

- a0: endereço base da sequência
- a1: número de elementos de cada sequência
- a2: valor limiar a
- a3: valor limiar b

```
# a0: endereco base da sequencia,
# a1: numero de elementos (assumir > 0)
# a2: valor limiar a
# a3: valor limiar b
bounds:
        mv      t0, a0
        mv      t1, a1
Ltop:   beq     t1, zero, Lret
        lw      t2, 0(t0)
        blt     t2, a2, Lcnt
        sw      a2, 0(t0)
        j       Lcnt
        bgt     t2, a3 Lcnt
        sw      a3, 0(t0)
Lcnt:   addi    t0, t0, 4
        addi    t1, t1, -1
        j       Ltop
Lret:   ret
```

Exercício 17

```
setzero:
        mv      t0, zero
L1:     beq     a1, zero, final         # parar?
        __lw__  t1, 0(__a0__)           # obter elemento
        andi    t2, t1, 1
        beq     __t2__, zero, L2
        __sw__  zero, 0(__a0__)         # colocar a zero
        add     t0, t0, t1
L2:     addi    a0, a0, ___4___         # preparar a proxima iteração
        addi    a1, __a1__, __-1__
        j       L1
final:  mv      __a0__,t0
        ret
```

Exercício 18

```
rotl:
    sll     a2, a0,  a1
    sub     a4, zero, a1
    srl     a3, a0, a4
    or      a0, a2, a3
    ret


rotr:
    srl    a2, a0, a1
    sub    a4, zero, a1
    sll    a3, a0, a4
    or     a0, a2, a3
    ret
```

Exercício 21

```
atoi:   # converter uma string
        # a0 - resultado
        # a1 - endereco inicial da sequencia
        addi    t0, zero, 10    # constante auxiliar 10
        lb      t1, 0(a1)       # obter primeiro caracter (-", "+",
                                # ou um digito)
        addi    t3, zero, '0'   # codigo ASCII de '0', util adiante
        addi    t4, zero, 9     # util adiante
        addi    t5, zero, '\0'  # codigo ASCII de NULL, util adiante
isneg:  addi    t2, zero, '-'   # codigo ASCII 0x2D
        bne     t1, t2, ispos   # comparar primeiro caracter com '-'
        addi    t2, zero, 0     # t2 fica a "0- o numero e negativo
                                # (util adiante)
        addi    a1, a1, 1       # endereco do proximo caracter
        lb      t1, 0(a1)       # obter proximo caracter
        j       loop            # iniciar o ciclo
ispos:  addi    t2, zero, '+'   # codigo ASCII 0x2B
        bne     t1, t2, loop    # comparar primeiro caracter com '+'
                                # se nao for igual a "+",
                                # entao nao e -"nem "+",
                                # logo e um numero --> ir para o ciclo
        addi    a1, a1, 1       # endereco do proximo caracter
        lb      t1, 0(a1)       # obter proximo caracter
loop:   beq     t1, t5, end     # e nulo? (i.e., fim da string)
        sub     t1, t1, t3      # subtrair o codigo ASCII do
                                # caracter '0'
        bgt     t1, t4, err     # o resultado e maior que 9
        blt     t1, zero, err   # o resultado e menor que 0
        mul     a0, a0, t0      # avancar a casa decimal
        add     a0, a0, t1      # somar digito ao resultado
        addi    a1, a1, 1       # endereco do proximo caracter
        lb      t1, 0(a1)       # obter proximo caracter
        j       loop
err:    addi    a0, zero, -1    # algo que nao e um numero!
        j       pos
end:    bne     t2, zero, pos   # o numero era positivo
        sub     a0, zero, a0    # era negativo, trocar o sinal
pos:    ret
```

```
    strlen:
          add    a0, zero, zero # inicializa o resultado a 0
    L0:   lb     t1, 0(a1) # obter caracter
          beqz   t1, L1
          addi   a0, a0, 1
          addi   a1, a1, 1  # incrementar endereco
          j      L0
    L1:
          ret
```

Exercício 22

```
main:
    la      t0, vector
    sw      t0, 64,(sp)
    li      a0, 2
    call    cgoto           # chamar goto calculado(2)
                            # nunca voltamos aqui...
    ebreak

cgoto:
    sw      ra, 0(sp)
    lw      t0, 64(sp)      # obter da pilha o apontador para o vetor
    addi    sp, sp, 64      # ajustar a pilha

    lw      t1, 0(t0)       # t1 e o tamanho do vetor de goto
    ble     t1, x0, Lerr    # verificar se <= 0; se sim, ir para erro
    sub     t2, t1, a0      # t2 = (tamanho - N)
    bltz    t2, Lerr        # se <= 0, ir para erro
    slli    t2, a0, 2       # multiplicar o valor por 4
    add     t2, t0, t2      # obter o endereco da etiqueta
    lw      t2, 0(t2)       # carregar a etiqueta (endereco destino)
    jalr    x0, t2, 0       # saltar para la.
Lerr:
    ebreak

# abaixo, exemplos de codigos destino (muito simples)
L0:  lw  ra, 0(sp)
    addi sp, sp, -64
    ret
L1: addi a0, zero, 0
    j Lend
L2: addi a0, zero, 1
    j Lend
L3: addi a0, zero, 0
    j Lend
Lend: ebreak
```

Exercício 23

```
main:
    la t0, vals
    sw t0, 64,(sp)
    call dyna        # chamar o acesso ao vetor
    ebreak

# assumir que o endereco de vals em memoria esta no registo a0
# assumir que a variavel de indice i esta no registo a1
# assumir que o acesso ao vetor fica no registo a2.

dyna:
    sw    ra, 0(sp)
    lw    t0, 64(sp)     # obter o apontador para o dope vector
    addi  sp, sp, 72     # ajustar a pilha

    lw    t1, 0(t0)      # t1 e o numero de dimensoes do vetor
    lw    t2, 4(t0)      # t2 e o tamanho da primeira dimensao
    addi  a1, a1, 1      # calcular i + 1
    blt   a1, x0, Lerr   # verificar se (i+1) < 0; se sim, erro
    sub   t2, t1, a1     # t2 = (tamanho - N)
    bltz  t2, Lerr       # se t2 <= 0, erro
    slli  a1, a1, 2      # multiplicar o indice por 4 (inteiros de 32 bits)
    add   t2, a0, a1     # obter o endereco do elemento
    lw    a2, 0(t2)      # carregar o elemento
    ebreak

Lerr:   nop
    ebreak
```

Exercício 24

```
hash:   addi sp, sp, 4
        sw ra, 0(sp)

        li a2, 5381     # h = 5381

top:    beq a1, x0, bot
        lbu t0, 0(a0)  # loads byte 8 bits without extending signal

        slli t1, a2, 5 # t1 = h << 5
        add a2, a2, t1 # h = h + (h << 5)
        xor a2, a2, t0 # exor ith the first char
        addi a1, a1, -1  # subtracts length
        addi a0, a0, 1  # advances to next byte
        beq x0,x0,top

bot:    mv a0, a2      #result is already in a2
        lw ra, 0(sp)
        addi sp, sp, -4
        ret
```

Exercício 25

```
multSign:
    addi  sp, sp, -16
    sw    ra, 0(sp)
    xor   a0, a0, a1
    srli  a0, a0, 31
    lw    ra, 0(sp)
    addi  sp, sp, 16
    ret
```

Exercício 26

```
checkNaN:
     addi   a1, a0, 0    # copia a0 para a1
     srli   a0, a0, 23   # coloca o expoente no sitio
     xori   a0, a0, 255
     beqz   a0, L1       # salta para L1 se o expoente for 0xFF
     addi   a0, x0, 0    # claramente nao e NaN, devolver 0
     j      L2
L1:  slli   a1, a1, 8    # verificar se a mantissa e nao nula
     srli   a1, a1, 24
     beqz   a1, L3
     addi   a0, x0, 1
     j      L2
L3:  addi   a0, a0, 0
L2:  ret
```

Exercício 27

```
    .data
    emask: .word 0x7F800000
    .text
    nextFP:
         srai   t3, a0, 31
         bne    t3, x0, L2
         addi   t3, t3, 1
    L2:  add    t0, a0, t3
         la     t5, emask
         lw     t1, 0(t5)
         and    t4, t0, t1
         srli   t4, t4, 23
         addi   t2, x0, 255
         beq    t4, t2, L1
         beq    t4, x0, L1
         add    a0, x0, t0
    L1:  ret
```

7 Organização do Processador

Exercício 5

a) `sw a0, -64(t0)`

b) `beq a1, a2, -4`

Exercício 6

Note que `PCSrc = AND(Zero, Branch)`.

a) `RegWrite=1, MemRead=1, ALUSrc=1, MemWrite=0, MemtoReg=1, PCSrc=0, ALUOp=00`

O único módulo que não realiza trabalho útil é o somador em cima, à direita.

b) `RegWrite=0, MemRead=0, ALUSrc=1, MemWrite=1, MemtoReg=X, PCSrc=0, ALUOp=00`.

Os módulos que não realizam trabalho útil são o somador em cima, à direita, e o multiplexador controlado por `MemtoReg`.

c) `RegWrite=0, MemRead=0, ALUSrc=0, MemWrite=0, MemtoReg=X, ALUOp=01`

$$\texttt{PCSrc} = \begin{cases} 0 & \text{se xx9} \neq 0 \\ 1 & \text{se xx9} = 0 \end{cases}$$

Os módulos que não realizam trabalho útil são a memória de dados e o multiplexador controlado por `MemtoReg`.

Exercício 7

a) A instrução beq.

b) A instrução sw.

Exercício 8

a) `0x0288A403`.

b) 17. Sim.

c) 8. Não. Este valor faz parte do campo imediato desta instrução em particular.

d) 8. Sim.

e) `MemRead=1, MemWrite=0`.

f) `0x00058063`; 11, Sim; 0, Sim; 0, Não; `MemRead=0, MemWrite=0`.

Exercício 9

a) Não são necessários blocos lógicos adicionais. O hardware disponível pode ser usado para implementar a instrução `addi` sem alterações ao caminho de dados. Apenas a unidade de controlo precisa de ser alterada.

b) `Branch`: 0; `MemRead`: 0; `MemToReg`: 0; `ALUOp`: 10_2; `MemWrite`: 0; `ALUSrc`: 1; `RegWrite`: 1.

Exercício 10

$0 \to 1 \to 6 \to 7 \to 0 \to 1 \to 2 \to 3 \to 4 \to 0 \to 1 \to 8$

Exercício 11

a) A partir da descrição da máquina de estados, podemos obter o número de ciclos de relógio de cada classe de instruções.

	ALU/Lógica	Salto	Load	Store
	45 %	20 %	20 %	15 %
n.º de ciclos	4	3	5	4

O número médio de ciclos de uma instrução é

$$0.45 \times 4 + 0.2 \times 3 + 0.2 \times 5 + 0.15 \times 4 = 4.$$

O tempo total de execução seria exatamente o mesmo. Os períodos de relógio seriam mais longos, mas o número de ciclos de relógio seria menor, pelo que, no final, o tempo seria exatamente o mesmo.

$$4 \times 5\,\text{ns} \times 10^{12} = 20 \times 10^{3}\,\text{s}.$$

Para a implementação de ciclo único, o tempo de execução seria:

$$20\,\text{ns} \times 10^{12} = 20 \times 10^{3}\,\text{s}.$$

b) Há 3 ciclos de relógio em que a ALU está a realizar trabalho útil, para todas as instruções. Como o número médio de ciclos de relógio por instrução é 4, obtemos para o valor pedido $\frac{3}{4} = 75\,\%$.

8 Desempenho de Processadores

Exercício 9

a) $\text{IPC}_{P_1} = 1{,}43$ (instruções/ciclo)

$\text{IPC}_{P_2} = 2{,}00$ (instruções/ciclo)

$\text{IPC}_{P_3} = 3{,}33$ (instruções/ciclo)

b) 2,1 GHz

c) 27×10^9

Exercício 10

a) P_1: t_{exec} = 1,87 ms P_2: t_{exec} = 1,00 ms

P_2 é o processador mais rápido.

b) CPI_{P_1} = 2,8 (ciclos/instrução)

CPI_{P_2} = 2,0 (ciclos/instrução)

c) P_1: $2{,}8 \times 10^6$ P_2: 2×10^6

Exercício 11

P_1: 4×10^9 (instruções/segundo) P_2: 3×10^9 (instruções/segundo)

Exercício 12

a) C é o computador mais rápido (45 s).

b) C é 1,67 vezes mais rápido do que B e 4,93 vezes mais rápido do que A.

Exercício 13

a) t_{exec} = 675 ns

b) CPI = 1,93 (ciclos/instrução)

c) A execução fica 1,23 vezes mais rápida; CPI = 1,69.

Exercício 14

88 %

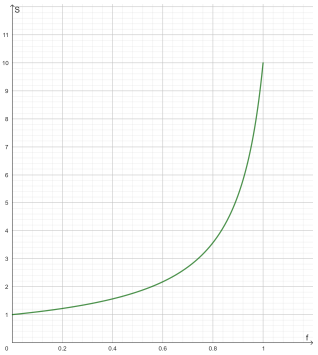
Exercício 15

a) 1,67 b) 5

Exercício 16

a) Desenhar o gráfico da função $S(f)$ para $0 \leq f \leq 1$:

$$S(f) = \frac{1}{\frac{f}{10} + (1 - f)} = \frac{10}{10 - 9 \times f}$$



b) $f = \frac{5}{9} \approx 0{,}56 = 56\,\%$

c) Percentagem de tempo gasto em modo vetorial após as modificações: $\frac{1}{9} \approx 11\,\%$

d) Ganho máximo: 10; para obter um ganho de 5, temos de ter $f = \frac{8}{9} \approx 88{,}9\,\%$.

e) Duplicando o desempenho da unidade vetorial: $S \approx 2{,}99$. Sem alterar a unidade vetorial, o mesmo ganho seria obtido se $f \approx 0{,}74 = 74\,\%$. Provavelmente é mais fácil passar de 70 % para 74 % do que duplicar o desempenho da unidade vetorial.

Exercício 17

A nova versão atinge um ganho de 2.